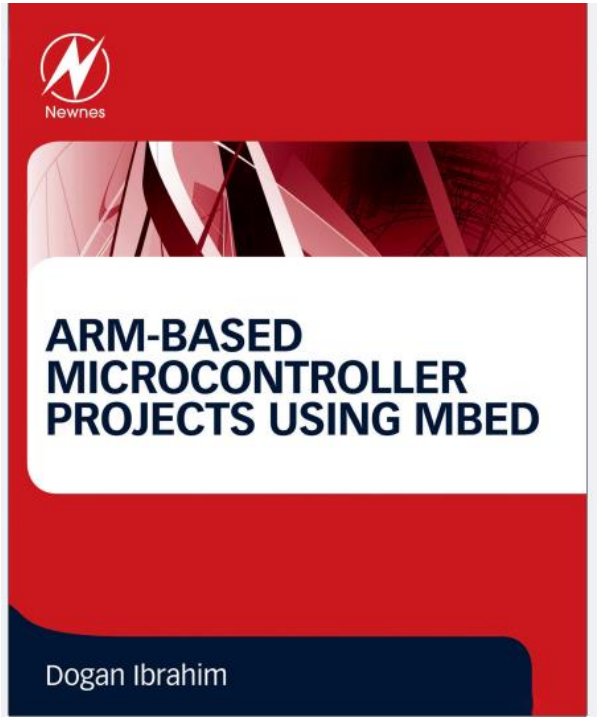
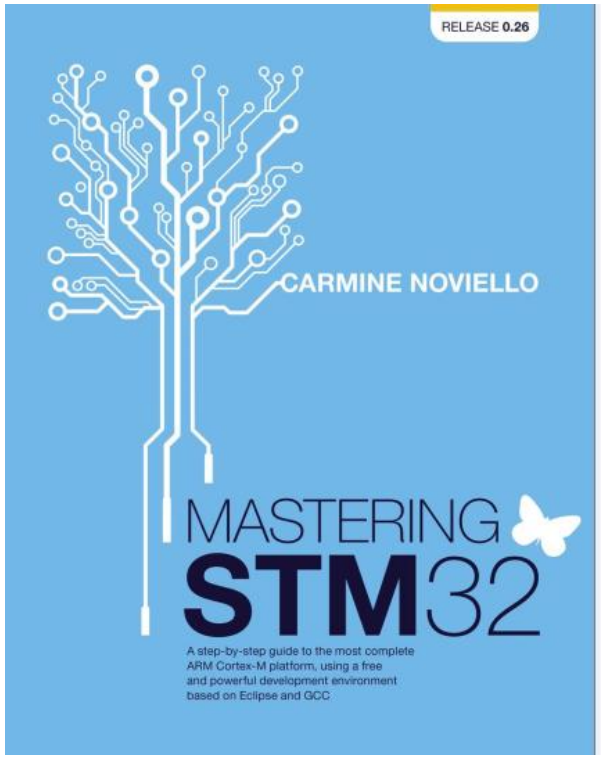
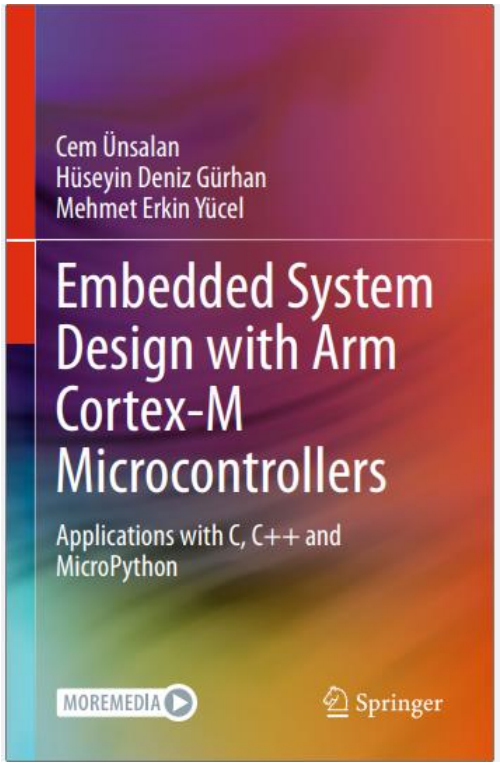
ee2004

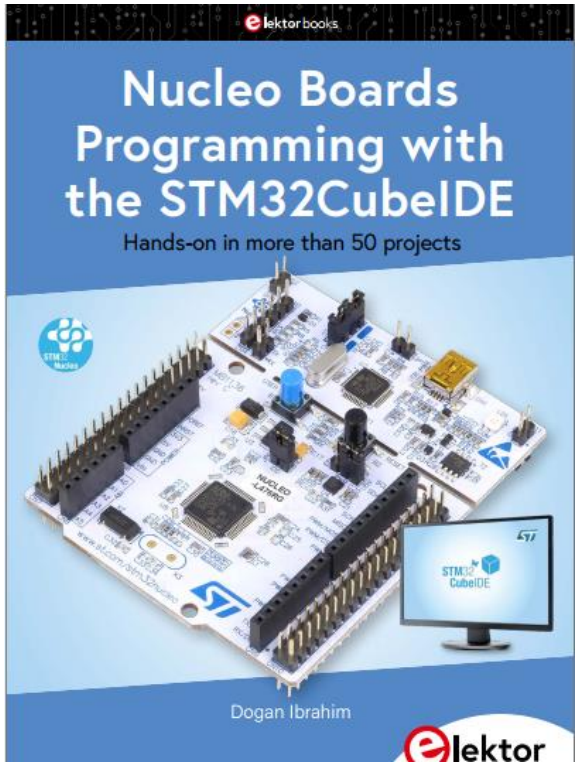
Week1,2

textbooks


Newnes
ARM-BASED MICROCONTROLLER PROJECTS USING MBED
Dogan Ibrahim


RELEASE 0.26
CARMINE NOVIELLO
MASTERING STM32
A step-by-step guide to the most complete ARM Cortex-M platform, using a free and powerful development environment based on Eclipse and GCC


Cem Ünsalan
Hüseyin Deniz Gürhan
Mehmet Erkin Yücel
Embedded System Design with Arm Cortex-M Microcontrollers
Applications with C, C++ and MicroPython
MOREMEDIA
Springer

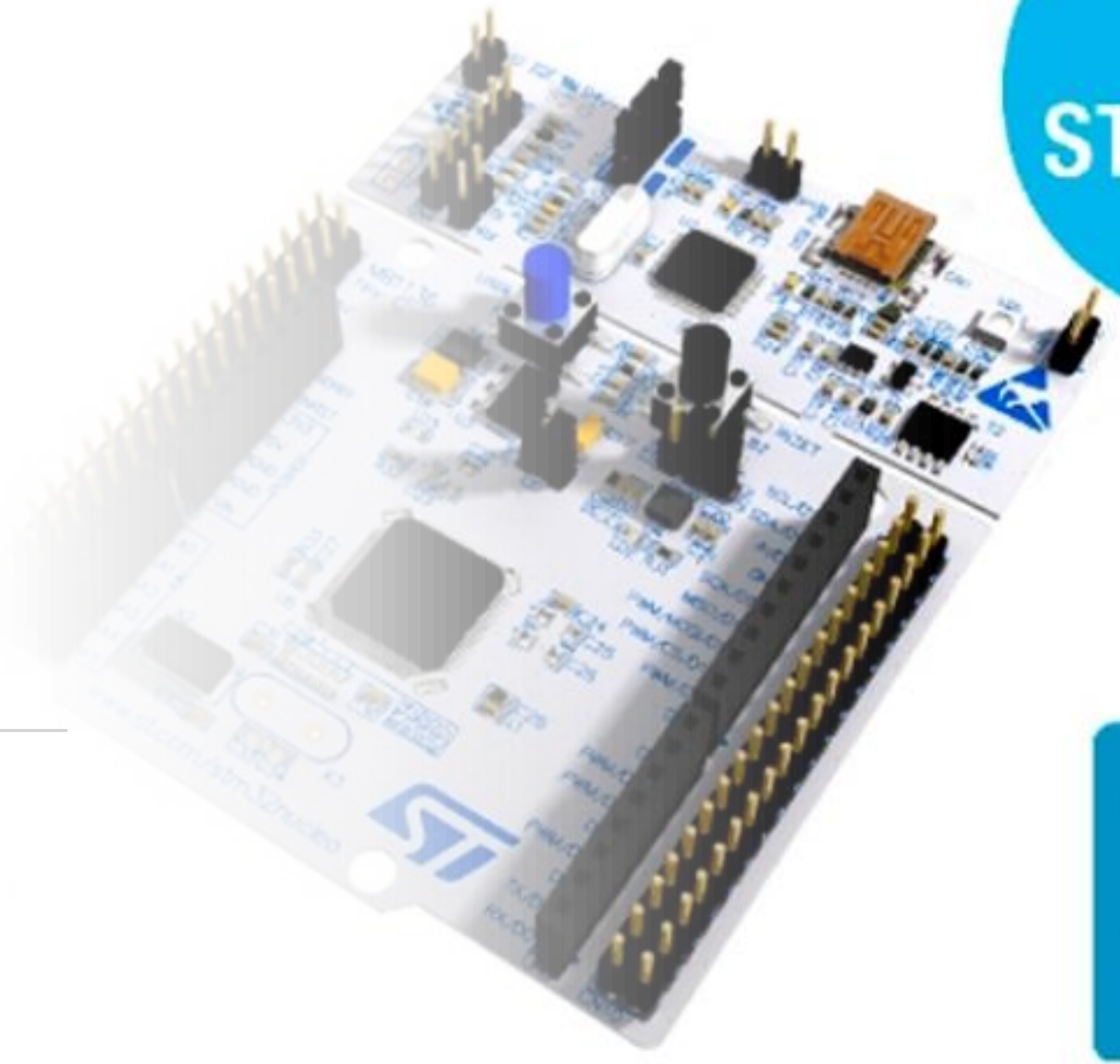

elektor books
Nucleo Boards Programming with the STM32CubeIDE
Hands-on in more than 50 projects
Dogan Ibrahim
elektor

software tools

- VisUAL2: ARM Simulator
- Arm Mbed OS Simulator
- Keil Studio Cloud / Mbed OS
- STM32CubeIDE
- Arduino IDE

board

Stm32 Nucleo



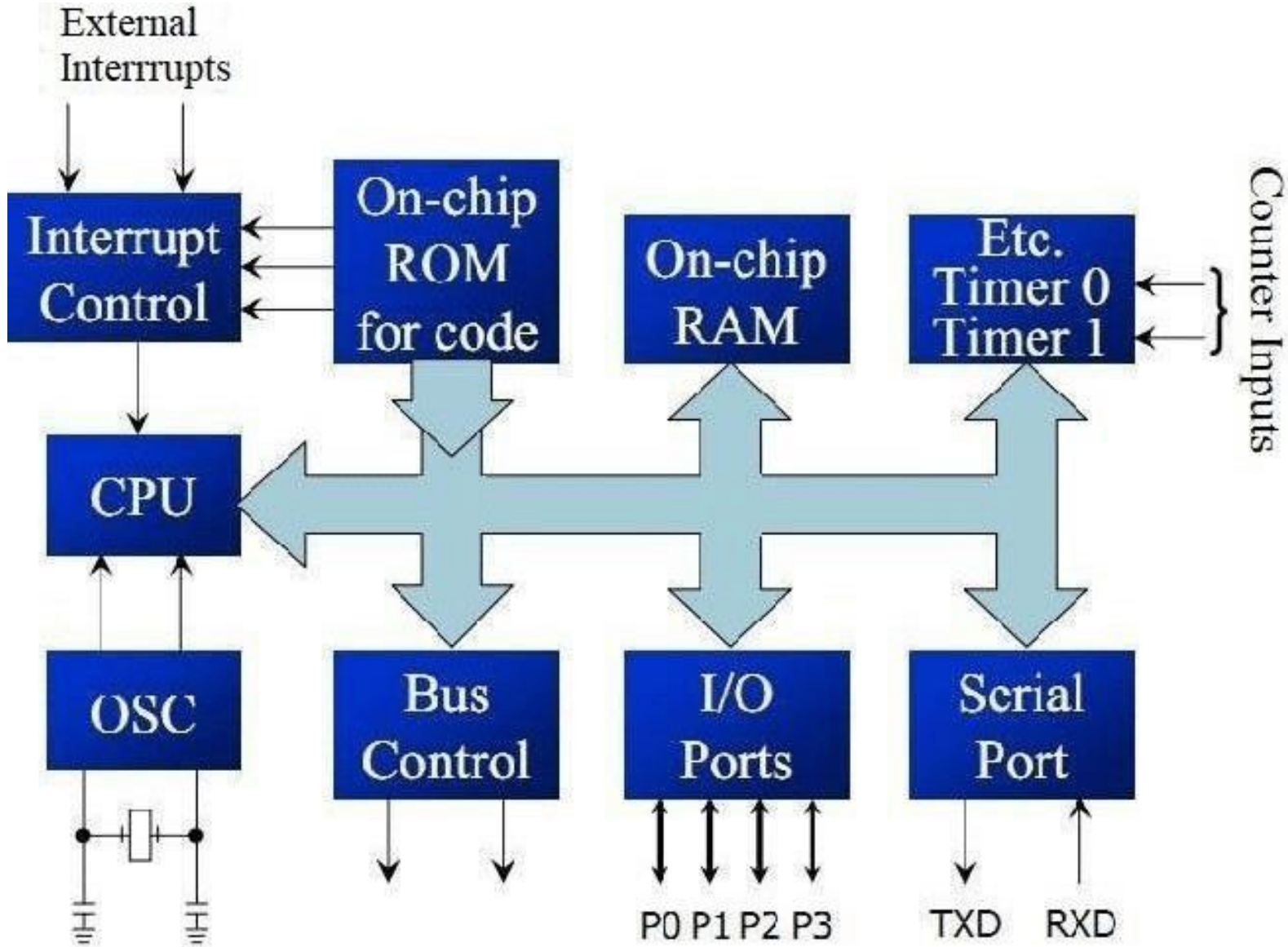
Topics

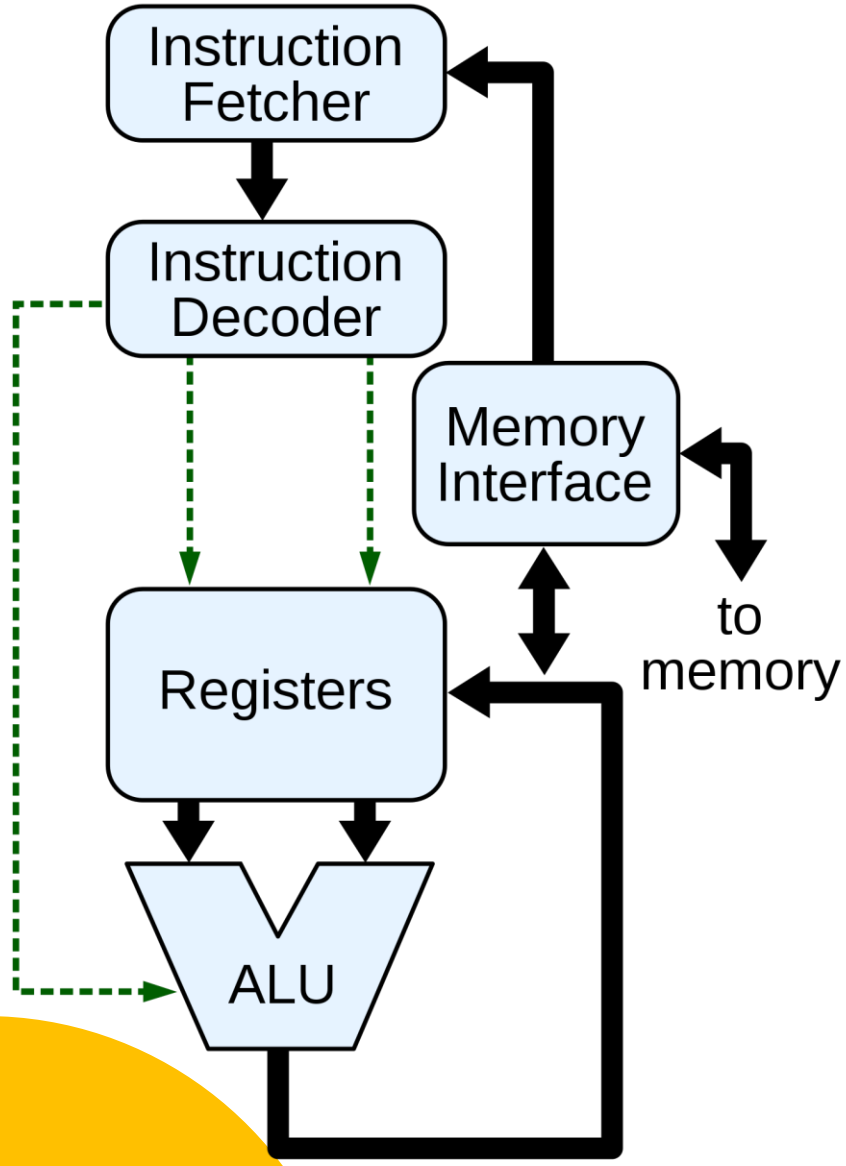
- Introduction
- Architecture
- Assembly Language
- Development Platforms
- Digital Inputs / Outputs
- Interrupts
- Timing Operations
- Data Acquisition
- Digital Communication
- Memory Operations

- An embedded system is a computing device developed for solving a specific problem.
- To do so, it interacts with the environment as acquiring data, processes the acquired data, and produces the corresponding output accordingly.
- To perform all these operations, joint usage of hardware and software becomes mandatory.
- The embedded system will be working in stand-alone form most of the times.
- It may communicate with other nearby devices as with Internet of Things (IoT) applications. Each embedded system can work in stand-alone form and share data to perform a complex operation.
- Since the embedded system works in stand-alone form most of the times, it depends on battery or energy harvesting module to operate.
- Therefore, energy dissipation becomes one of the main concerns in embedded system development.

- We can group embedded systems based on their hardware properties as follows.
- Field-programmable gate arrays (FPGA) form the first group. They provide the most flexible but hard to master hardware. Due to their properties, they have been extensively used in forming custom embedded systems at gate level.
- The second hardware group in forming embedded systems consists of microcontrollers. A microcontroller has limited memory and computation power. However, it can be programmed in assembly or high-level language to perform the operations on it.
- The microprocessors form the third embedded system group. In these, embedded Linux is used most of the times to control and organize operations. The most well known microprocessor-based embedded systems are the Raspberry Pi family and its clones.
- The fourth hardware group for embedded systems consists of system on chip (SoC) devices. They have FPGA and microprocessor modules on them. Hence, they aim to benefit from both device properties.
- We picked the microcontroller as the embedded system in this course.

- Each microcontroller family has its own programming language called assembly.
- The assembly language allows the programmer to reach and modify the microcontroller hardware at its lowest level.
- Although this is a fairly powerful tool, there are some shortcomings as well.
- First and most important of all, an assembly code written for a microcontroller family cannot be ported to a microcontroller from another family.
- Hence, C became the de-facto programming language for microcontrollers.
- Integrated development environments allow the user to control and debug the code running on the microcontroller fairly easily.
- Likewise, Arm® introduced the Mbed development environment for its microcontrollers. This platform depends on C++ language to operate.

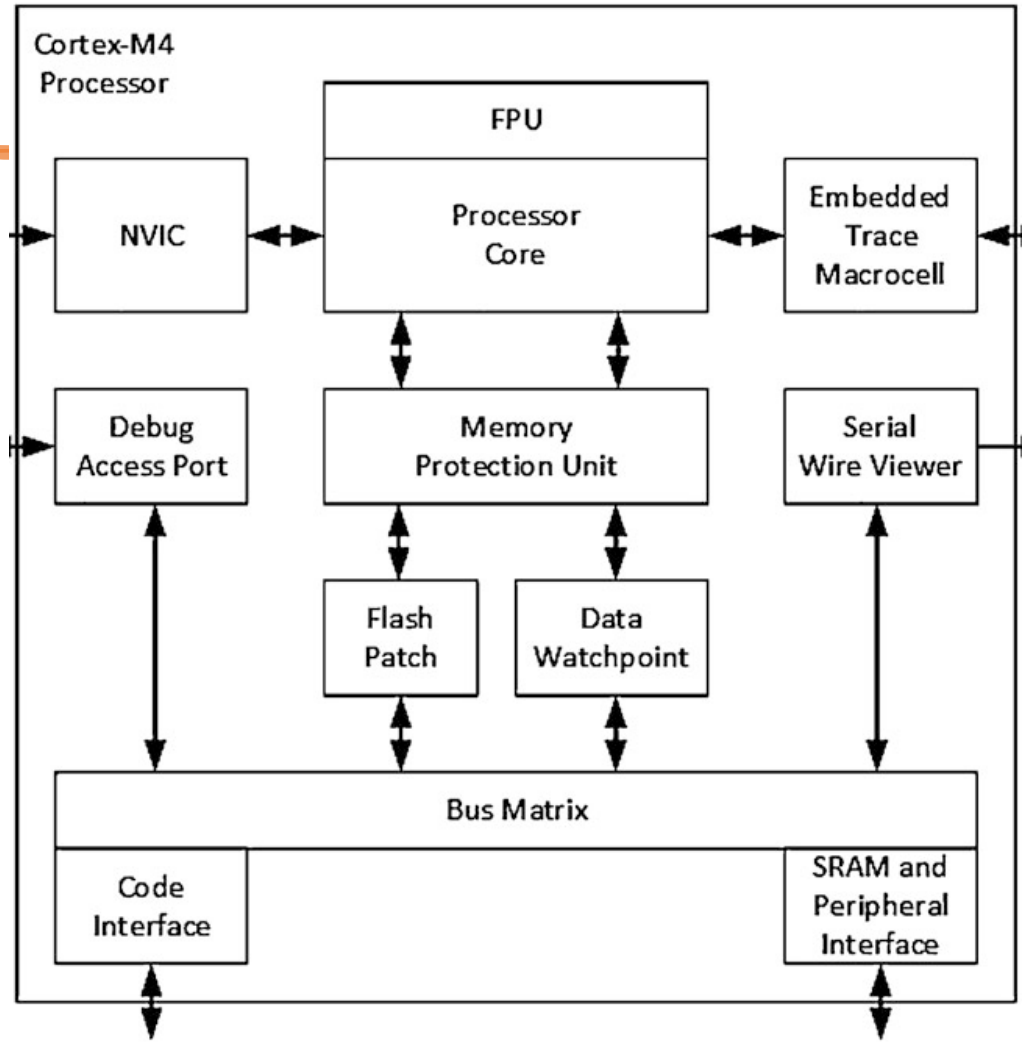

External
Interrrupts
Interrupt
Control
On-chip
ROM
for code
On-chip
RAM
Etc.
Timer 0
Timer 1
Counter Inputs
CPU
OSC
Bus
Control
I/O
Ports
Serial
Port
P0 P1 P2 P3
TXD
RXD



- Grouping hardware and peripheral units of the microcontroller as CPU, memory, general-purpose input and output ports, clock and timer modules, analog modules, digital communication modules, and other modules is beneficial for simplicity.
- Central processing unit (CPU) is responsible for organizing all operations within the microcontroller. This is done by the instructions provided to it by the programmer.
- The programmer constructs algorithm and forms the corresponding C, C++, or assembly code. This code is debugged and embedded into flash memory of the microcontroller by the help of an integrated development environment (IDE, such as STM32CubeIDE) running on PC.
- The CPU executes the commands by using its resources (such as peripheral units).

- CPU of the STM32F4 microcontroller:

- This diagram gives insight about the inner working principles of the CPU as will be explained next.
- Wire connections and their direction are not random in this figure, they indicate the direction of dataflow from and to the CPU.
- The processor core (as indicated in the figure) has an accompanying floating-point unit (FPU). This unit is responsible for handling operations on
g floating-point unit (FPU). This unit is responsible for handling operations on floating-point numbers in hardware. Hence, they can be executed in a fast manner.

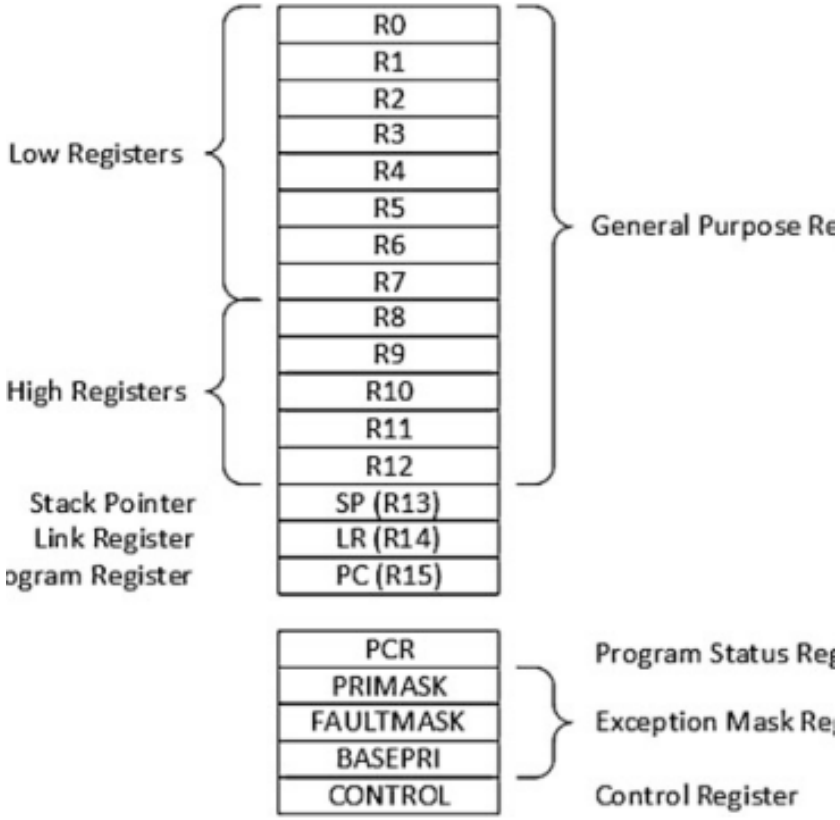


- The nested vectored interrupt controller (NVIC) is responsible for managing interrupt- and exception-based operations within the CPU. We will talk about what an interrupt is and how it is handled by NVIC in detail.
- The debug access port (DAP) helps us to reach and monitor the CPU during operation. Hence, we can observe what is going on inside the CPU while executing code. In other words, DAP allows us to monitor CPU registers without affecting the CPU operation.
- The other option to perform the same operation is leading the CPU to the debug state. Here, the CPU operation is halted. Then, its registers can be monitored.
- There is also the serial wire viewer (SWV) performing a similar operation. Through it, the user can trace the program counter, variable and peripheral register values, exceptions, time stamp, and CPU cycles.
- Data and codes are stored in memory will be explained in detail next week. Memory protection unit, flash patch, and data watchpoints modules within the CPU are used to manage memory.

- The embedded trace macrocell (ETM) module allows monitoring the code execution within the CPU (without stopping it). This is done by the GPIO pins. Therefore, the user can check when the CPU enters an interrupt, which task uses a common variable, and where an error has occurred if there is any. The ETM module needs external modules to perform all these operations.
- There is a bus matrix which connects modules within the CPU. Let's first explain what bus means. In simplest terms, bus is composed of parallel wires in which data (and code blocks) are transferred within the CPU.
- Adjacent to the bus matrix, there are two blocks: the first one being code and the second one being SRAM and peripheral interface.
- The code interface block is responsible for retrieving code blocks to be processed from memory.
- The SRAM and peripheral interface block is important for two reasons. First, an external memory module can be added to the microcontroller through this interface. Second, all peripheral units will be reached as if they are memory addresses. The SRAM and peripheral interface block module will be used for this purpose.

- The microcontroller needs a medium to transfer data to and from the outside world. This is done by using ports of the microcontroller. A port is a collection of pins grouped together. Each pin can be taken as a wire with its electronic control circuitry.
- The CPU and peripheral units within the microcontroller are synchronous devices. This means that operations within them are synchronized by a common clock signal.
- Frequency of the clock signal decides on the speed of operation in actual time. The clock tick idiom indicates the period of the clock signal. The time needed to perform an operation in the microcontroller is indicated by clock pulses (or clock ticks). Therefore, the higher the clock frequency, the faster that operation is performed in actual time. Clock signals are generated by oscillator circuitry which can be located inside or outside the microcontroller.
- The ADC module generates digital representation of an analog voltage level fed to the microcontroller.
- The DAC module converts a digital representation to analog voltage level and feeds it to output.
- Digital communication modules are used to communicate with external devices using dedicated communication protocols.

- Registers are data storage elements on the CPU. The STM32F4 microcontroller has 21 core registers in which 13 of them (R0 to R12) can be used for general purpose. The remaining eight registers are reserved for specific operations by the CPU.
- General-purpose registers (R0–R12) can be used for data processing operations. These are divided into two groups as low register (R0–R7) and high register (R8– R12) depending on their usage by 16- or 32-bit instructions.
- The remaining eight registers are used for specific purposes. Among these, the program counter (R15, PC) is the register responsible for holding address of the instruction to be executed. Related to this, there is a link register (R14, LR) which holds return address during branching and exception operations such as interrupts. Stack pointer (R13, SP) is responsible for storing the location of local variables and is used when a function is called in code.

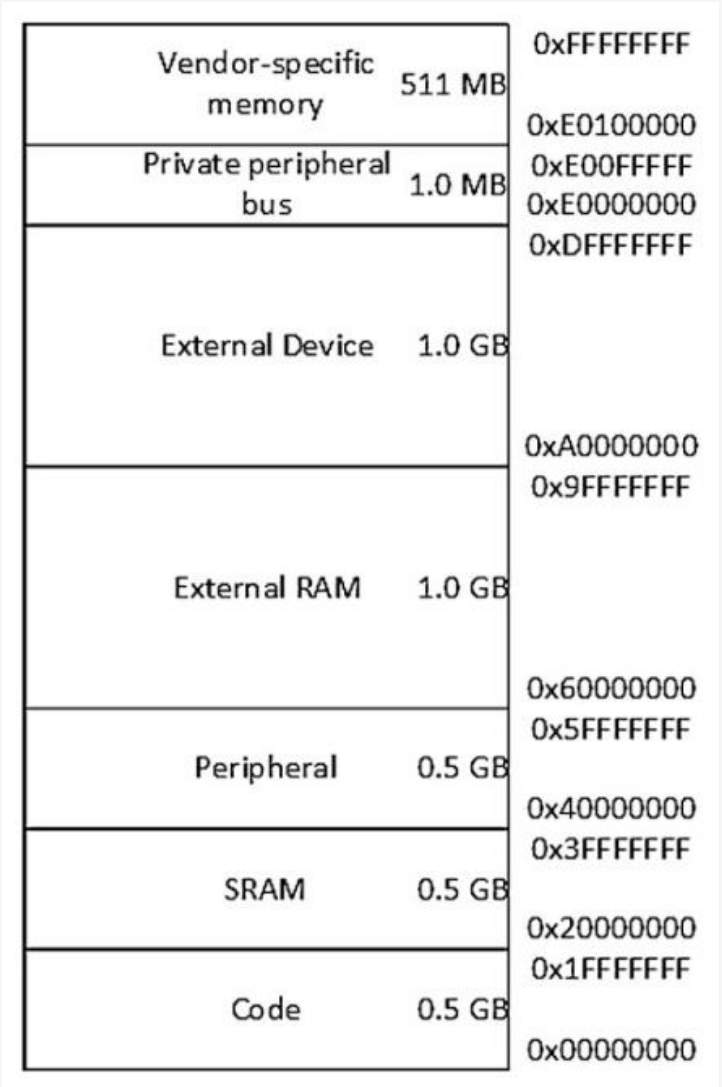


- When an instruction is executed within the CPU, the output is summarized as being negative, zero, or an overflow, carry, or saturation has occurred. These results are stored as condition flags (each being one bit) within the program status register (PSR).
- The negative (N) flag is set to logic level 1 when the result of an operation is negative. It is set to logic level 0 otherwise.
- The zero (Z) flag is set to logic level 1 when the result of an operation is zero. It is set to logic level 0 otherwise.
- The overflow (V) flag is set to logic level 1 when the result of an operation causes an overflow. It is set to logic level 0 otherwise.
- The carry (C) flag is set to logic level 1 when the result of an operation generates a carry. It is set to logic level 0 otherwise.
- The saturation (Q) flag is set to logic level 1 when the result of an operation causes saturation. It is set to logic level 0 otherwise.

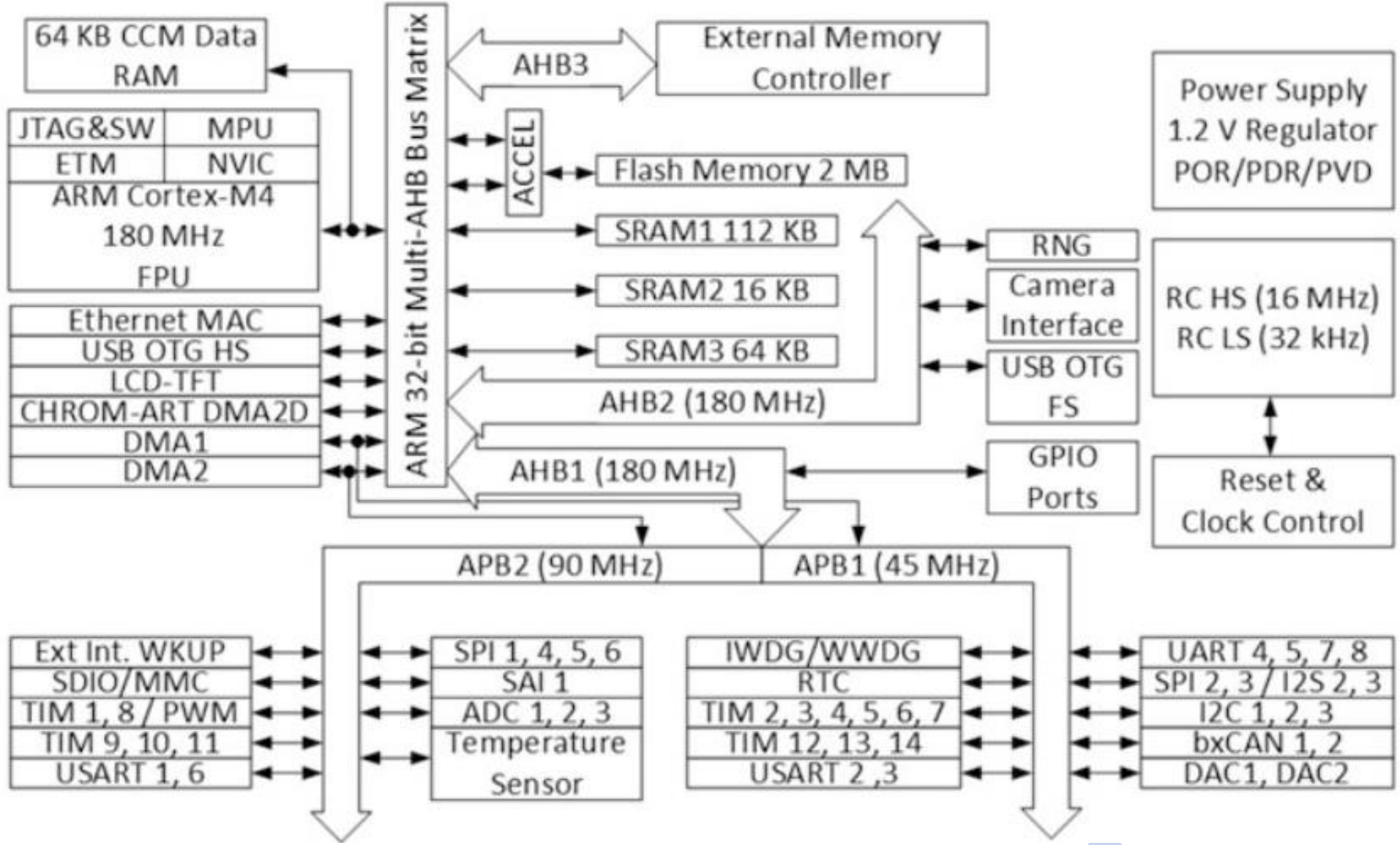
- There are two memory types in microcontrollers. There is random access memory (RAM) to store temporary data. Contents of RAM are lost when the power fed to it is turned off. There is also read-only memory (ROM) to store code or data permanently. Even if the power fed to it is turned off, contents of ROM are kept.
- RAM. In fact, it tells us that any randomly picked region within RAM can be accessed (reached) at the same time duration. Likewise, ROM may be misleading such that we only read its content. So, when and how was code or data written to it? Well, sometime in the operation, we should have written data on ROM.
- The STM32F4 microcontroller has two memory types as static RAM (SRAM) and flash. SRAM is a special memory type which does not need periodic refreshing as RAM. It also provides faster access to data. The STM32F4 microcontroller has 256 kB SRAM.
- The first 64 kB of SRAM is called core coupled memory (CCR) which provides fast code execution. There is also an extra 4 kB backup SRAM within the microcontroller which keeps data written to it even if the microcontroller goes into standby mode.

- The STM32F4 microcontroller has 2 MB flash memory. This memory is divided into two banks and each being composed of 12 sectors. These sectors are formed by 16 kB, 64 kB, or 128 kB memory regions. This structure allows the programmer to erase a specific sector, bank, or the overall flash memory.
- Arm® partitioned the memory address range for its microcontroller cores based on usage. Since the STM32F4 microcontroller is based on the Arm® Cortex™-M4 architecture, it has the same memory map.
- There is an extremely important concept to be mentioned here. The memory map of the STM32F4 microcontroller does not only represent SRAM and flash memory addresses. It also represents the interrupt and reset vector table, on-chip peripheral units, off-chip memory, off-chip peripherals, system components, and on- and off-chip debug components.
- The first 512 MB memory space is for code to be executed on the microcontroller. The next 512 MB memory space is used for on-chip SRAM. Code to be executed can also be stored here. However, this creates extra wait states and longer execution times. Therefore, running the code from this region is not recommended by Arm®.

- The next 512 MB memory space is used for all on-chip peripheral unit (GPIO, timers, ADC, and DAC) registers.
- All peripheral units are also treated as if they are separate memory locations. Hence, if the CPU tries to reach a peripheral unit, this is done in the same way as reaching a memory location.
- Although the user does not see this operation while writing code in C or C++ languages, it is important to keep this in mind.
- The next 2 GB memory space is used for external memory and peripheral units.
- The final 512 MB memory space is used for system peripheral units such as NVIC, memory protection unit (MPU), and SysTick (specific timer to be explored in Chap. 6). This memory space is also used for internal-external debug and vendor specific components



- There are also memory related modules within the STM32F4 microcontroller. The first one is the direct memory access (DMA) module. We can explain the importance of this module as follows. The CPU is used in most memory reaching operations. However, it may be busy for some cases.
- The DMA module allows reaching memory without interfering the CPU. Hence, the CPU can work on its own task. At the same time, reaching memory can be done in parallel by the DMA module.
- The STM32F4 microcontroller has DMA controllers.
- Besides these, STM32F4 has flexible memory controller (FMC) and chrom-art accelerator controller (DMA2D) modules.


64 KB CCM Data RAM
JTAG&SW
MPU
ETM
NVIC
ARM Cortex-M4
180 MHz
FPU
Ethernet MAC
USB OTG HS
LCD-TFT
CHROM-ART DMA2D
DMA1
DMA2
ARM 32-bit Multi-AHB Bus Matrix
AHB3
External Memory Controller
ACCEL
Flash Memory 2 MB
SRAM1 112 KB
SRAM2 16 KB
SRAM3 64 KB
AHB2 (180 MHz)
AHB1 (180 MHz)
RNG
Camera Interface
USB OTG FS
GPIO Ports
Power Supply
1.2 V Regulator
POR/PDR/PVD
RC HS (16 MHz)
RC LS (32 kHz)
Reset & Clock Control
APB2 (90 MHz)
APB1 (45 MHz)
Ext Int. WKUP
SDIO/MMC
TIM 1, 8 / PWM
TIM 9, 10, 11
USART 1, 6
SPI 1, 4, 5, 6
SAI 1
ADC 1, 2, 3
Temperature Sensor
IWDG/WWDG
RTC
TIM 2, 3, 4, 5, 6, 7
TIM 12, 13, 14
USART 2 ,3
UART 4, 5, 7, 8
SPI 2, 3 / I2S 2, 3
I2C 1, 2, 3
bxCAN 1, 2
DAC1, DAC2

Assembly Language

- When we write code in C or C++ language, in fact we are not writing in the format the microcontroller understands.
- The compiler converts our code to machine language and embeds it to code region of the microcontroller memory. Hence, it can be executed by the CPU.
- Since the code in machine language is stored in memory, it is in binary form. Therefore, understanding it is neither trivial nor feasible for us.
- Assembly language offers an option to write the code as close as possible to machine language but at the same time understandable by a human when inspected.
- In other words, we can write the code in assembly language in an editor which can be understood when inspected.
- Besides, assembly language allows us to reach low-level hardware of the microcontroller.
- Therefore, using it provides a good insight in understanding inner working principles of the microcontroller.

VisUAL2

```
ORR R1, R1, #16 ; sets bit 4 in R1
BICS R10, R10, 0x80000; clear bit 19 of R10, setting falgs NZ
```

OpCode	Function	Flags changed if S Suffix
AND	dest := op1 Bitwise AND op2	NZ
ORR	dest := op1 Bitwise OR op2	NZ
EOR	dest := op1 Bitwise exclusive OR op2	NZ
BIC	dest := op1 Bitwise AND NOT op2	NZ

```
MVN R1, R1   ; inverts bits in R1
MOVS R10, R3 ; load R10 with copy of value in R3, write flags `NZ`.
```

OpCode	Function	Flags changed if S suffix
MOV	dest := op2	NZ
MVN	dest := Bitwise invert op2	NZ

```
SUB R10, R5, R4 ; set R10 equal to R5 - R4
ADD R1, R1, #1  ; add 1 to R1
RSBS R1, R1, #0 ; negate R1 setting NZCV (R1 = 0 - R1)
```

OpCode	Function	Flags changed if S Suffix
ADD	dest := op1 + op2	NZCV
SUB	dest := op1 - op2	NZCV
RSB	dest := op2 - op1	NZCV
ADC	dest := op1 + op2 + C	NZCV
SBC	dest := op1 - op2 + C - 1	NZCV
RSC	dest := op2 - op1 + C - 1	NZCV

OpCode	Example	Notes
DCD	`DAT DCD d1, d2,...,dn`	Set next n data words to numeric values given by operands d1,...,dn (no #). Label `DAT` is usual but not compulsory
DCB	`DAT1 DCB d1, d2,...,dn`	Set next n data bytes to numeric values given by operands d1,...,dn (no #). n must be divisible by 4. Bytes are written in little-endian address order. Label `DAT1` is usual but not compulsory.
FILL	`DAT2 FILL 40`	`40` can be any literal numeric expression (without #). Fill next 40 words of data memory with 0 Label `DAT2` is usual but not compulsory.
ADR	`ADR Ra, LABEL`	Set Ra to `LABEL`. Same as `MOV Ra, #LABEL` but literal values close to PC are allowed. Useful to load data area label values.
EQU	`LAB1 EQU LABEL + 4`	`EQU` sets its label, in this case `LAB1`, equal to the given numeric expression. Forward references are allowed.

```
LSLS R0, R1, #4     ; same as MOVS R0, R1, LSL #4
ROR  R1, R1, #2     ; same as MOV R1, R1, ROR #2
RRX  R1, R2         ; same as MOV R1, R2, RRX
```

These are shorthand for the corresponding `MOV` with shifted *op2*.

OpCode	Function
LSR	Logical Shift Right (shift 0s in)
ASR	Arithmetic Shift Right (shift sign bit in)
LSL	Logical Shift Left
ROR	Rotate Right (shift RHS bit in)
RRX	Rotate Right eXtended - always 1 bit

```
CMP R0, R1 ; set `NZCV` flags on subtraction R0 - R1
TEQ R3, R4 ; set `NZ` flags on R3 XOR R4
```

OpCode	Function	Flags changed
CMP	set flags on op1 - op2	NZCV
CMN	set flags on op1 + op2	NZCV
TST	set flags on op1 Bitwise AND op2	NZ
TEQ	set flags on op1 Bitwise XOR op2	NZ

```
LDR R0, [R1,#8] ; load into R0 mem32[R1+8]
STRB R11, [R2,#3]! ; store LS byte R11 in mem8[R2+3], set R2 := R2+3
LDR R0, [R1],#8 ; load into R0 mem32[R1]. then set R1 := R1+8.
LDR R5, =DATA ; load 32 bit immediate value of symbol DATA into R5
```

See Address Modes for details of **EA** modes.

OpCode	Instruction	Function	Notes
LDR	`LDR Rd, EA`	Rd := mem32[EA]	1
LDRB	`LDRB Rd, EA`	Rd := mem8[EA]	2
STR	`STR Rs, EA`	mem32[EA] := Rs	1
STRB	`STRB Rs, EA`	mem8[EA] := Rs	3

LDR =	`LDR Rd, =Literal`
	`LDR Rd, =DATA+4`
	`LDR Rd, =1571`
		Rd := Literal	4

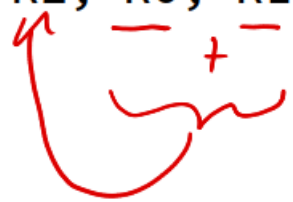
1.EA must be divisible by 4.

2.This loads one byte from memory into the LS 8 bits of Rd. The MS 24 bits are zeroed.

3.This writes the LS 8 bits of Rd into the specified memory byte. Other bytes in the same memory word are unaffected.

4.The literal can be any numeric expression of constants and symbols, and does not have a #. Unlike MOV there is no restriction on value. Technically this instruction is a pseudo-instruction, translated by assembler into an LDR with EA offset from PC, and a DCD that defines the the 32 bit constant. Programmers can use it exactly as MOV with literal op2, noting that it is slower than MOV.

```
MOV R0, #2 ; Put 2 into R0
MOV R1, #3 ; Put 3 into R1
ADD R2, R0, R1 ; R2 = R0 + R1
```



```
LDR R0, [R1] ; Load the memory at address R1 and put it into R0
```

Register to load into

Address

```
STR R0, [R1] ; Store the value of R0 into memory at address R1
```

Register to save

Address

Adding to a memory location

```
MOV R1, #0x1000 ; Put the value 0x1000 into R1
LDR R0, [R1] ; The part in brackets is an address (a pointer)
ADD R0, R0, #2 ; Do the addition
STR R0, [R1] ; Store the result back
```

Equivalent C code:

```
int* p = (int*)0x1000
*p += 2;
```

Append "**S**" to an ALU command to make it set the CPSR
CMP is like **SUBS**, but doesn't put the result anywhere

Append "**EQ**", "**NE**", "**GE**", etc. to make an operation conditional

C code

```
if (a >= 100) {
    b = 5;
}
```

Equivalent assembly

```
CMP R0, #100
MOVGE R1, #5
```

Practice

Write a program that computes the absolute value of a number stored in **R0**.

Absolute value

```
CMP R0, #0 ; Compare it to zero
RSBLE R0, R0, #0 ; If < or =, do R0 = 0 - R0
```

Challenge

Write a program that computes the absolute value of a value stored in memory. You can use these lines to define data in memory and load its address:

```
DATA DCD -130; or whatever value you want
MOV R1, #DATA
```

```
MOV R1, #DATA ; Put the address of DATA into R1
LDR R0, [R1] ; Load the value at that address into R0
CMP R0, #0 ; Compare it to zero
RSBLE R0, R0, #0 ; If < or =, do R0 = 0 - R0
STR R0, [R1] ; Store the result back
```

Branches

A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        B       START
```

```
int* p = (int*)0x1000;
while(1){
    *p++;
}
```

Branches

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        CMP     R0, #100
        BNE     START
```

```
int* p = (int*)0x1000;
while(*p != 100){
    *p++;
}
```

(not quite! C checks the condition at the top.)

Practice!

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

Loops

A while loop can be written in the form:

```
TOP
    check if condition is false, and branch to BOTTOM if so
    [ body of loop ]
    unconditionally branch back to TOP
BOTTOM
```

```
1   data    DCD     0xFF, 0xEE, 0xDD
2
3
4           MOV     R0, #5
5           MOV     R1, #-5
6           MOV     R2, #018
7           MOV     R3, #0x7FFFFFFF
8
9           ADDS    R3, R3, #1
10
11          SUBS    R4, R0, R1
12
13          ADR     R0, 0x300
14
15          STR     R4, [R0]
16
17
18          LDR     R6, [R0]
19
20
```

```
1   value   DCD     44, 0x44
2
3           MOV     R0, #0x200
4
5           LDR     R1, [R0]
6
7           ADD     R0, R0, #4
8
9           LDR     R2, [R0]
10
11
12          ;       AND R3, R1, R2
13
14          ;       ORR R4, R1, R2
15          ;       EOR R5, R1, R2
16
17          MVN     R3, R2
18
19          BIC     R4, R1, R2
20
```

```
1   dat1    DCD     -5
2
3
4           ;       LDR R0, =dat1
5           MOV     R0, #dat1
6
7
8           LDR     R1, [R0]
9
10          CMP     R1, #0
11
12          RSBLT   R2, R1, #0
13
14
15          LSL     R3, R1, #1
16
17          LSR     R4, R2, #1
18
```

ee2004

Week3,4

testbench

wa2.s * | wa2_tb.s *

```
1   ##TESTBENCH
2   #TEST 1
3   IN R0 is 1
4   IN R1 is 2
5   OUT R2 is 3
6   >>;          Test 1 PASSED.
7   #TEST 2
8   IN R0 is 10
9   IN R1 is 200
10  OUT R2 is 210
11  >>;          Test 2 PASSED.
```

wa2.s * | wa2_tb.s *

```
1
2       ADD     R2,R1,R0
3
4
```



A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        B       START
```

```
int* p = (int*)0x1000;
while(1){
    *p++;
}
```

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000
START   LDR     R0, [R1]
        ADD     R0, R0, #1
        STR     R0, [R1]
        CMP     R0, #100
        BNE     START
```

```
int* p = (int*)0x1000;
while(*p != 100){
    *p++;
}
```

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

```
1   main
2           mov     R1,#0 ; R1 is i
3           mov     R0,#0 ; R0 is sum
4
5   loop
6           add     R0,R0,R1 ; sum = sum + i
7           add     R1,R1,#1 ; i++
8           cmp     R1,#10 ; i <= 10
9           ble     loop
10
11  halt
12          b       halt
```

```c
int x[10];
int sum = 0;

for (int i = 0; i < 10; i++)
    sum += x[i];
```

```
1   veri    DCD     1, 3, 5, 7, 9, 11, 13, 15, 17, 19
2
3   sum     DCD     0
4
5   main    mov     r0,#veri
6           mov     r1,#sum
7           ldr     r2,[r1]
8           mov     r3,#0 ; r3 is i
9
10  for     ldr     r4,[r0,r3,lsl #2]
11          add     r2,r2,r4
12          add     r3,r3,#1
13          cmp     r3,#10
14          blt     for
15          str     r2,[r1]
16
17  halt    b       halt
```

```
        MOV     R3, #5
        MOV     R4, #5


        CMP     R3,R4     ; if (i >= j)
        BLT     else
        SUB     R0,R3,R4  ; x = i - j;
        B       endif     ; else
else    SUB     R0,R4,R3  ; x = j - i;
endif

        CMP     R3,R4     ; x = (i >= j) ? i - j : j - i;
        SUBGE   R0,R3,R4
        SUBLT   R0,R4,R3
```

```
2   main
3           mov     r0, #35
4           mov     r1, #55
5           adr     r5, 0x300
6           bl      GCD
7
8   GCD
9           cmp     r0,r1
10          beq     halt
11          subgt   r0,r0,r1
12          sublt   r1,r1,r0
13          b       GCD
14
15  halt
16          str     r0, [r5]
17
```

```
int gcd(a,b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}

int x = 35;
int y = 55;
int z;

z = gcd(x, y);
```

Creating an Assembly Project in STM32CubeIDE

- We provide a template assembly code using the mentioned directives in Listing 15.1. To note here, this structure is specific to STM32CubeIDE. There may be extra directives or a different format for other development environments. Therefore, the reader should check them before preparing his or her assembly code in other IDEs.

- **.text** assembles the following code into text (executable code) section.
- **.align** defines the length of instructions to be processed in terms of bytes.
- **.global** defines one or more global symbols.
- **.end** identifies end of the code.

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

- The first directive .text indicates that the following lines correspond to assembly code. Hence, they should be placed in the code section of the flash memory.
- The second directive .align defines the length of instructions to be processed in terms of bytes. If the instructions to be executed have 16-bit length, then we should use .align 2 as in our case.
- The third directive .global defines the global variable main which indicates the starting address of our code. Finally, the directive .end indicates the end of the assembly code.

Creating an Assembly Project in STM32CubeIDE

- There is no direct method to create an assembly project in STM32CubeIDE.
- Therefore, we should first follow the steps to form an empty C project.
- To create the debug file (with extension ".debug"), we should debug our (empty) C code for the first time.
- As the mentioned file is created, we should delete the "main.c" file from the "Src" folder of the project.
- In fact, we can delete all files with extension ".c" from the project's "Src" folder.
- Then, we should add a new file with extension ".s" to this folder.
- To do so, right-click on the "Src" folder, and press "New" and "File."
- In the opening window, enter the "File Name" such as "main.s" and click "Finish."
- An empty file will be created under the "Src" folder.
- We can add our assembly code to this file. Here, we will use the sample codes:

- As we compile the code, we can execute it either by the “step into” option in STM32CubeIDE or directly by the “Resume” command. We can observe output of the code; the registers have been modified according to our assembly code.
- We can apply operations on the memory of the STM32F4 microcontroller by assembly language. In the second code, we modify and reach content of the memory address 0x20000020 by the LDR and STR instructions. As we compile and run the code, we can observe its output in the “Registers” and “Memory Browser” Windows.

```
.text
.align 2
.global main

Mem_Addr=0x20000020

main:

 MOVS R0, #0x09
 MOVS R1, #0x11
 MOVS R2, #10

 LDR R3, =Mem_Addr
 STR R2, [R3]
 LDR R0, [R3]

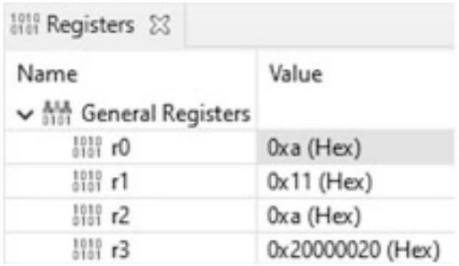
END: B END
  .end
```

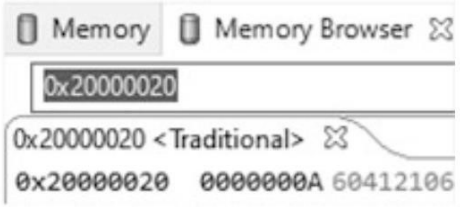
```
.text
.align 2
.global main

main:

 MOVS R0, #0x12
 MOVS R1, #0x2
 MOVS R2, #0x4
 ADD R3, R0, R1

END: B END
  .end
```





Lab1 solution

```
1
2
3           MOV     R0, #TESTDATA
4           MOV     R3, #8
5
6   loop    LDRB    R1, [R0]
7           ADD     R0, R0, #1 ; adres
8           ADD     R2, R2, R1 ; sonuç
9           SUB     R3, R3, #1 ; indeks
10          CMP     R3, #0
11          BNE     loop
12
13          LSR     R4, R2, #3
14
15          ADR     R5, 0x300
16          STR     R4, [R5]
17
18
19  TESTDATA DCD    0x0F1F2F3F, 0x4F5F6F7F
```

ee2004

Week3,4

testbench

wa2.s * | wa2_tb.s *

```
1   ##TESTBENCH
2   #TEST 1
3   IN R0 is 1
4   IN R1 is 2
5   OUT R2 is 3
6   >>;          Test 1 PASSED.
7   #TEST 2
8   IN R0 is 10
9   IN R1 is 200
10  OUT R2 is 210
11  >>;          Test 2 PASSED.
```

wa2.s * | wa2_tb.s *

```
1
2           ADD     R2,R1,R0
3
4
```



A branch lets you jump to another place in the program

```
        MOV     R1, #0x1000         int* p = (int*)0x1000;
START   LDR     R0, [R1]            while(1){
        ADD     R0, R0, #1              *p++;
        STR     R0, [R1]            }
        B       START
```

Branching *conditionally* allows you to build if/while/for/case/etc.

```
        MOV     R1, #0x1000         int* p = (int*)0x1000;
START   LDR     R0, [R1]            while(*p != 100){
        ADD     R0, R0, #1              *p++;
        STR     R0, [R1]            }
        CMP     R0, #100
        BNE     START
```

Write code to loop through the numbers 1-10, and compute their sum.

Example implementation in C:

```c
int sum = 0;
for(int i = 1; i <= 10; i++){
    sum = sum + i;
}
```

```
1   main
2           mov     R1,#0 ; R1 is i
3           mov     R0,#0 ; R0 is sum
4
5   loop
6           add     R0,R0,R1 ; sum = sum + i
7           add     R1,R1,#1 ; i++
8           cmp     R1,#10 ; i <= 10
9           ble     loop
10
11  halt
12          b       halt
```

```c
int x[10];
int sum = 0;

for (int i = 0; i < 10; i++)
    sum += x[i];
```

```
1   veri    DCD     1, 3, 5, 7, 9, 11, 13, 15, 17, 19
2
3   sum     DCD     0
4
5   main    mov     r0,#veri
6           mov     r1,#sum
7           ldr     r2,[r1]
8           mov     r3,#0 ; r3 is i
9
10  for     ldr     r4,[r0,r3,lsl #2]
11          add     r2,r2,r4
12          add     r3,r3,#1
13          cmp     r3,#10
14          blt     for
15          str     r2,[r1]
16
17  halt    b       halt
```

```
2           MOV     R3, #5
3           MOV     R4, #5
4
5
6           CMP     R3,R4     ; if (i >= j)
7           BLT     else
8           SUB     R0,R3,R4  ; x = i - j;
9           B       endif     ; else
10  else    SUB     R0,R4,R3  ; x = j - i;
11  endif
12
13          CMP     R3,R4     ; x = (i >= j) ? i - j : j - i;
14          SUBGE   R0,R3,R4
15          SUBLT   R0,R4,R3
```

```
2   main
3           mov     r0, #35
4           mov     r1, #55
5           adr     r5, 0x300
6           bl      GCD
7
8   GCD
9           cmp     r0,r1
10          beq     halt
11          subgt   r0,r0,r1
12          sublt   r1,r1,r0
13          b       GCD
14
15  halt
16          str     r0, [r5]
17
```

```
int gcd(a,b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        } else {
            b = b - a;
        }
    }
    return a;
}

int x = 35;
int y = 55;
int z;

z = gcd(x, y);
```

Creating an Assembly Project in STM32CubeIDE

- We provide a template assembly code using the mentioned directives in Listing 15.1. To note here, this structure is specific to STM32CubeIDE. There may be extra directives or a different format for other development environments. Therefore, the reader should check them before preparing his or her assembly code in other IDEs.

- **.text** assembles the following code into text (executable code) section.
- **.align** defines the length of instructions to be processed in terms of bytes.
- **.global** defines one or more global symbols.
- **.end** identifies end of the code.

Listing 15.1 Template assembly code

```
.text
.align 2
.global main

main:

// Assembly Code Goes Here

END: B END
  .end
```

Listing 15.1 Template assembly code

```
 .text
 .align 2
 .global main

 main:

 // Assembly Code Goes Here

END: B END
  .end
```

- The first directive .text indicates that the following lines correspond to assembly code. Hence, they should be placed in the code section of the flash memory.
- The second directive .align defines the length of instructions to be processed in terms of bytes. If the instructions to be executed have 16-bit length, then we should use .align 2 as in our case.
- The third directive .global defines the global variable main which indicates the starting address of our code. Finally, the directive .end indicates the end of the assembly code.

Creating an Assembly Project in STM32CubeIDE

- There is no direct method to create an assembly project in STM32CubeIDE.
- Therefore, we should first follow the steps to form an empty C project.
- To create the debug file (with extension ".debug"), we should debug our (empty) C code for the first time.
- As the mentioned file is created, we should delete the "main.c" file from the "Src" folder of the project.
- In fact, we can delete all files with extension ".c" from the project's "Src" folder.
- Then, we should add a new file with extension ".s" to this folder.
- To do so, right-click on the "Src" folder, and press "New" and "File."
- In the opening window, enter the "File Name" such as "main.s" and click "Finish."
- An empty file will be created under the "Src" folder.
- We can add our assembly code to this file. Here, we will use the sample codes:

```
.text
.align 2
.global main

Mem_Addr=0x20000020

main:

 MOVS R0, #0x09
 MOVS R1, #0x11
 MOVS R2, #10

 LDR R3, =Mem_Addr
 STR R2, [R3]
 LDR R0, [R3]

END: B END
  .end
```

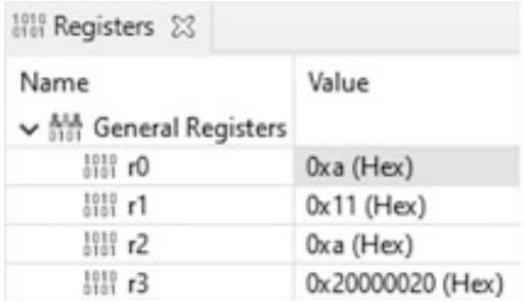
```
.text
.align 2
.global main

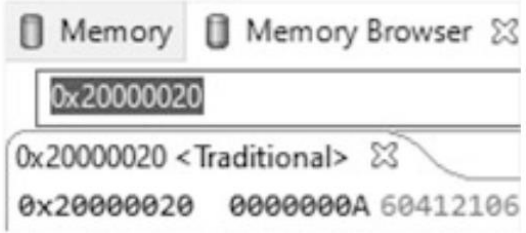
main:

 MOVS R0, #0x12
 MOVS R1, #0x2
 MOVS R2, #0x4
 ADD R3, R0, R1

END: B END
  .end
```

- As we compile the code, we can execute it either by the “step into” option in STM32CubeIDE or directly by the “Resume” command. We can observe output of the code; the registers have been modified according to our assembly code.
- We can apply operations on the memory of the STM32F4 microcontroller by assembly language. In the second code, we modify and reach content of the memory address 0x20000020 by the LDR and STR instructions. As we compile and run the code, we can observe its output in the “Registers” and “Memory Browser” Windows.





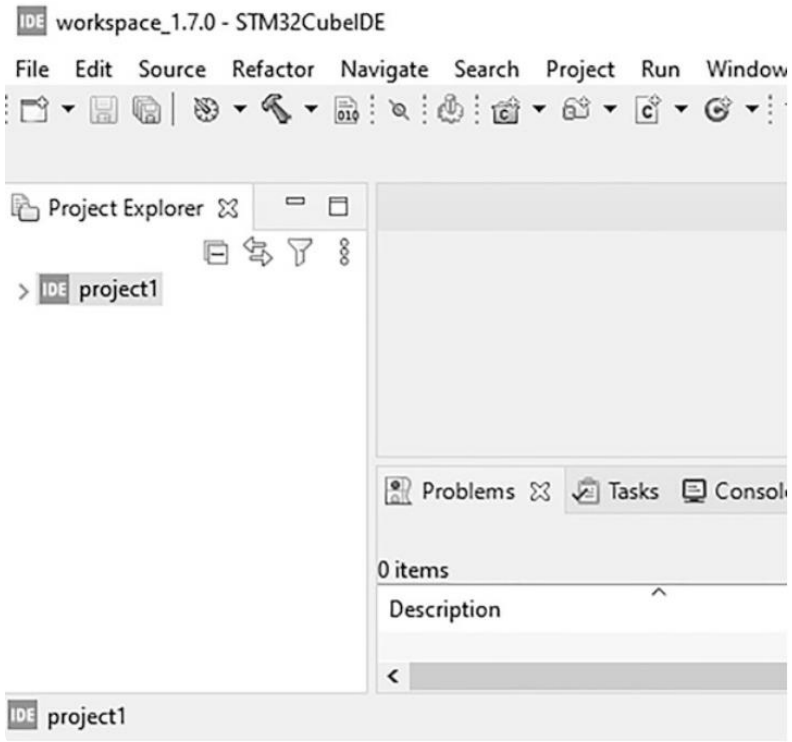
Lab1 solution

```
1
2
3           MOV     R0, #TESTDATA
4           MOV     R3, #8
5
6   loop    LDRB    R1, [R0]
7           ADD     R0, R0, #1 ; adres
8           ADD     R2, R2, R1 ; sonuç
9           SUB     R3, R3, #1 ; indeks
10          CMP     R3, #0
11          BNE     loop
12
13          LSR     R4, R2, #3
14
15          ADR     R5, 0x300
16          STR     R4, [R5]
17
18
19  TESTDATA DCD    0x0F1F2F3F, 0x4F5F6F7F
```

The STM32CubeIDE Platform

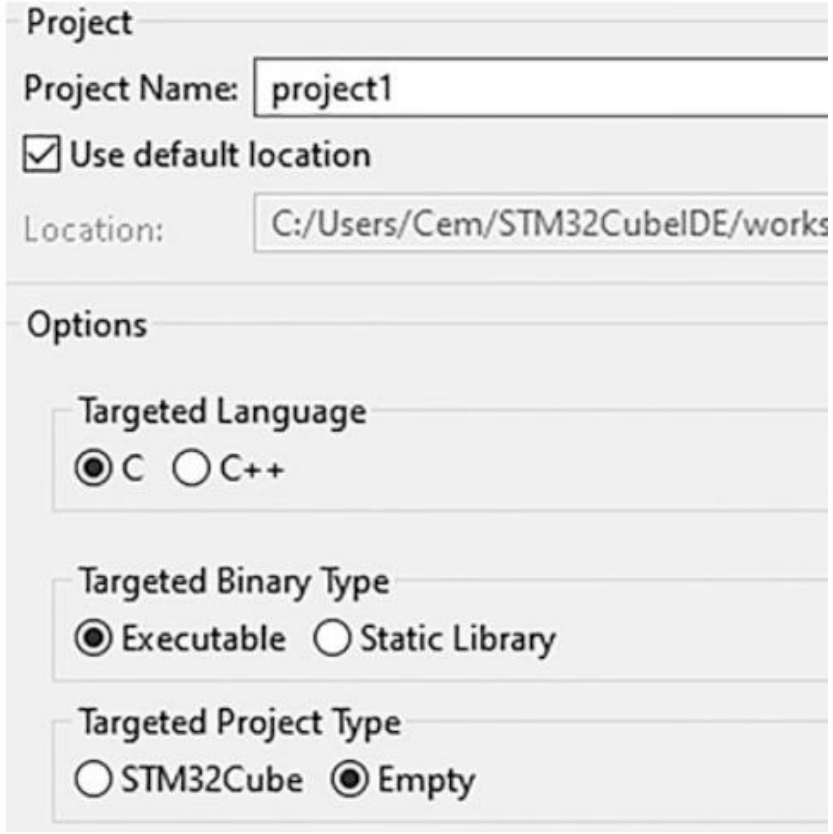
- We will benefit from the STM32CubeIDE platform to program the STM32F4 microcontroller via C language. This platform has several advantages. First, it is free. Second, it is specifically developed for microcontrollers by STMicroelectronics. Third, it is possible to reach and modify all properties of the microcontroller including its peripheral units using STM32CubeIDE.
- To get familiar with STM32CubeIDE, we will start with its installation. Afterward, we will explain how to create and manage a project. Here, we will not adjust peripheral units of the microcontroller. Instead, we will focus on fundamental properties of creating and executing a project in STM32CubeIDE. As we master this step, we will see how the peripheral units can be modified. To do so, we will benefit from STM32CubeMX which will be of great use in managing microcontroller hardware.
- STM32CubeIDE can be downloaded from the web site https://www.st.com/en/ development-tools/stm32cubeide.html after registering or logging in.

- There exists the “Project Explorer” panel on the left side of the welcome window. The user can create a new project through it.
- As a new project is created, this panel summarizes all its properties. Code files will be opened as they are added to the project at the center of STM32CubeIDE.
- Hence, the reader will modify his or her code in this section. There will be other sections which summarize hardware properties of the microcontroller on the right side of the panel.
- The reader can choose available perspectives such as “C/C++,” (default) “Debug,” and “Device Configuration Tool” in STM32CubeIDE.
- There are also grouped buttons on the top of the panel. These will be of use in debugging and modifying the code. There are tabs which will summarize all build and debug operations at the bottom of the panel. We will explain all these properties in the following sections as we focus on parts of a project.

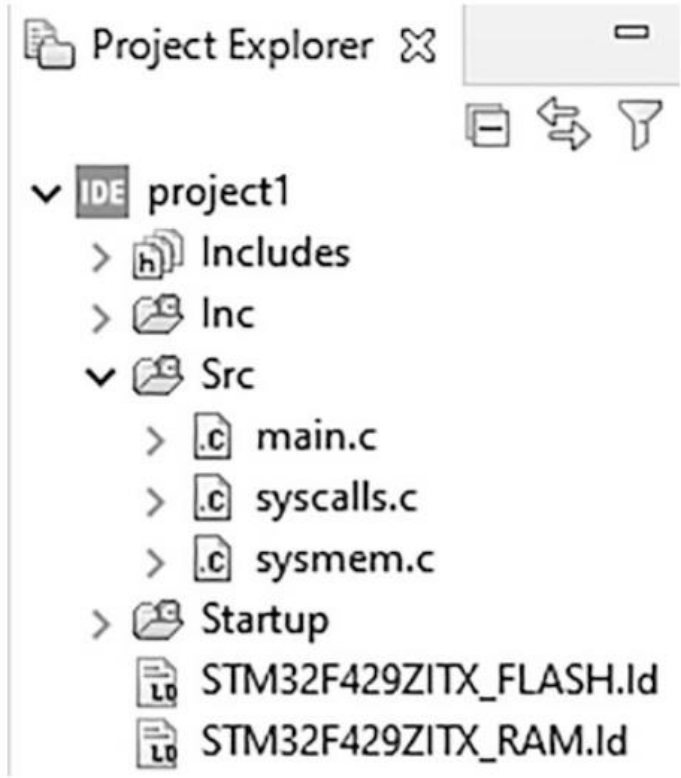


Creating a New Project

- A project typically contains source, header, and includes files. STM32CubeIDE generates an executable file from these to be embedded to the microcontroller. This section is about creating a C project. We have two options here. The first one does not deal with peripheral and pin properties of the microcontroller. The second one deals with these. We will focus on the first option in this section to get familiar with the basics of STM32CubeIDE.
- To create a new project, click “File,” “New,” and “STM32 Project.” In the opening window, select the hardware platform to be used in the project. For our case, we should choose “L073rz nucleo” from the “board selector” tab and press “Next.” Afterward, the “Project Setup” window opens as in Fig.



- We should give a name to our project in this window. For our case, we set it as “project1.”
- We should set the location of the project which may be done by the option “use default location.”
- We should set the “Targeted Language” as C.
- We should set the “Targeted Binary Type” as “Executable.”
- The final selection is titled “Targeted Project Type.” Here, we have two options as “STM32Cube” and “Empty.” The first option allows modifying the peripheral units.
- We select the “Empty” option to generate the project as for now. As we press the “Finish” button, a new project will be generated which can be observed under the “Project Explorer” tab as in Fig.



Listing 3.1 The first C code for the STM32F4 microcontroller

```
int a = 1;
int b = 2;
int c;

int main(void)
{
int d;


c = a + b;
d = c;

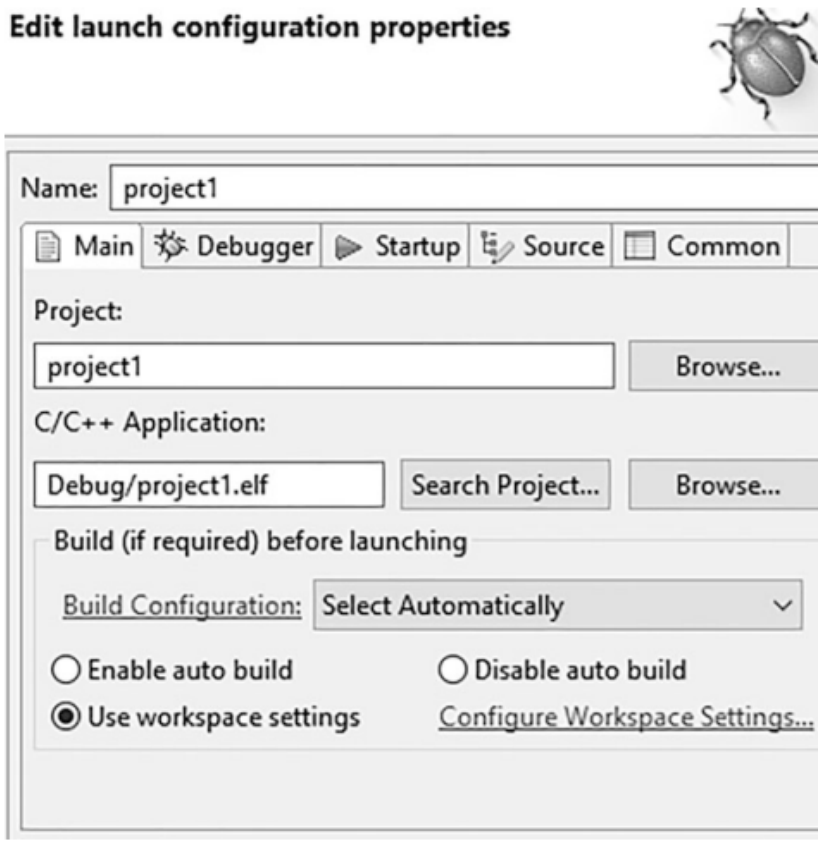
while (1);
}
```

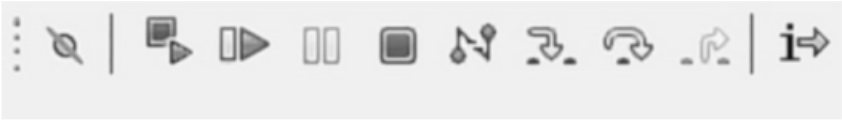
- The only step left is adding our C code to the “main.c” file generated under the “Src” folder. In this section, we will use the code given in Listing 3.1 as an example. To do so, open the “main.c” file by double-clicking on it. Add your code to this file. Then, save it by clicking the “Save” button on the upper left corner of the menu.

Building, Debugging, and Executing the Project

- As we create our project, the next step is its execution. To do so, we will first consider the building and debugging steps. Afterward, we will focus on the execution step.
- The first step in executing the code on an embedded platform is building and debugging it. There is a button with hammer shape on STM32CubeIDE horizontal toolbar as “Build”. As the user presses the right bottom arrow there, two options emerge as “Debug” and “Release”.
- The first option builds the project with the necessary debug information. In this default option, no optimization is done on the built output.
- The second option is specific to deploying the built output to the microcontroller. Hence, the main focus here is optimizing output of the project and minimizing its size. Running steps in both options (such as warnings and errors) can be observed in the “CDT Build Console” window. If there is an error in a code line, it is indicated by a red cross.

- The second step in executing the code is debugging and loading it to the target device. There is a button on STM32CubeIDE's horizontal toolbar as "Debug." It can also be reached from the "Run" tab. As we press this button, a new window appears asking for the "Edit Configuration" as in Fig. 3.4. Within this window, we can make all adjustments related to the debugging process. The default settings should be kept as they are in this window unless otherwise stated.
- As we press "OK" in the "Edit Configuration" window, the code is debugged and downloaded to the microcontroller. Afterward, the STM32CubeIDE interface turns to the "Debug" perspective.





- As the debugged C code is downloaded to the microcontroller, the next step is its execution. At this stage, the code will be executed till the beginning of the main function. The microcontroller waits for further command. Buttons for program execution are located on the toolbar as in Fig. 3.5.
- The name of each button can be observed by moving the cursor over it. These buttons and their function are explained briefly in the below list.
- **Skip all breakpoints**: Disable all previously set breakpoints. We will talk about what a breakpoint is in the following paragraphs.
- **Terminate and relaunch**: If a modification is done in the program while in debug mode, this button rebuilds and debugs it.
- **Resume**: Resumes execution of the code from the last executed location. Execution of the code continues until a breakpoint when this button is clicked.
- **Suspend**: Halts execution of the code. All windows used to observe software and hardware are updated with recent data.
- **Terminate**: Ends execution of the code. The debug session is also ended by this command.
- **Disconnect**: Disconnect from the target device.
- **Step into**: Executes the next line of code. If this line calls a function, the compiler executes the next line in it. Then, it stops.
- **Step over**: Executes the next line of code. If this line calls a function, the compiler executes the function completely. Then, it stops.
- **Step return**: Completes execution of the function and exits the code.
- **Instruction stepping mode**: The C code is executed based on its generated assembly code within the step into, step over, and step return commands

- Observing variables, registers, or memory is important while developing the project. Code execution should halt in order to perform this operation. A breakpoint should be added to stop execution of the code at a specific code line. To do so, left-click on the desired code line and select “Add Breakpoint. . . ” from the popup window. A new window appears asking for the breakpoint type. Here, we can select the “Regular” type. As the breakpoint is added, a blue circle will appear by the code line to indicate that there is a breakpoint there. The inserted breakpoint can be deleted by double-clicking on it.
- The “Expressions” window (given in Fig) can be used to observe selected variables. In order to add a variable to this window, select the variable to be observed and right-click on it. Select the “Add Watch Expression” option in the opening list. Then, click “OK.” The reader can also double-click on the “Add new expression” button in the “Expressions” window and enter the name of the variable to the opened box

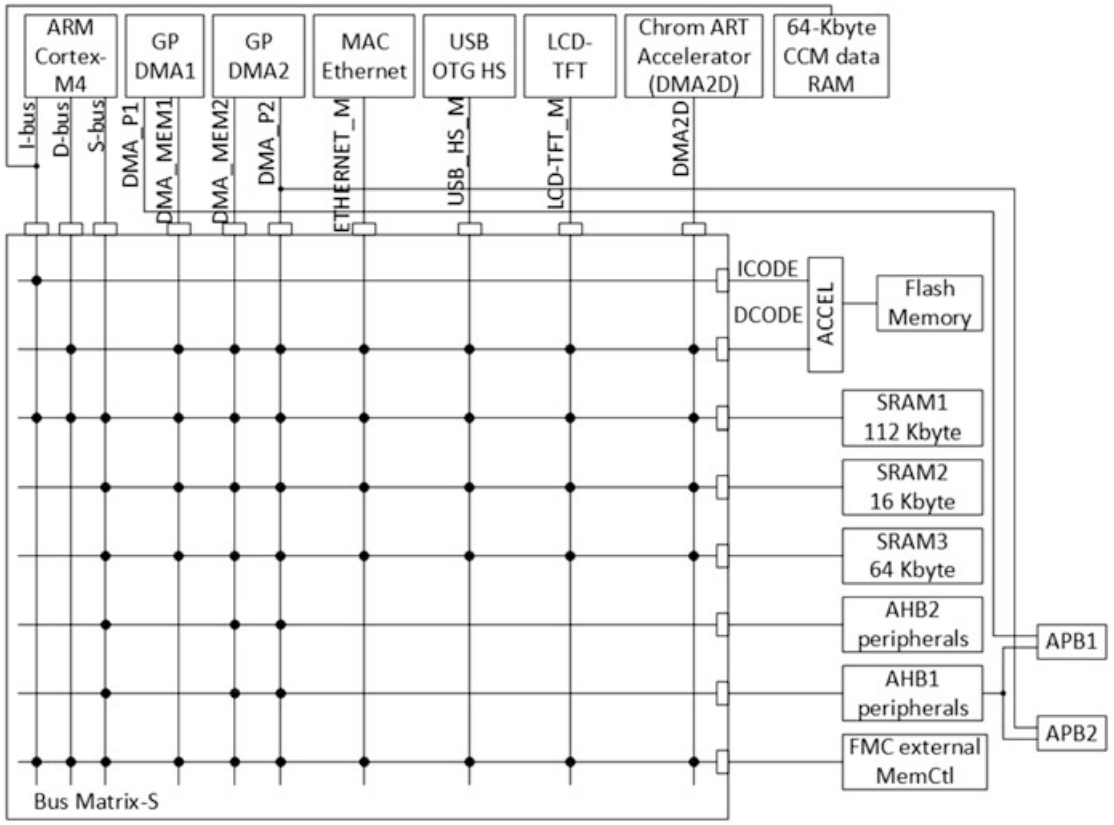
Expression	Type	Value
(x)= a	int	1
(x)= b	int	2
(x)= c	int	0
(x)= d	int	0

- We can define a variable either as local or global in C language. As the name implies, the global variable is available to all code sections. The local variable is only available to the function it is defined in. We will explore these variable types in detail. Although local and global variables can be observed in the “Expressions” window, local variables can also be observed in the “Variables” window. Here, all local variables are automatically added to the mentioned window.
- There is also “Live Expressions” window which can be enabled from the “Window/Show View” list besides other options. In this window, variables are updated in real time while executing the code. Hence, value of a variable can be observed without stopping the code.

Bus Architecture

- A bus is composed of parallel wires in which data and instructions are transferred between microcontroller units. Arm® microcontrollers use advanced microcontroller bus architecture (AMBA) for this purpose. AMBA controls all data transfer between microcontroller units. Here, the unit which initiates data transfer is called master. The unit which responds to master is called slave.
- There are two different bus structures in AMBA. These are called advanced high-performance bus (AHB) and advanced peripheral bus (APB). AHB supports high-speed and high bandwidth operations. Therefore, it is generally used by CPU, DMA modules, and external and internal memory units. AHB supports pipelined operations and burst transfer. Burst transfer is used to realize multiple data transactions at once. This is especially useful for communicating with slave units with high initial access latency. APB supports low bandwidth operations. This leads to low power consumption. Communication between most peripheral units is performed via this interface.

- The STM32F4 microcontroller has a multilayer bus matrix as in Fig. 9.1. This bus matrix allows connecting several master and slave units. To be more specific, the STM32F4 microcontroller has 10 master and 8 slave units.
- As can be seen in Fig. 9.1, there are three bus types connecting to the bus matrix as instruction bus (I-bus), data bus (D-bus), and system bus (S-bus). The instruction bus is used by the CPU to fetch instructions from flash memory, SRAM, or external memory (wherever the code is running from). The prefetch operation in the adaptive real-time accelerator, to be introduced in Sect. 9.1.4, is also performed via the instruction bus.
- The CPU controls code or data in memory locations by the data bus. The system bus allows the CPU to reach peripheral units or SRAM. Besides, DMA buses connect the related DMA master devices to desired peripheral unit and memory blocks. Hence, DMA can be used for providing high-speed data transfer between peripheral units to memory, memory to peripheral units, and memory to memory without the CPU intervention. We will evaluate these operations in detail in Sect. 9.4.



Memory in General

- The memory in the STM32F4 microcontroller is based on Harvard architecture. Hence, data and instructions are transferred through their own dedicated data and instruction bus.
- The number of possible memory locations is determined by the address bus size. There are $2k$ addressable memory locations for k address bits. AHB and APB of the STM32F4 microcontroller have 32 bit address bus. Hence, there are a total of 4 GByte possible usable address locations. Besides, each address location can keep 1 byte data as will be explained next. Hence, the maximum data size that can be stored in memory is 4 GByte.

- Memory is composed of data storage blocks. Each block has its own address value and content. As the name implies, address of a memory block indicates its unique location in memory. The content is the value stored in that address.
- The minimum memory block size for the STM32F4 microcontroller is 1 byte. In other words, the minimum addressable memory block can have size of 1 byte. Hence, these memory blocks are called byte-addressable.
- For RAM, the addressable memory size can be 1 byte, 2 bytes (half-word), and 4 bytes (word).
- For the flash memory, the addressable memory size can be 1 byte, 2 bytes (half-word), and 4 bytes (word) for the writing operation.
- We can also have 8 byte (double word) addressable memory size in flash memory for reading operation. Since the minimum addressable memory location is 1 byte, half-word and word data are stored in the memory in little endian format.

RAM

- SRAM of the STM32F4 microcontroller is divided into different blocks as tabulated in Table 9.1. In this table, CCM stands for core coupled memory. Backup SRAM can be used as internal EEPROM in standby mode. Read and write operations to SRAM are performed at any clock speed without delay.
- As can be seen in Table 9.1, SRAM1, SRAM2, and SRAM3 form a total of 192 kB SRAM used by global and static variables, heap, and stack.

Table 9.1 SRAM blocks and their properties

Name	Size (kB)	Start address	Accessed by
SRAM1	112	`0x20000000`	All AHB master units
SRAM2	16	`0x2001C000`	All AHB master units except D-bus and I-bus
SRAM3	64	`0x20020000`	All AHB master units except D-bus and I-bus
CCM data RAM	64	`0x10000000`	D-bus
Backup SRAM	4	`0x40024000`	All AHB master units

- Heap is a specific RAM region in which dynamic memory allocation can be performed.
- Stack is the specific memory region initially located at the end of usable SRAM. Here local variables and temporary data are stored during code execution. Data is moved to and removed from stack in last-in first-out (LIFO) order. This means last data moved to stack is removed first.
- Stack memory allocation and deallocation is done automatically in C and C++ languages. However, stack should be used carefully to prevent stack overflow. This overflow occurs when variables added to the stack exceed its free area during code execution. For such cases, either existing variables or critical code segments in nearby RAM regions are overwritten. This may cause serious problems.

Flash Memory

- The STM32F4 microcontroller has 2 MB flash memory divided into two banks, each having 1 MB space. Each bank is further divided into 12 sectors. Hence, sectors from 0 to 11 are in bank 1 and sectors from 12 to 23 are in bank 2. For each bank, first four sectors are 16 kB, fifth sector is 64 kB, and remaining sectors are 128 kB. Starting address of flash memory (beginning of sector 0) is 0x08000000 and end address (end of sector 23) is 0x081FFFFF. Flash memory erasing operation can be done at sector level, bank level, or whole (mass flash erase).
- The CPU has higher clock speed compared to flash memory. Therefore, when CPU performs an operation connected to flash memory, it should wait for the end of operation. This is called wait state.

- The CPU has higher clock speed compared to flash memory. Therefore, when CPU performs an operation connected to flash memory, it should wait for the end of operation. This is called wait state. To be more specific, when reading data or code from flash memory, there is a relation between HCLK frequency, supply voltage level, and number of flash wait states. If the supply voltage is kept constant, then the wait state increases with respect to the HCLK frequency. As an example, when the HCLK frequency is in the range 0–30 MHz, the wait state will be zero clock cycle. When the HCLK frequency is in the range 150–180 MHz, the wait state becomes five clock cycles. Hence, increasing the clock speed does not directly increase performance. To overcome this problem, the STM32F4 microcontroller has the adaptive real-time (ART) accelerator unit. Normally, 128-bit read operations can be performed sequentially and flash is stalled during number of wait states between each read. The ART accelerator uses an instruction prefetch queue and branch cache. Hence, latency between successive reads may be prevented. As a result, 0 wait state code execution from flash memory may be achieved at CPU clock frequencies up to 180 MHz

- The STM32F4 microcontroller also has other memory regions besides its 2 MB flash memory. These are tabulated in Table 9.2. In this table, OTP stands for onetime programmable. System memory region is used to boot the device in system memory boot mode. As the name implies, OTP bytes can be programmed only once and cannot be rewritten again. Option bytes region is used to configure read and write protection, brown-out reset (BOR) level, watchdog module, and dual bank boot mode. When the device is in standby or stop mode, it can also be used to adjust the reset operation.

Name	Size (bytes)	Start address
System memory	32,768	`0x1FFF0000`
OTP bytes	528	`0x1FFF7800`
Option bytes, bank 1	16	`0x1FFFC000`
Option bytes, bank 2	16	`0x1FFEC000`

Memory Management in C and C++ Languages

- We will consider memory management in C and C++ languages in this section. Since both languages share common properties, we handle them together. We will start memory management with RAM and flash memory modification. Then, we will focus on pointers and their usage for memory management. Afterward, we will handle local and global variables in C and C++ languages.

Lab2 solution

Memory filling

```
1   Loc1    EQU     0x20000400
2   Loc2    EQU     0x30002000
3
4
5   memPrep LDR     R0, =Loc1
6           LDR     R1, =0xF0F3F7F1
7           STR     R1, [R0]
8           ADD     R0, R0, #4
9           LDR     R1, =0xFaFbFcFd
10          STR     R1, [R0]
11          ADD     R0, R0, #4
12          LDR     R1, =0xF1FaFeF3
13          STR     R1, [R0]
14          ADD     R0, R0, #4
15          LDR     R1, =0xFfFcF9F1
16          STR     R1, [R0]
17
```

```
1   Loc1    EQU     0x20000400
2   Loc2    EQU     0x30002000
3
4
5   memPrep LDR     R0, =Loc1
6           LDR     R1, =0xF0F3F7F1
7           STR     R1, [R0]
8           ADD     R0, R0, #4
9           LDR     R1, =0xFaFbFcFd
10          STR     R1, [R0]
```

```
main    MOV     R1, #0x0 ; summation
        MOV     R2, #0x0
        MOV     R3, #0x1 ; mask
        LDR     R0, =Loc1
        LDR     R2, [R0]
        MOV     R7, #2 ; dış indeks


loop0   LDR     R2, [R0]
        MOV     R6, #4; indeks

loop1   AND     R5, R3, R2
        ADD     R1, R1, R5
        LSR     R2,R2, #8
        SUB     R6,R6, #1
        CMP     R6, #0
        BNE     loop1

        ADD     R0, R0,#4
        SUB     R7, R7, #1
        CMP     R7, #0
        BNE     loop0

result  LDR     R0, =Loc2
        STR     R1, [R0]; tot num of odd numbers
```

```
main    MOV     R1, #0x0 ; summation
        MOV     R2, #0x0
        MOV     R3, #0x1 ; mask
        LDR     R0, =Loc1
        LDR     R2, [R0]
        MOV     R7, #4 ; dış indeks


loop0   LDR     R2, [R0]
        MOV     R6, #4; indeks

loop1   AND     R5, R3, R2
        ADD     R1, R1, R5
        LSR     R2,R2, #8
        SUB     R6,R6, #1
        CMP     R6, #0
        BNE     loop1

        ADD     R0, R0,#4
        SUB     R7, R7, #1
        CMP     R7, #0
        BNE     loop0

result  LDR     R0, =Loc2
        STR     R1, [R0]; tot num of odd numbers
```

ee2004

Week5

Pointer-Based Operations

Pointers and pointer arithmetic allow reaching and modifying a specific memory address in C and C++ languages.

This is one of the most powerful and possibly confusing properties of C language. Since we are using pointers in the STM32F4 microcontroller, we can grasp the basic idea behind them more easily as we can observe the memory map of the microcontroller via STM32CubeIDE.

This will be of great help in understanding the usage of pointers and pointer arithmetic.

Next, we provide several usage areas of pointers in C language. We can revisit pointer-based operations in C++ language in Mbed Studio as well.

Pointer to a Variable

- We provide a pointer definition and usage in Listing 9.1. In this code, we define an integer variable a and assign value 3 to it. In fact, when we make such a definition and assignment, the variable is kept in a specific memory address. In order to reach this memory address, we define a pointer aPointer with integer type. This pointer can keep memory address of the variable a by the code line aPointer=&a.

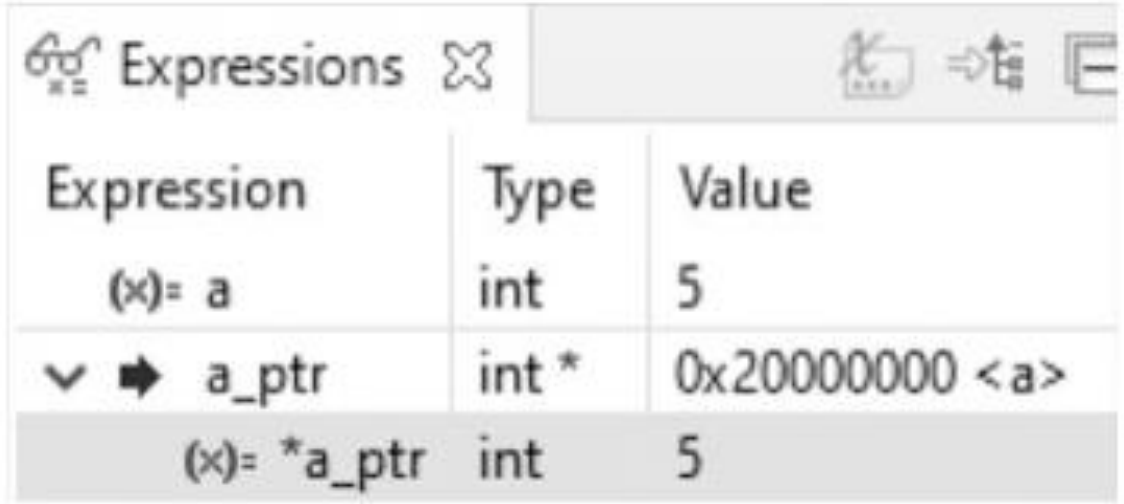
Listing 9.1 Pointer to a variable

```
int a = 3;
int *aPointer;

int main(void)
{
aPointer = &a;
*aPointer = 5;

while (1);
}
```

- For our case, this memory address is 0x20000000 as can be seen in the “Expressions” window in Fig. 9.2 after the code is executed in STM32CubeIDE. We can indirectly reach and modify this memory address content by the code line *aPointer=5. Hence, the a value is modified as can be seen in Fig. 9.2.



Pointer to a Pointer

- The pointer is itself another variable kept in memory. We can reach this memory address by using pointer to a pointer. This method is also called double pointer in literature. To note here, this is an advanced and powerful operation. We only use it for reaching the pointer address in this book.
- We provide the pointer to a pointer (double pointer) usage in Listing 9.2. In this code, we extend our previous example in Listing 9.1 such that we define the pointer to a pointer by the code line int **pPointer and equate it to the address of aPointer by the code line pPointer = &aPointer.

Listing 9.2 Pointer to pointer usage

```
int a = 3;
int *aPointer;
int **pPointer;

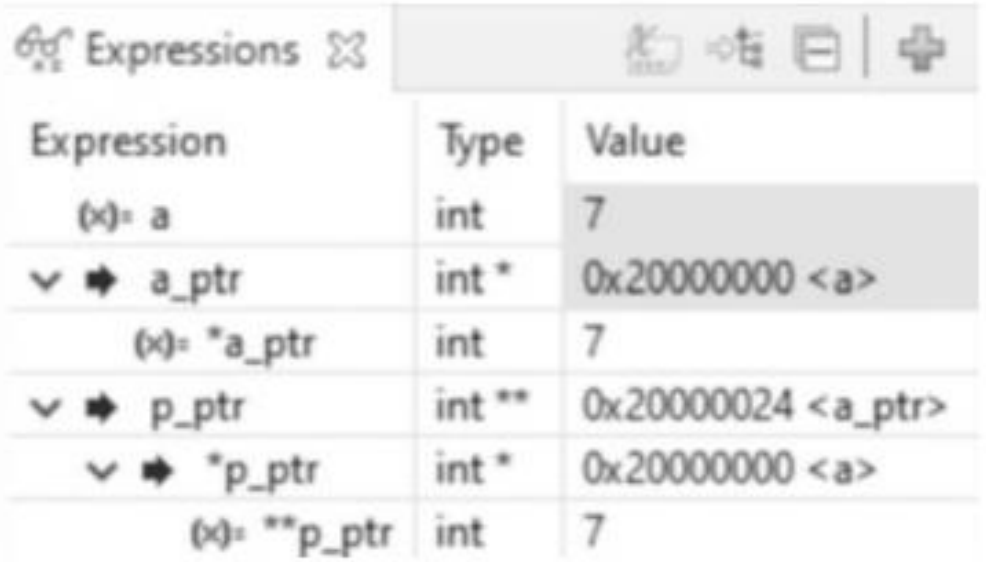
int main(void)
{
aPointer = &a;

pPointer = &aPointer;

*aPointer = 5;

**pPointer = 7;
```

- As we run the code in STM32CubeIDE, we can observe that the aPointer is kept in the memory address 0x20000024 as can be seen in the “Expressions” window in Fig. 9.3.
- In Listing 9.2, we modify the value of our variable a by the pointer to pointer usage as **pPointer=7.



Reaching a Specific Memory Address by Pointers

- Pointers can be used to reach a specific memory address in the STM32F4 microcontroller. We provide two methods for this purpose in this section. The first method is based on pointer arithmetic and casting operation. We provide such an example in Listing 9.3.

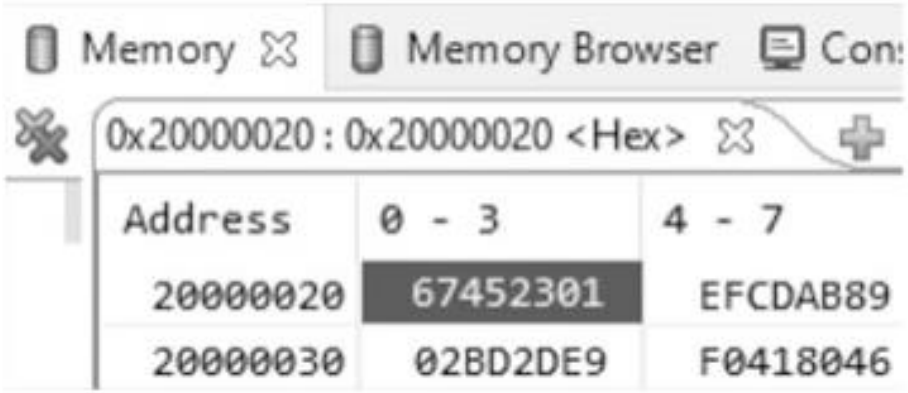
Listing 9.3 Reaching a specific memory address using pointer arithmetic and casting operation, the C code

```
unsigned int *memLoc;

int main()
{
memLoc = (unsigned int *)0x20000020;

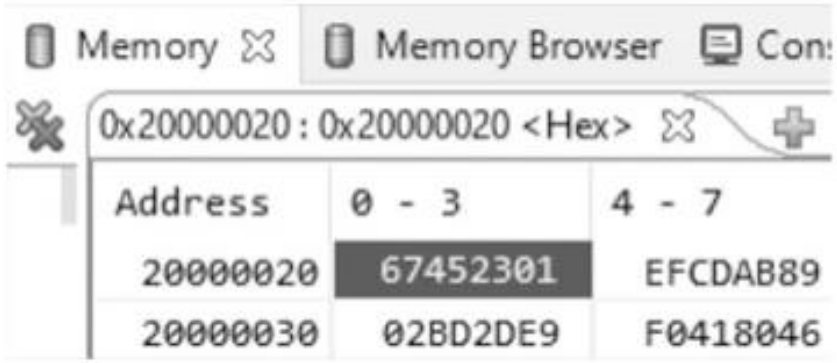
*(memLoc) = 0x01234567;
*(memLoc + 1) = 0x89ABCDEF;

while (1);
}
```



- In this code, we modify the memory addresses 0x20000020 and 0x20000024 via pointer arithmetic and casting operation. We provide the memory map after this operation in Fig. 9.4. As can be seen in this figure, the memory content has been modified as expected.

- The second method in reaching a memory address is using the define statement. We provide such an example in Listing 9.4. Here, we modify the memory addresses 0x20000020 and 0x20000024 via define statement. The memory map after this operation will again be as in Fig. 9.4.



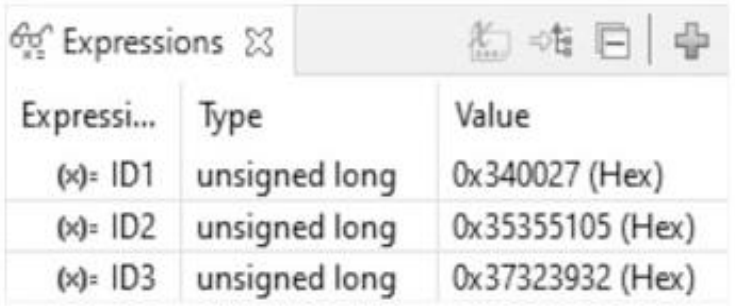
Listing 9.4 Reaching a specific memory address using the define statement, the C code

```
#define memLoc1 (*(unsigned long *)0x20000020)
#define memLoc2 (*(unsigned long *)0x20000024)

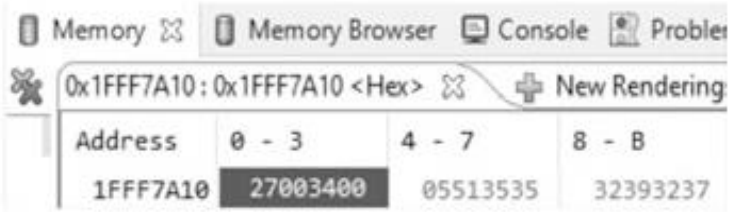
int main(void)
{

memLoc1=0x1234567;
memLoc2=0x89ABCDEF;

while (1);
}
```



(a)



(b)

- The second method allows obtaining the unique identification number (ID) of the STM324 microcontroller. As a side note, each STM microcontroller has its unique ID composed of 96 bits assigned to it. We provide the sample code for this operation in Listing 9.5. The obtained unique ID for our microcontroller can be observed both in the “Expressions” window and “Memory map” as in Fig. 9.5.

Listing 9.5 Obtaining unique ID of the STM32F4 microcontroller,

```
#define ID1 (*(unsigned long *)0x1FFF7A10)
#define ID2 (*(unsigned long *)0x1FFF7A14)
#define ID3 (*(unsigned long *)0x1FFF7A18)

int main(void)
{

while (1);
}
```

Pointers and Arrays

- When an array is defined in C language, it is in fact treated as pointer. We provide one such example in Listing 9.6. In this code, we first define an integer array, a, with entries {1, 2, 3, 4, 5}. We then assign the array to the pointer with the code line aPointer=a. We next reach the second element of the array and modify it by the code line *aPointer = 0. We can observe the effect of this operation in the “Expressions” window in Fig. 9.6. This window also provides another insight on the array usage. As can be seen there, the array a is kept by its starting address only. Hence, we can use it as a pointer.

Listing 9.6 Array usage as pointer example

```
int a[5] = {1, 2, 3, 4, 5};
int *aPointer;

int main(void)
{

aPointer = a;
aPointer += 1;
*aPointer = 0;

while (1);
}
```

Expressions

Expression	Type	Value
a	int [5]	0x20000000 <a>
(x)= a[0]	int	1
(x)= a[1]	int	0
(x)= a[2]	int	3
(x)= a[3]	int	4
(x)= a[4]	int	5
a_pointer	int *	0x20000004 <a+4>
(x)= *a_pointer	int	0

Pointer to a Structure

- Structure (struct) is a powerful tool in representing data in grouped form. We will be using structures in digital image processing operations. We can use pointers in reaching a specific field in a structure.

Listing 9.7 Structures and pointer usage on them,

```
#include <stdint.h>

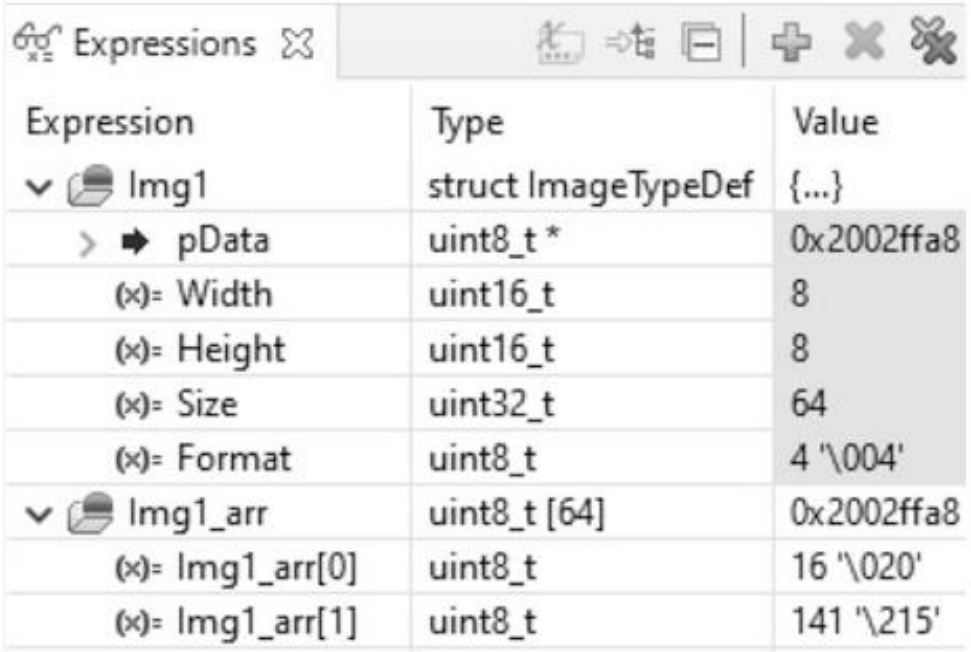
struct ImageTypeDef
{
uint8_t *pData;
uint16_t width;
uint16_t height;
uint32_t size;
uint8_t format;
};

int main(void)
{

struct ImageTypeDef img1;
uint8_t img1Arr[64];

img1.pData = img1Arr;
img1.width = 8;
img1.height = 8;
img1.size = 64;
img1.format = 4;
img1.pData[5] = 8;

while (1);
}
```



- We provide an example on structure usage in Listing 9.7. Here, we define an image struct, ImageTypeDef, with five fields as *pdata, width, height, size, and format. Then, we create an image struct as img1 and assign values to its elements. We also provide how a pointer is used to reach specific field elements in the structure in the same example. We provide the “Expressions” window in Fig. 9.7 as we run the code in STM32CubeIDE.

Function Call by Reference

- Pointers allow passing array addresses directly to a function. This is function call by reference which allows modifying a specific array element in the function. Here, we do not transfer all array elements to the function and receive them back afterward. This becomes extremely important while processing long arrays such as images or audio signals.

Listing 9.8 The first function call by reference example,

```
#include <stdint.h>

char a[5] = {1, 2, 3, 4, 5};
char *aPointer;

void replace(char *pointer);

int main()
{

aPointer = a;
replace(aPointer);

while (1);
}

void replace(char *pointer)
{
pointer += 1;
*pointer = 0;
}
```

- We provide two examples on the usage of function call by reference next. In the first example, given in Listing 9.8, the function replace is used to replace the second element of any array with value 0. Here, only starting address of the array is passed to the function instead of passing all its elements.

- In the second example, given in Listing 9.9, the function squareFunc is used to take square of the array elements. Here, only the starting address and size of the array are fed to the function instead of passing all array elements. Size of the array is fed to the function since it must know the number of array elements to operate on.

Listing 9.9 The second function call by reference example, the C code

```
char a[5] = {1, 2, 3, 4, 5};
char *aPointer;

void squareFunc(char *pointer, int size);

int main()
{
aPointer = a;
squareFunc(aPointer, sizeof(a));

while (1);
}

void squareFunc(char *pointer, int size)
{
int i;

for (i = 0; i < size; i++)
{
*(pointer + i) *= *(pointer + i);
}
}
```

Function Pointers

- Pointers can also be used to call functions. This is different from previous applications such that we call a code script to be executed by an assigned pointer to it. There are two main reasons for the usage of such a method. The first usage area is when a callback function is associated with its interrupt source in Mbed. A simple example for this usage is the code line button.rise(&toggle()). Here, toggle()) is the callback function associated with the button rise interrupt. The second usage area is forming a function which can perform the same operation on different variable types.

- We provide a function pointer usage example in Listing 9.10. This code consists of a function to modify array elements as desired. We feed start address of the array, size of the array, size of each array element, and the function (dummy) to perform the desired change as a pointer. The (dummy) function is called and operated on the given data. However, we do not know what this (dummy) function is until it is called. In our example, we set this (dummy) function as the one taking square of the array elements.

Listing 9.10 Function pointer usage example, the C code

```
/* USER CODE BEGIN PV */
float arr[5] = {1.7, 2.8, 3.6, 4.5, 5.2};
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void change(void *arr, size_t arrSize, size_t elem
    dummy)(const void *))
{
char *ptr = (char *)arr;

int i;
for (i = 0; i < arrSize; i++)
{
dummy(ptr + i * elemSize);
}
}

void squareFunc(const void *x)
{
*(float *)x = (*(float *)x) * (*(float *)x);
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
change(arr, 5, sizeof(float), squareFunc);
/* USER CODE END 2 */
```

Local, Global, and Static Variables

- We can define a variable either as local or global in C and C++ languages. As the name implies, a global variable is available to all code blocks. However, a local variable is only available to the function it is defined in. These two variables are kept in different memory locations in RAM. Global variables are kept starting from the lowest possible memory address (0x20000000) in SRAM. As a new global variable is added, the memory address is incremented and the new variable is saved there. On the other hand, local variables are kept in the highest memory address in the stack. For STM32F4 microcontroller, this address is initially 0x2002FFFF which is the end of SRAM. Based on the definition of stack, when a new local variable comes, it is saved to the recent memory address. The address value is decremented afterward. Hence, local variables are saved from top to bottom in SRAM in stack. It is important to note that the C and C++ languages take main() as a function. Hence, a variable defined within the main function is also treated as local. Therefore, a global variable should be defined before the main() declaration.

- We should also explain how a static variable works in C and C++ languages. Such a variable is created with the static keyword and kept in RAM during code execution. Even if a static variable is declared as local, it is not kept in stack. Instead, the static variable is placed at the beginning of RAM as with global variables. The importance of the static variable is as follows. It can be defined in a function. Hence, it will be local to that function. However, whenever the function is called more than once, the value of the static variable is not lost unlike other local variables.

- We can observe how global and local variables are stored in memory by the help of pointer usage. We provide one simple example in C language in Listing 9.11. Here, the variable a is defined as global and the variable b is defined as local. We provide the “Expressions” window after executing the code in Fig. 9.8. As can be seen in this window, the two variables are kept in different memory locations. We can observe this effect by pointer usage.

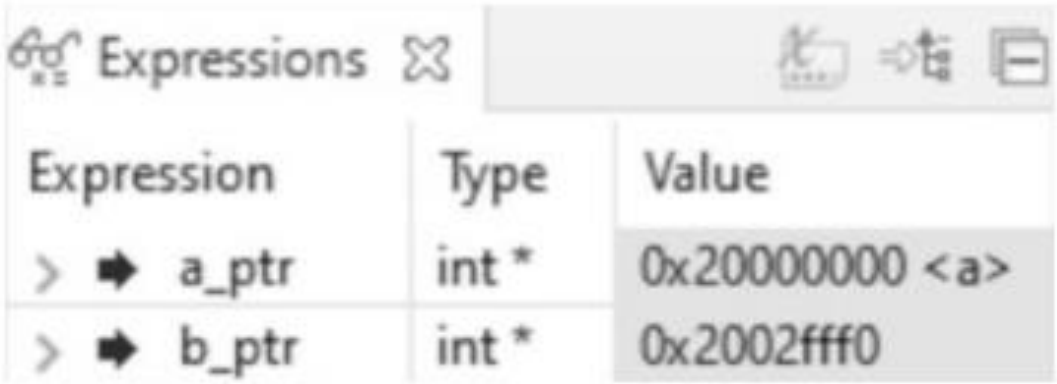
Listing 9.11 Local and global variables, the C code

```
//Global variable
int a = 1;
int *aPointer;

int main(void)
{
//Local variable
int b = 2;
int *bPointer;

aPointer = &a;
bPointer = &b;

while (1);
}
```





ee2004

week6

Data Types in C and C++ Languages

- In general C programming textbooks, we see *char*, *short*, *int*, *long*, *float*, and *double* data types. We need to examine the size of C data types in the light of 32-bit processors such as Arm.
- The C standards do not specify the size of data types. The compiler designers are free to decide the size for each data type. The *float* and *double* data types are standardized by the IEEE754.
- The sizes of char and short are often set at 1 byte and 2 bytes.
- The size of int is often depending on the data size of the CPU but rarely go below 16 or above 32.
- The sizes of long and long long are implemented the same way everywhere.
- Use pseudo function sizeof(): C compilers supports a pseudo function sizeof(), which returns the size of the parameter in the number of byte(s). The parameter may be a data type or a variable. For example, sizeof(int) returns the number of bytes in an int variable.

Data type Size Range

- **char** 1 byte -128 to 127
- **unsigned char** 1 byte 0 to 255
- **short int** 2 bytes -32,768 to 32,767
- **unsigned short int** 2 bytes 0 to 65,535
- **int** 4 bytes -2,147,483,648 to 2,147,483,647
- **unsigned int** 4 bytes 0 to 4,294,967,295
- **long** 4 bytes -2,147,483,648 to 2,147,483,647
- **unsigned long** 4 bytes 0 to 4,294,967,295
- **long long** 8 bytes -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- **unsigned long long** 8 bytes 0 to 18,446,744,073,709,551,615

These integer data types are defined in a header file called *stdint.h*. You need to include this header file in order to use these data types

Data type	Size	Range
int8_t	1 byte	-128 to 127
uint8_t	1 byte	0 to 255
int16_t	2 bytes	-32,768 to 32,767
uint16_t	2 bytes	0 to 65,535
int32_t	4 bytes	-2,147,483,648 to 2,147,483,647
uint32_t	4 bytes	0 to 4,294,967,295
int64_t	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
uint64_t	8 bytes	0 to 18,446,744,073,709,551,615

Bit-wise Operations in C

- One of the most important and powerful features of the C language is its ability to perform bit manipulations.
- While every C programmer is familiar with the logical operators AND (&&), OR (||), and NOT (!),
- many C programmers are less familiar with the bitwise operators AND (&), OR (|), EX-OR (^), invert (~), right shift (>>), and left shift (<<).
- These bit-wise operators are widely used in software engineering for embedded systems and control; consequently, their understanding and mastery are critical in microprocessor-based system design and interfacing.
- 0x35 & 0x0F results in 0x05 /* ANDing */
- 0x04 | 0x68 results in 0x6C /* ORing: */
- 0x54 ^ 0x78 results in 0x2C /* XORing */
- ~0x55 results in 0xAA /* Inverting 0x55 */

Setting and Clearing (masking) bits

- OR can be used to set a bit or bits, and AND can be used to clear a bit or bits.
- Anything ‘OR’ed with a 1 results in a 1;
- anything ‘OR’ed with a 0 results in no change.
- Anything ‘AND’ed with a 1 results in no change;
- anything ‘AND’ed with a 0 results in a zero.
- Anything ‘EX-OR’ed with a 1 results in the complement;
- anything ‘EX-OR’ed with a 0 results in no change.

```c
#include <stdio.h>

int main(void) {
volatile unsigned char temp; /* declare volatile otherwise the optimizer will
remove it. */
temp = 0x35 & 0x0F; /* ANDing : 0x35 & 0x0F = 0x05 */
temp = 0x04 | 0x68; /* ORing : 0x04 | 0x68 = 0x6C */
temp = 0x54 ^ 0x78; /* XORing : 0x54 ^ 0x78 = 0x2C */
temp = ~0x55; /*Inverting : ~0x55 = 0xAA */
while (1);
return 0;
}
```

```c
int main(void)
{
unsigned char var1;

while(1)
{
var1 = var1 | 0x10; /* Set bit 4 (5th bit) of var1 */
var1 = var1 & 0xEF; /* Clear bit 4 (5th bit) of var1 */
var1 = var1 ^ 0x10; /* Togggle bit 4 (5th bit) of var1 */
}
}
```

Testing bit with bit-wise operators in C

- In many cases of system programming and hardware interfacing, it is necessary to test a given bit to see if it is high or low.
- For example, many devices send a high signal to signify that they are ready for an action or to indicate that they have data available.
- How can the bit (or bits) be tested?
- In such cases the unused bits are masked and then the remaining data is tested.
- EX: Write a C program to monitor bit 5 of var1. If it is HIGH, change value of var2 to 0x55; otherwise, change value of var2 to 0xAA.

```
while(1)
{
if (var1 & 0x20) /* check bit 5 (6th bit) of var1 */
var2 = 0x55; /* this statement is executed if bit 5 is a 1 */
else
var2 = 0xAA; /* this statement is executed if bit 5 is a 0 */
}
```

Bit-wise shift operation in C

- >> data >> number of bit-positions to be shifted right
- << data << number of bit-positions to be shifted left

The following shows some examples of shift operators in C:

1. 0b00010000 >> 3 /* it equals 00000010. Shifting right 3 times */
2. 0b00010000 << 3 /* it equals 10000000. Shifting left 3 times */
3. 1 << 3 /* it equals 00001000. Shifting left 3 times */

Compound Operators

- In C language, whenever the left-hand-side of the assignment operator (=) and the first operand on the right-hand-side are identical we can avoid repeating the operand by using the compound operators.

a = a + 6;	a += 6;
a = a – 23;	a –= 23;
y = y * z;	y *= z;
z = z / 25;	z /= 25;
w = w | 0x20;	w |= 0x20;
v = v & mask;	v &= mask;
m = m ^ togBits;	m ^= togBits;

Review Questions

1. What is result of 0x2F & 0x27?
2. What is result of 0x2F | 0x27?
3. What is result of 0x2F ^ 0x27?
4. What is result of 0x2F >> 3?
5. What is result of 0x27 << 4?

Bit-wise operations using compound operators

- The majority of hardware access level code involves
  - setting a bit or bits in a register,
  - clearing a bit or bits in a register,
  - toggling a bit or bits in a register,
  - monitoring the status bits.
- For the first three cases, the operations read the content of the register, modify a bit of bits then write it back to the same register.
- The compound operators are very suitable for these operations.

- To set bit(s) in a register,
- register |= MASK; //where MASK is a number that has ‘1’ at the bit(s) to be set.
- register |= 0x08; //The number 0x08 has a ‘1’ at bit 3, therefore the statement sets bit 3 of the register.
- register |= 0x42; // The number 0x42 has a ‘1’ at bit 6 and bit 1, therefore the statement sets bit 6 and bit 1 of the register.

- To clear bit(s) in a register,
- register &= ~MASK; //where MASK is a number that has ‘1’ at the bit(s) to be cleared.
- register &= ~0x20; //The number 0x20 has a ‘1’ at bit 5, therefore the statement clears bit 5 of the register.
- register &= ~0x12; //The number 0x12 has a ‘1’ at bit 4 and bit 1, therefore the statement clears bit 4 and bit 1 of the register.

Notice the mask for clearing the bits is the same as the mask for setting the bits, where the bits to be modified are ‘1’ and the rest of the bits are ‘0’ except that in clearing the bits, the mask is complemented in the statements.

Using shift operator to generate mask

- With the statements above, one challenge may be to generate the mask with the correct bit(s) set to 1 depending on how proficient you are with converting binary numbers to hexadecimal.
- Some compilers allow you to write a literal binary number in the format of 0b00000000 but since it is not in the C standards, many compilers do not accept this notation.
- One way to ease the generation of the mask is to use the left shift operator.
- To generate a mask with bit n set to 1, use the expression: 1 << n
- If more bits are to be set in the mask, they can be “or” together. To generate a mask with bit n and bit m set to 1, use the expression: (1 << n) | (1 << m)

- Now to set bit 3 of the register, we can rewrite the statement

  register |= 0x08; as register |= 1 << 3;
- And to set bit 6 and bit 1 of the register, we can rewrite the statement

  register |= 0x42; as register |= (1 << 6) | (1 << 1);
- The same goes for clearing bit 5 of the register, we can use the statement

  register &= ~(1 << 5);
- And to clear bit 4 and bit 1 of the register, we can use the statement

  register &= ~((1 << 4) | (1 << 1));

Notice that regardless of setting or clearing bits, the mask always has 1s at the bit locations for the bits to be modified and when multiple bits are used in the mask, they are always 'OR'ed together.

STM Arm I/O Programming;
Using STM32CubeMX to Modify Hardware of the Microcontroller

STM32CubeMX (available under STM32CubeIDE) can be used to control and modify hardware of the STM32F4 microcontroller.

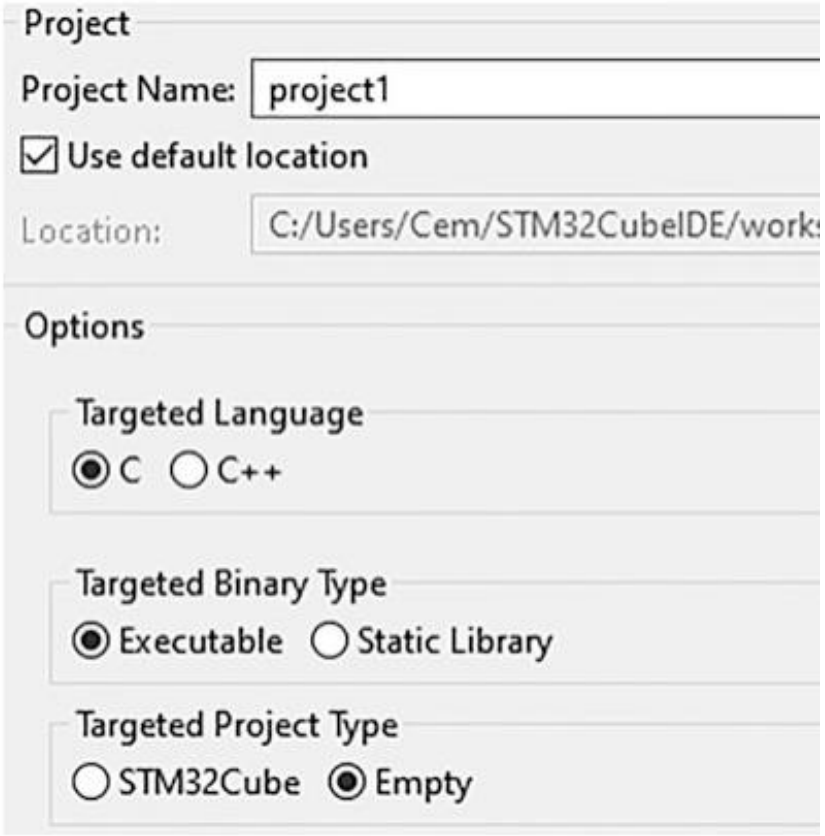
To note here, hardware properties can also be set via available libraries such as hardware abstraction layer (HAL).

The advantage of STM32CubeMX is that operations are done visually on it.

Moreover, it produces a template C code containing predefined functions to be used to control hardware properties.

Creating a New Project Using STM32CubeMX

- In order to modify hardware of the microcontroller, we can create a new project as explained in Sect. 3.1.3 with one difference.
- Since we will modify hardware of the microcontroller, we will set the “Targeted Project Type” as “STM32Cube” in Fig. 3.2.
- Afterward, a new window appears asking for the “Firmware Library Package Setup.” At this step, leave all default settings as they are and press “Finish.”
- The next pop-up window asks whether to initialize all peripheral units with their default mode. Click “No” since we will modify necessary pin properties as we need them.



- A window should open up as in Fig. 3.7.
- We can set all system hardware properties through STM32CubeMX.
- At this step, let's assume that we want to turn on the green LED on the STM32F4 board whenever the user button on it is pressed.

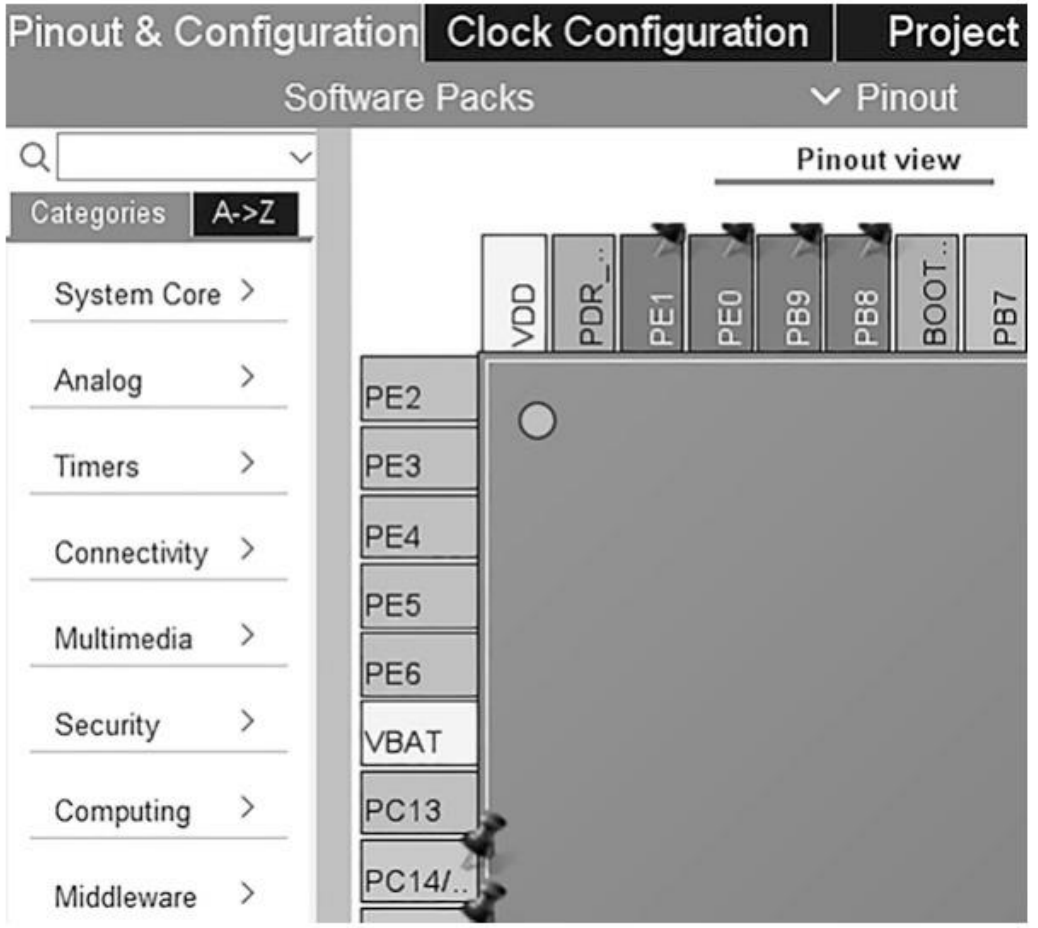


Fig. 3.7 Initial screen for STM32CubeMX

- To do so, we should first modify the microcontroller pin properties.
- Hence, we should set the pin PA0 of the STM32F4 microcontroller as “GPIO_Input” and pin PG13 as “GPIO_Output.”
- Fortunately, these modifications can be done in the STM32CubeMX interface easily.
- Within the "Pinout&Configuration” tab, locate the mentioned pins and left-click on them one by one.
- A pop-up window appears asking for which purpose the pin will be used for.
- Set the pin PA0 as “GPIO_Input” and pin PG13 as “GPIO_Output” and proceed to clock configuration by opening the “Clock Configuration” tab.
- We will benefit from the automatic clock setting property of STM32CubeMX here. To note here, this is the default setting and we suggest using this option whenever possible. Clock setup will be covered in detail in Chap. 6.

Generating and Modifying the Code

- Now, we are ready to generate the code corresponding to the hardware setup.
- Before doing so, we can open the “Project Manager” tab.
- Here, we can set the properties of our project from the “Project,” “Code Generator,” and “Advanced Settings” tabs.
- We suggest keeping all settings as they are since these are generated automatically.
- The next step is generating the code related to the hardware setup. To do so, press the “Save” button.
- A pop-up window appears asking for generating the code. Press “OK” there.

- As the code is generated, we can open the “main.c” file under the “Src” folder.
- As this file is opened, the reader will observe that there are sections labeled “USER CODE BEGIN” and “USER CODE END” asking for the user to add his or her code snippets.
- This setup is done to ensure that other parts of the code related to hardware setup are not modified by mistake.
- Finally, we will test the project. To do so, add the following C code snippet in Listing 3.2 to the appropriate place in the opened “main.c” source file.
- As we debug the project and embed the generated code on the STM32F4 microcontroller, it will be ready to be executed.
- The reader can check whether the code is running or not by pressing and releasing the user button on the STM32F4 board. Every time the button is pressed, the green LED on the board should turn on.

Listing 3.2 The C code snippet to be added

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0))
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
else
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Mbed and Mbed Studio Platforms

- Although STM32CubeIDE allows controlling all microcontroller properties, some users do not need such a detailed setup in their projects. Therefore, we introduce the web-based Mbed and its desktop version Mbed Studio in this section.
- Mbed on web has three main advantages.
- First, it does not require downloading and installing any IDE on PC. It is structured such that the user develops his or her project on a web-based platform.
- Second, Mbed is formed such that a code written for one platform in the Mbed ecosystem can easily be ported to another platform as long as hardware requirements are satisfied. This is possible since Mbed simplifies and generalizes low-level hardware setup and usage. From this perspective, Mbed resembles the popular Arduino platform.
- Third, Mbed allows code sharing between community members since it is an online platform. This is done by posting and importing the generated code through the web-based interface. In fact, the reader can easily use a project posted by another community member by importing it to his or her workspace. Then, this project can be developed further by the reader for his or her own needs.

Managing a Project in Mbed

- “main.cpp”
- We should replace the content of the file by the C++ code in Listing 3.3 for our purposes.

Listing 3.3 The main file for our project under Mbed

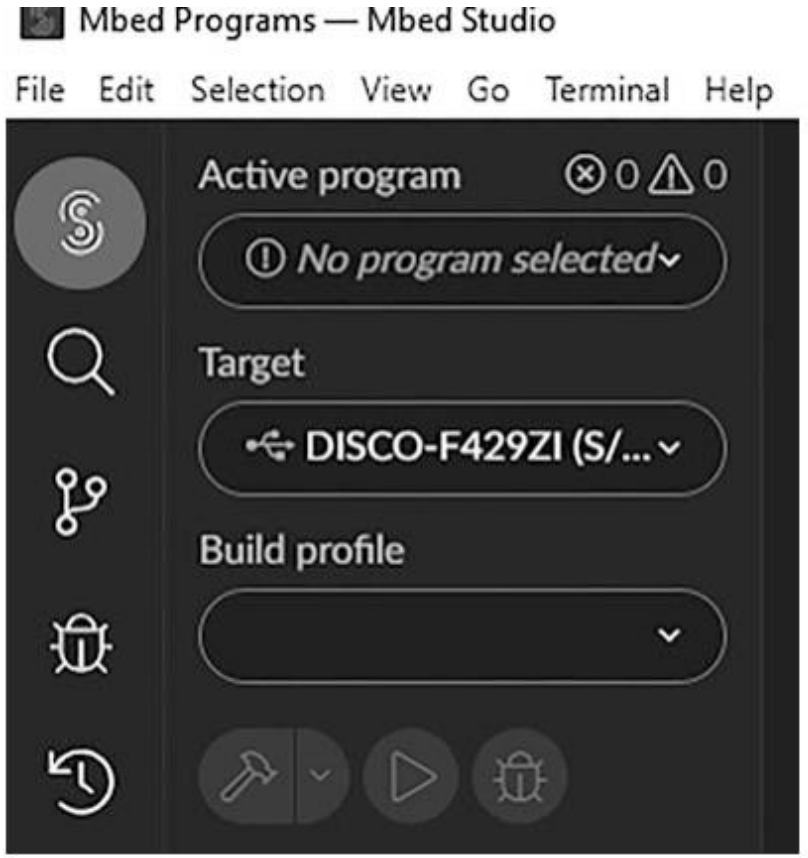
```
#include "mbed.h"

#define WAIT_TIME 500000
DigitalOut greenLED(LED1);

int main()
{
while (true)
{
greenLED = !greenLED;
wait_us(WAIT_TIME);
}
}
```

Mbed Studio on Desktop

- The procedure is straightforward in creating, building, and executing the project under Mbed online. However, the user should have an active Internet connection to perform all these operations. This may not be possible for some cases. Besides, it is not possible to debug the code during its execution. We can benefit from Mbed Studio to overcome these shortcomings.
- Mbed Studio is the desktop version of Mbed. The program will launch and the IDE will be as in Fig. 3.10. As can be seen in this figure, Mbed Studio has a fairly compact interface. There exists a panel on the left side of the window. Through it, the user can create a new project, select the target, and build the project for the selected target. At the center side of the window, the main code file will be opened as it is added to the project. There are tabs which will summarize all build and debug operations at the bottom of the panel.



Arduino IDE

- Arduino is an open-source electronics platform based on a simple hardware and software environment.
- Arduino boards can do what they do by programming the microcontroller on the board. The Arduino IDE is used for this, whichis open-source and is continually growing via a vast worldwide user-base.

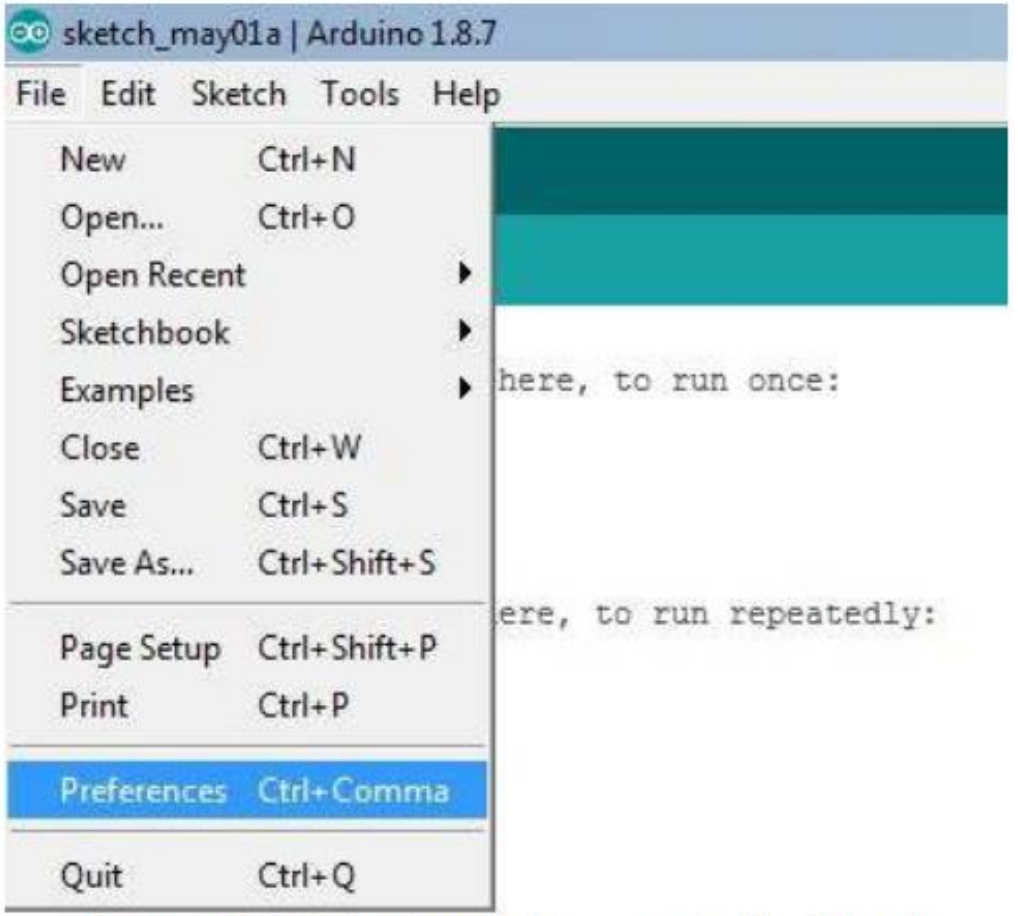


Figure 16.1: Selecting Preferences in the File tab

- In accordance with Figure 16.1 for installing the STM32duino library in the Arduino IDE environment, click the File tab and select preferences.
- In the preferences window, tick the compilation and output boxes (show verbose output during) to generate hex code for programming after successful compilation.
- Furthermore, in additional boards manager URL enter the following web link:

https://github.com/stm32duino/BoardManagerFiles/raw/master/STM32/package_stm_index.json

Click the OK button as shown in Figure 16.2. From the Tools tab and Board section, select the 'boards manage' icon as shown in Figure 16.3.

In the boards manager window, choose type as contributed as shown in Figure 16.4.

Select STM32 Cores by STMicroelectronics and install by pressing the install button as shown in Figure 16.5.

After installation, an INSTALLED message appears as shown in Figure 16.6.

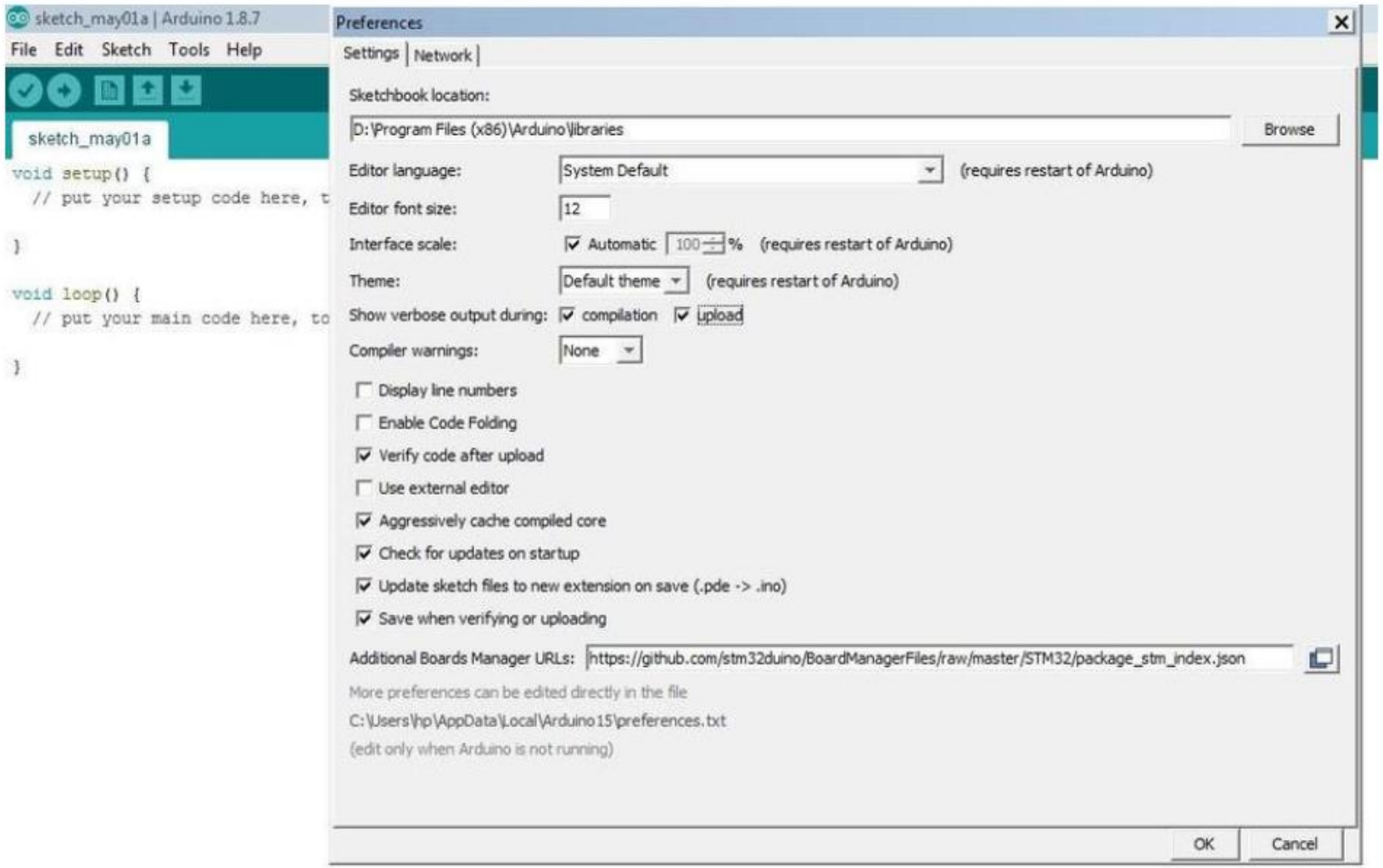


Figure 16.2: Preferences dialog settings

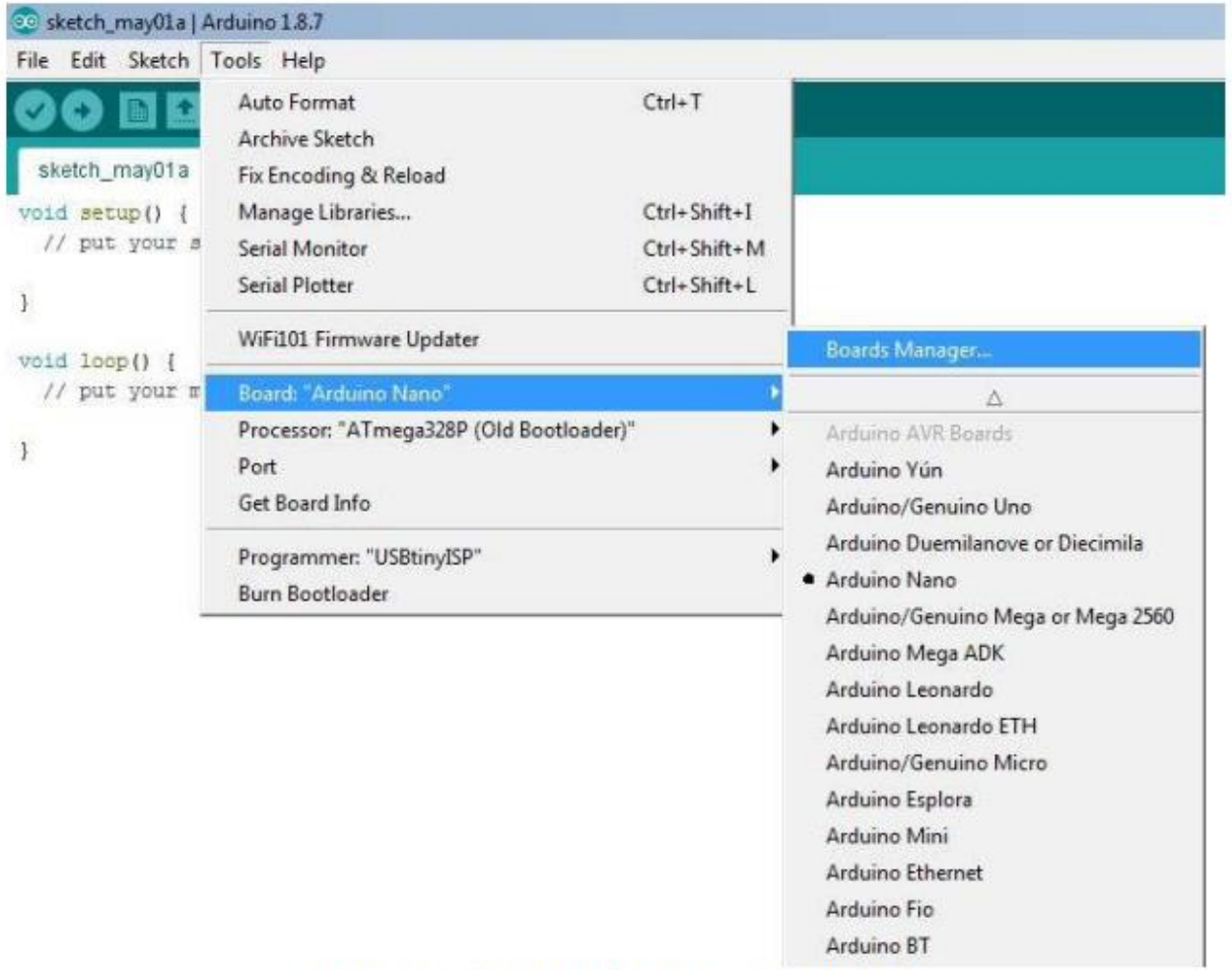


Figure 16.3: Selecting Boards Manager in the Tools tab

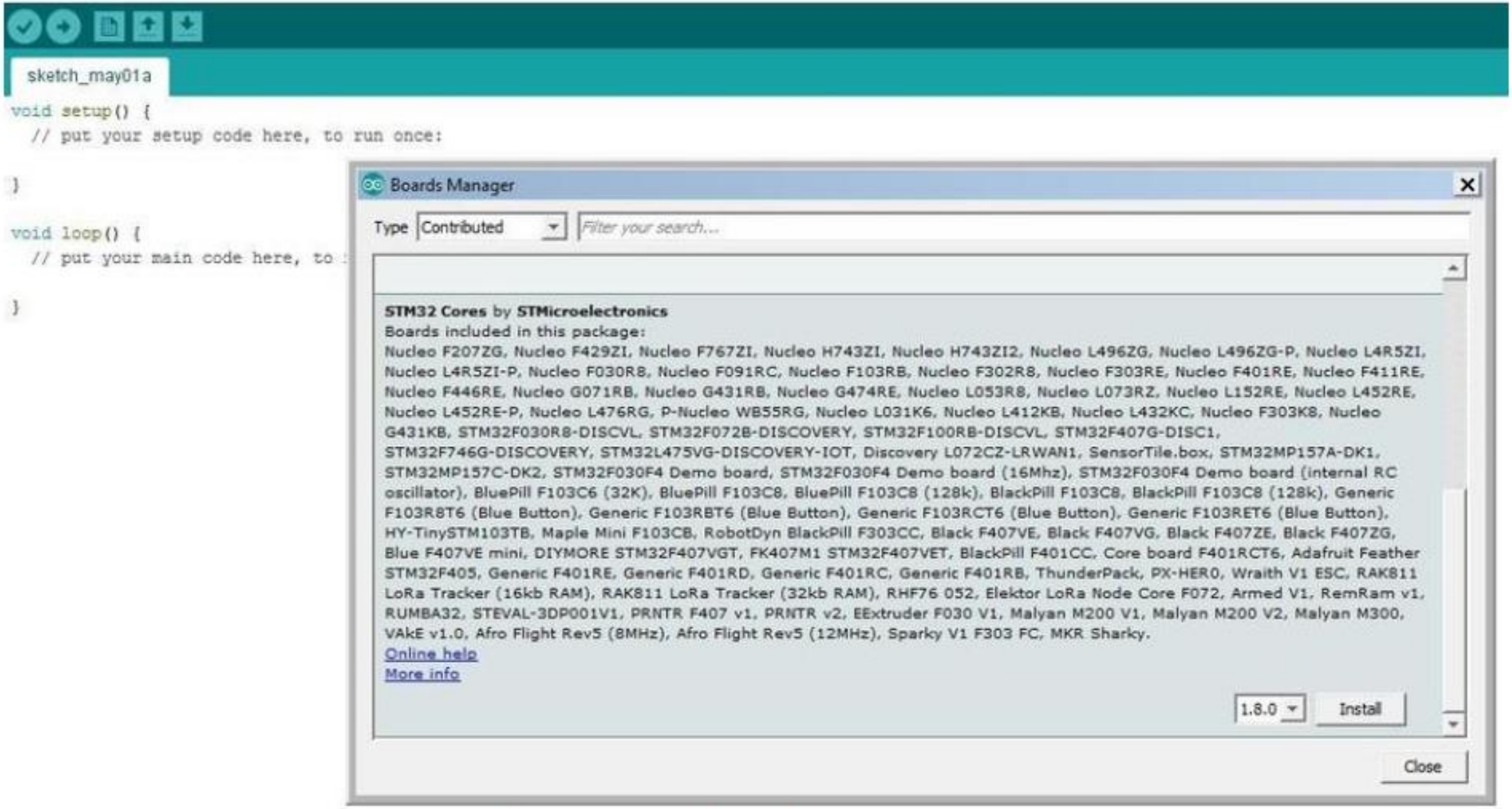


Figure 16.5: Installing STM32 Cores by STMicroelectronics

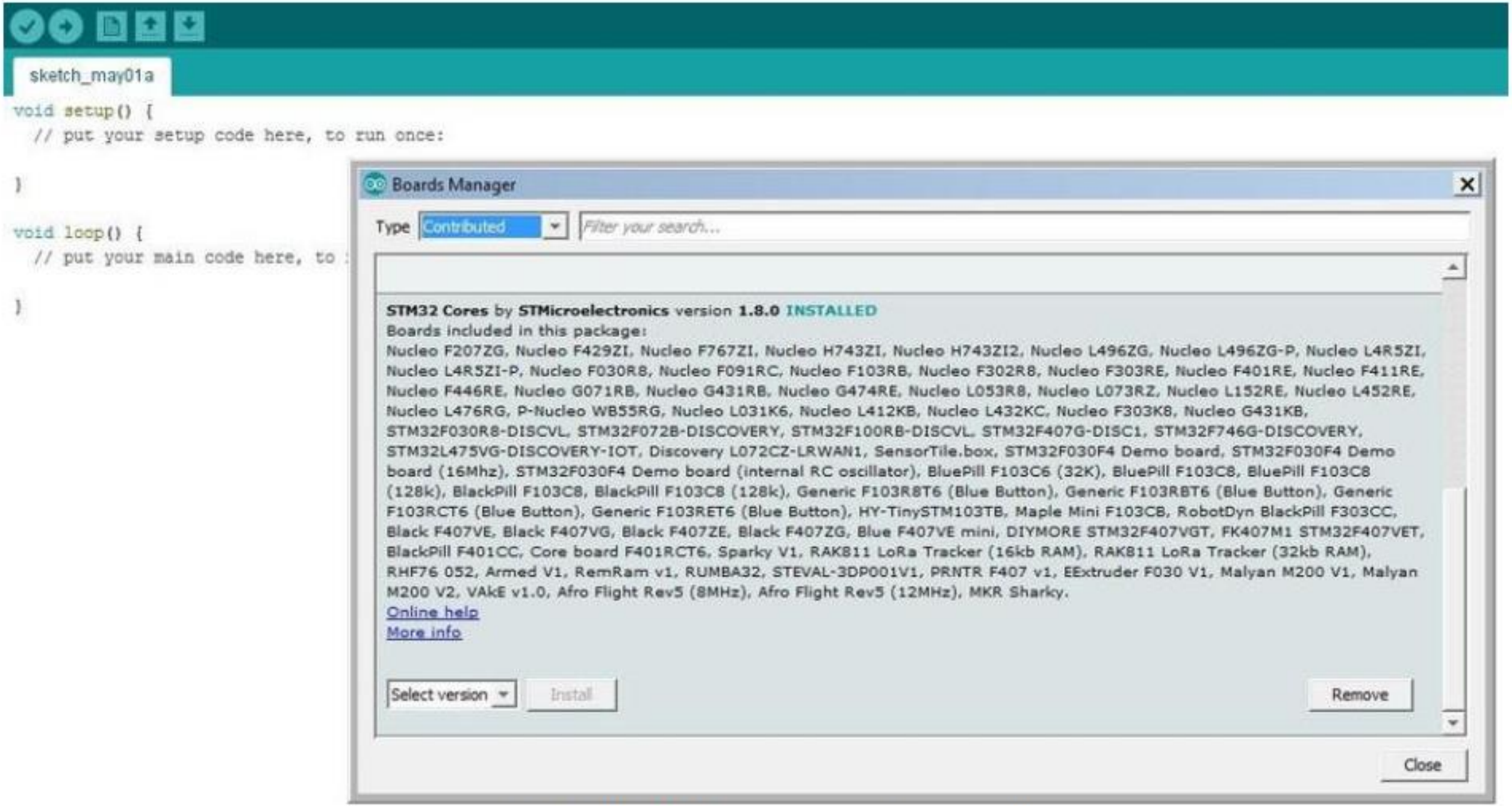


Figure 16.6: INSTALLED message after installation

- We can now check the installed STM32 boards by clicking on the Tools tab, and selecting the board and choosing, for example, the Generic STM32F1 series as shown in Figure 16.7.

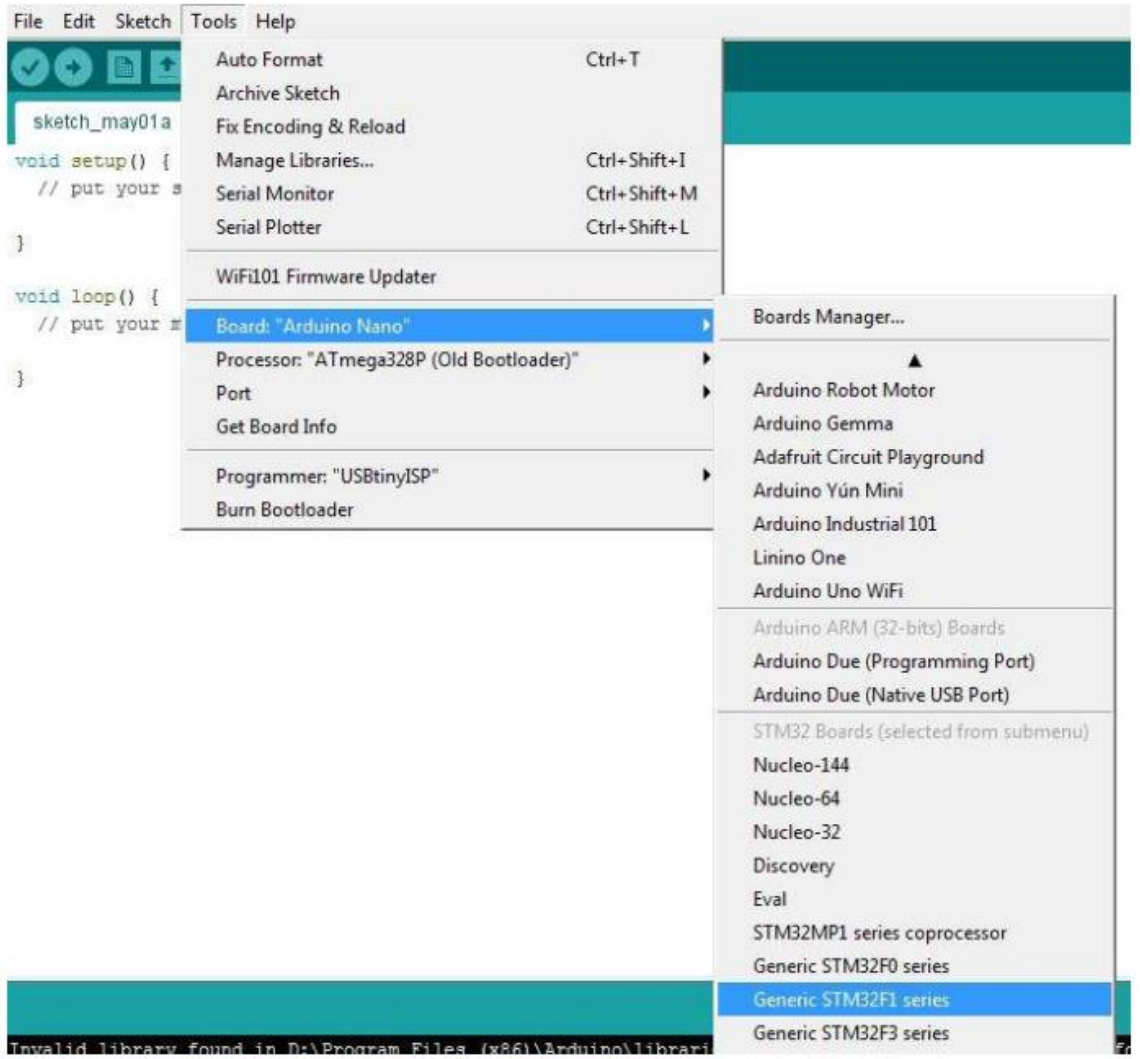


Figure 16.7: Choosing the Generic STM32F1 series from the board section

- If we click on the Tools tab again, we can see the Generic STM32F1 series board in the board part number section as illustrated in Figure 16.8.

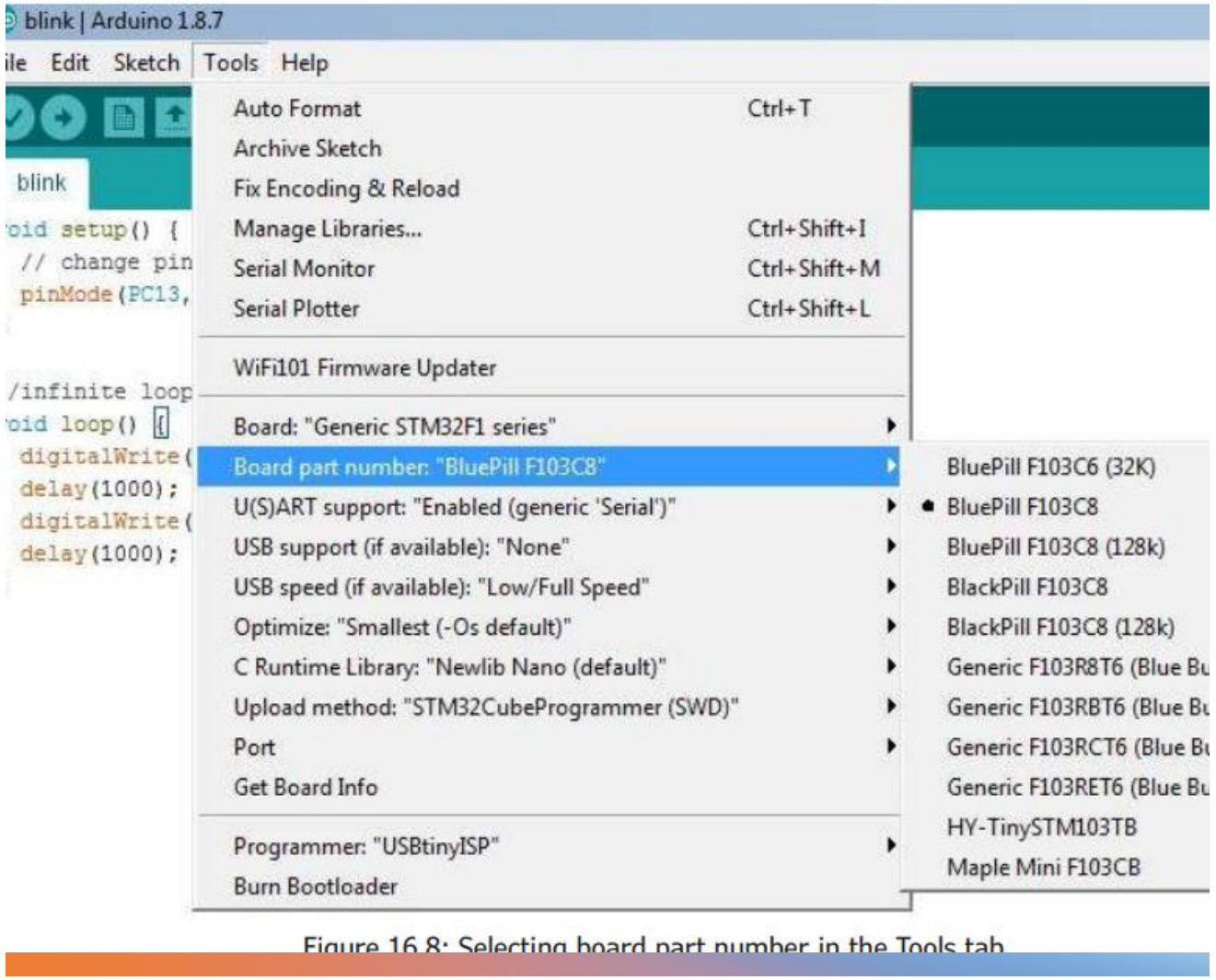


Figure 16.8: Selecting board part number in the Tools tab

- For flashing an LED connected to the PC13 pin of the STM32 blue pill board, the program is demonstrated in Figure 16.9.

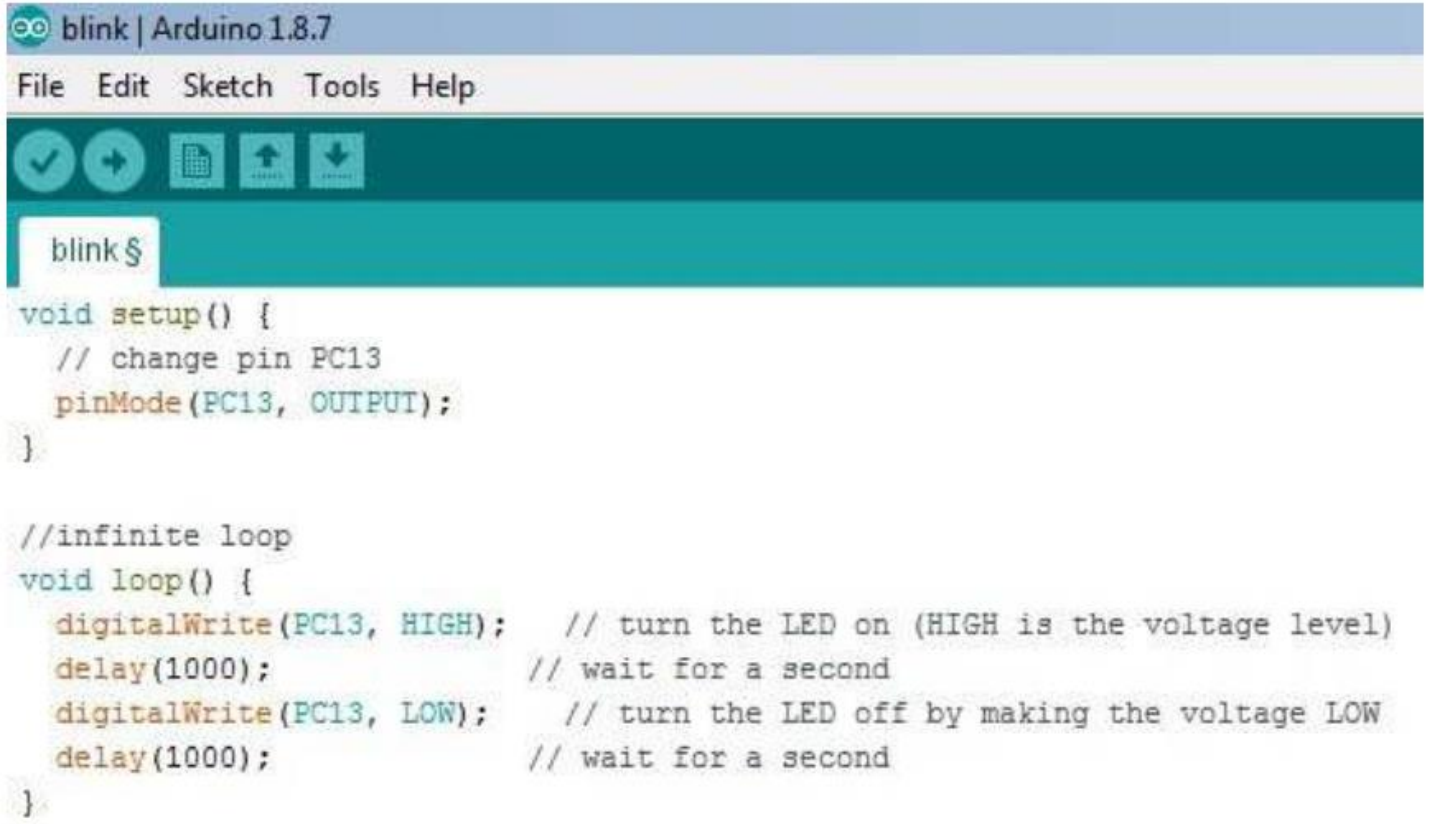


Figure 16.9: Flashing an LED program

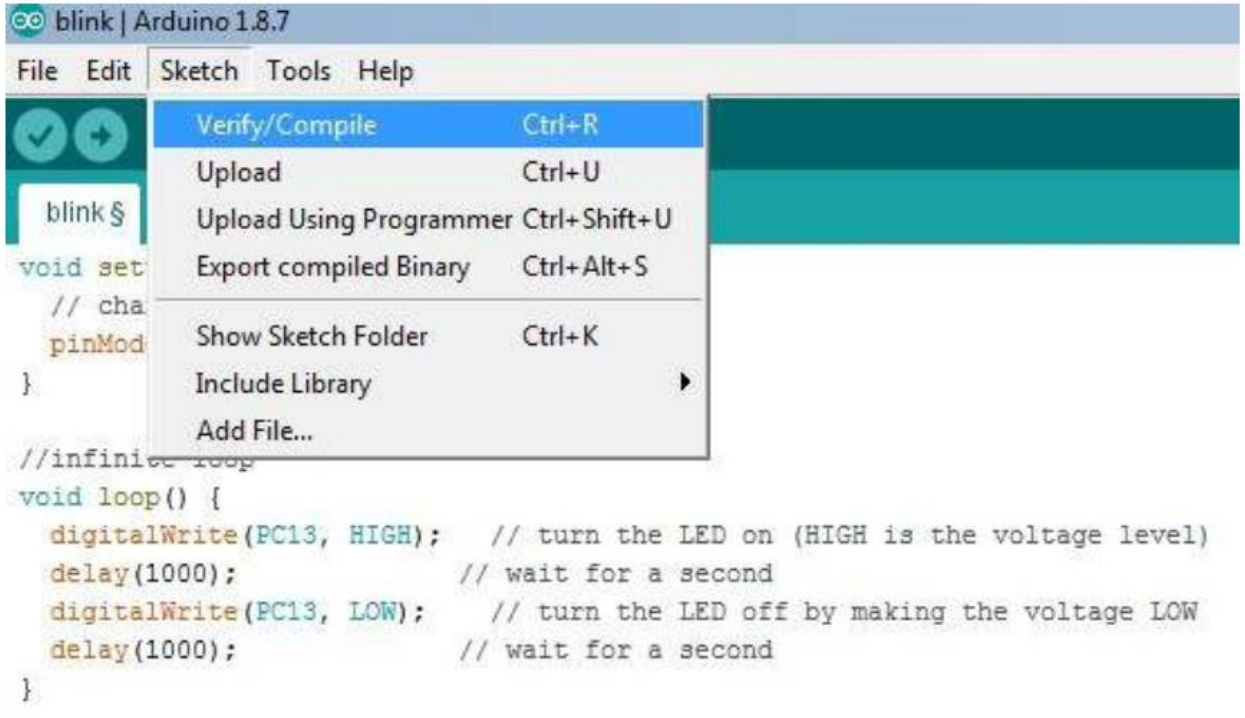


Figure 16.10: Compiling the program

- From the Sketch tab, select Verify/Compile to compile the project and generate the hex code for programming the microcontroller as shown in Figure 16.10.



Figure 16.11: Generated hex code accessing route message

- The hex code generated and its accessing route message are shown in Figure 16.11.

```
text
align 2
global main

ain:

equ     loc1,     0x20004000
equ     loc2,          0x30002000

emPrep:  LDR     R0, =loc1
        LDR     R1, =0xF0F1F2F3
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xF4F5F6F7
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xF8F9FAFB
        STR     R1, [R0]
        ADD     R0, R0, #4
        LDR     R1, =0xFCFDFEFF
        STR     R1, [R0]

        LDR     R3, =0x00//index
        LDR     R4, =0x00//jndex
        LDR     R0, =loc1//0x20004000
        MOV     R1, #0x1//mask to check odd
        MOV     R2, #0xff//mask to pick num
                MOV     R6, #0x0//summation
```

```
loop1:    LDR     R5, [R0]

loop2:   AND     R1, R1, R5
         CMP     R1, #1
         BNE     continue

         AND     R2, R2, R5
         ADD     R6, R6, R2

continue: LSR     R5, R5, #8
         MOV     R1, #0x1
         MOV     R2, #0xff
         ADD     R4, R4, #1
         CMP     R4, #4
         BLT     loop2

         LDR     R4, =0x00// reset jndex
         ADD     R0, R0, #4
         ADD     R3, R3, #1
         CMP     R3, #4
         BLT     loop1

         LSR     R6, R6, #3
         LDR     R0, =loc2
         STR     R6, [R0]

END: b END
 .end
```

Lab3 solution

ee2004

Week7, 8

Bit Values as Voltage Levels

- If we start interacting the microcontroller with the outside world, then abstract representations such as logic level 0 or 1 do not mean much. Therefore, we should associate these logic levels with actual voltage values used in the microcontroller.
- There are several voltage standards to represent logic levels 0 and 1 in digital systems. The STM32F4 microcontroller supports 3.3 V standard low voltage complementary metal-oxide semiconductor (LVCMOS) and transfer logic (LVTTL) levels.
- In this figure, *VDD* represents the supply voltage of the microcontroller.

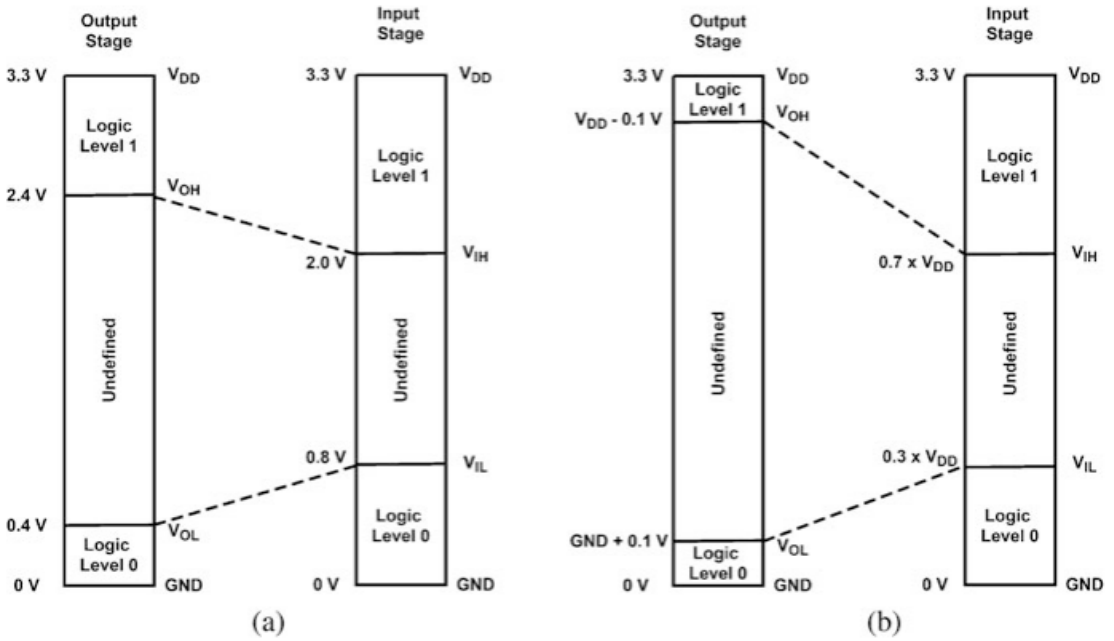


Fig. 4.1 3.3 V logic levels. (**a**) LVTTL. (**b**) LVCMOS

- When logic level 1 is fed to output from a pin, the voltage value there will be between *VDD* and *VOH* for both LVTTL and LVCMOS.
- Likewise, when logic level 0 is fed to output, the voltage value at the pin will be between 0 V and *VOL*.
- The voltage value to logic level correspondence also applies to input pin of the microcontroller.
- When we apply a voltage value between *VDD* and *VIH* to a pin, it will be recognized as logic level 1 in input.
- Likewise, applying a voltage value between *VIL* and 0 V to an input pin will be recognized as logic level 0.
- Hence, there are tolerance values for both input and output voltage values.
- This should be taken into account during digital I/O operations.
- Voltage values outside the tolerances are taken as uncertain within the microcontroller.

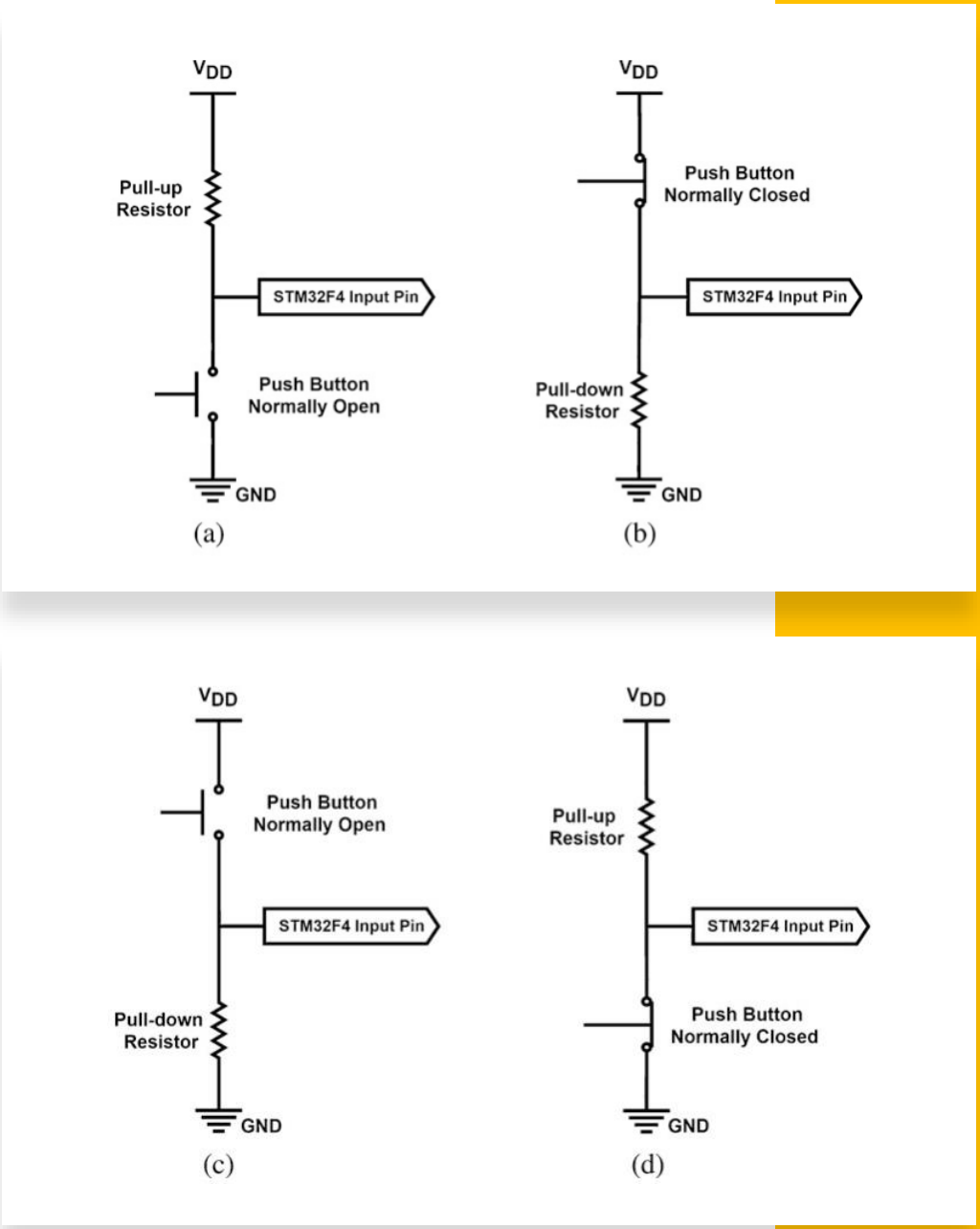
Digital Input from a Switch or Button

- While using a GPIO pin of the microcontroller, some applications may require special consideration. These can originate from input with contact closures (such as switch or button) or input with voltage values outside the microcontroller's tolerance levels. Likewise, when the microcontroller drives output loads with high current requirements, the user should be cautious.
- We can get digital input from a switch or button. In this setup, when the button is pressed (or switch is closed), a voltage value is applied to the microcontroller pin.
- The user should pay attention to two issues here. The first one is the setup (and related hardware) required for interfacing the switch or button to the microcontroller. The second one is avoiding switch bouncing.

Setting Up the Switch or Button

- There are two input setup options when a switch or button is connected to the pin of a microcontroller. In the first setup, the microcontroller gets *VDD* V (logic level 1) on its pin when the button (or switch) is interacted. Likewise, it gets 0 V (logic level 0) when the button or switch is left to its natural state. This is called active high input.
- In the second setup, the microcontroller gets 0 V (logic level 0) on its pin when the button (or switch) is interacted. It gets *VDD* V (logic level 1) when the button or switch is left to its natural state. This is called active low input.
- Active high or low inputs are not related to the microcontroller. Depending on the application, the input can be adjusted as active high or low using a pull-up or pull-down resistor. If input pin is left floating, then the input state cannot be determined.

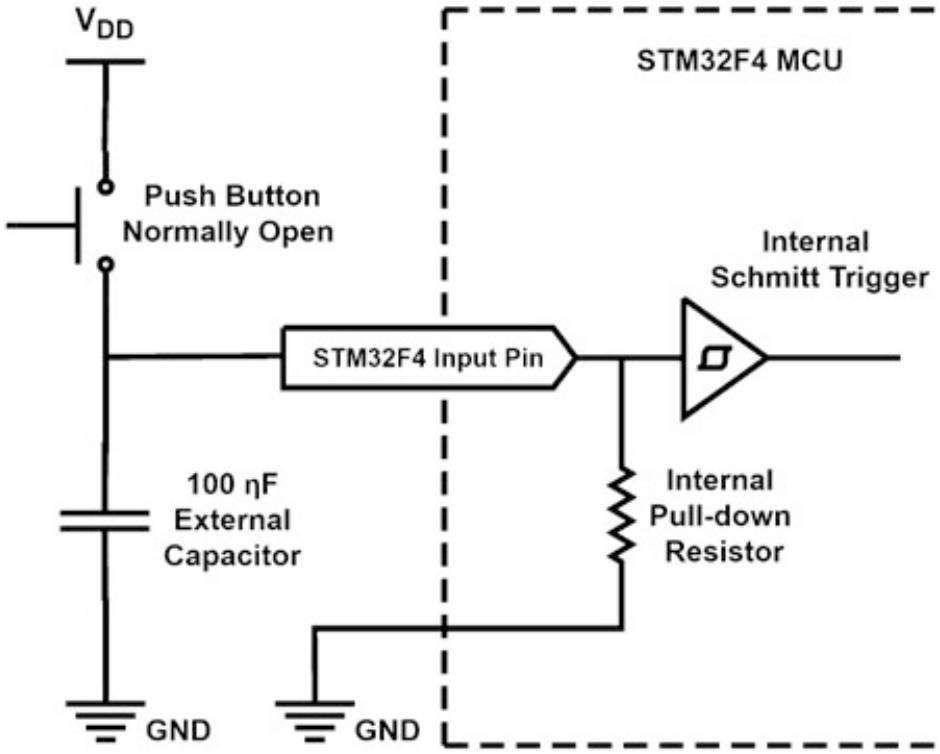
- The pull-up and pulldown resistors are used to pull the input to *VDD* or 0 V when the normally closed or open switch or button is interacted.
- These resistors can be externally connected or internal pull-up/pull-down resistors of the microcontroller can be used. Sample circuit diagrams for the active high/low input setup using normally opened and closed push buttons are as in Fig. 4.2.
- Active high and low input circuit diagrams for the push button. (**a**) Active low input, normally open push button. (**b**) Active low input, normally closed push button. (**c**) Active high input, normally open push button. (**d**) Active high input, normally closed push button



Avoiding Switch Bouncing

- Switch bouncing is a serious problem when a button (or switch) is used in a digital system.
- Bouncing may occur due to the following reason. If the button (or switch) is interacted or activated, two metal parts connect and disconnect several times before an actual stable connection. Hence, the device may generate output more than once, typically 10 to 100 times over a period of about 1 ms.
- GPIO pins of the microcontroller can capture these input values which result in false or multiple triggering while reading the input in the code.
- Either a software- or hardware-based solution can be used to eliminate the switch bouncing problem.
- The simplest software solution is adding a delay while reading the input in the code. Hence, once an input is captured, successive inputs are ignored during the delay. Although this method is easy to implement, actual inputs will also be eliminated at the same time. Moreover, this solution may block other operations and can cause unpredictable problems in the code. Therefore, it should be used with precaution.

- We can also use a shift register-based software solution to avoid switch bouncing.
- In this setup, the microcontroller captures the input values in 1 ms or a smaller time interval. These values are shifted through a shift register. During switch bouncing, the shift register content will be formed by logic level 0 and 1 values. However, the content will be formed by the same value after the bouncing effect. When such a pattern is observed, the stable input can be captured. This method may also block other software routines. However, the blocking period is usually much shorter compared to the plain delay method.
- There are also hardware-based solutions to avoid the switch bouncing problem. To do so, the most feasible circuitry is adding a low-pass RC filter (composed of a resistor and capacitor) followed by Schmitt trigger to the pin.



- Each digital I/O pin of the STM32F4 microcontroller has a Schmitt trigger.
- The internal pull-down resistor available at the pin hardware can be used in forming the RC filter.
- Resistance values for these are typically 40 K.
- An external capacitor should be connected to the pin as well. Hence, the actual setup to eliminate switch bouncing via RC circuitry will be as in Fig. 4.3 based on the available hardware.

Digital I/O with High Voltage Values

- Digital input to the microcontroller can originate from a source, such as sensor or external device, with high voltage values corresponding to logic level 0 and 1.
- Directly connecting such a source to the microcontroller can damage or burn out the GPIO pin circuitry. It may even damage other parts of the microcontroller.
- Likewise, some loads may require high voltage values to operate. Hence, the microcontroller should feed the necessary voltage levels to them.
- There are two methods to handle such situations. The first one is using a level shifter between the high voltage source or load and microcontroller. This method can be used for both high voltage input and output operations.
- The second method is using high voltage tolerant pins of the microcontroller, if there are any. To note here, even if there are such pins on the microcontroller, they can only be used for input.

- Let’s first consider level shifters with two options as unidirectional and bidirectional setup.
- Within the unidirectional setup, we can only convert a high voltage to a low value. In the bidirectional setup, we can convert both high and low voltage values to each other.
- The basic unidirectional high to low voltage level shifter is in fact a voltage divider formed by resistors. This shifter contains two resistors, as R1 and R2, which divide the high voltage by the ratio R1/(R1 + R2).

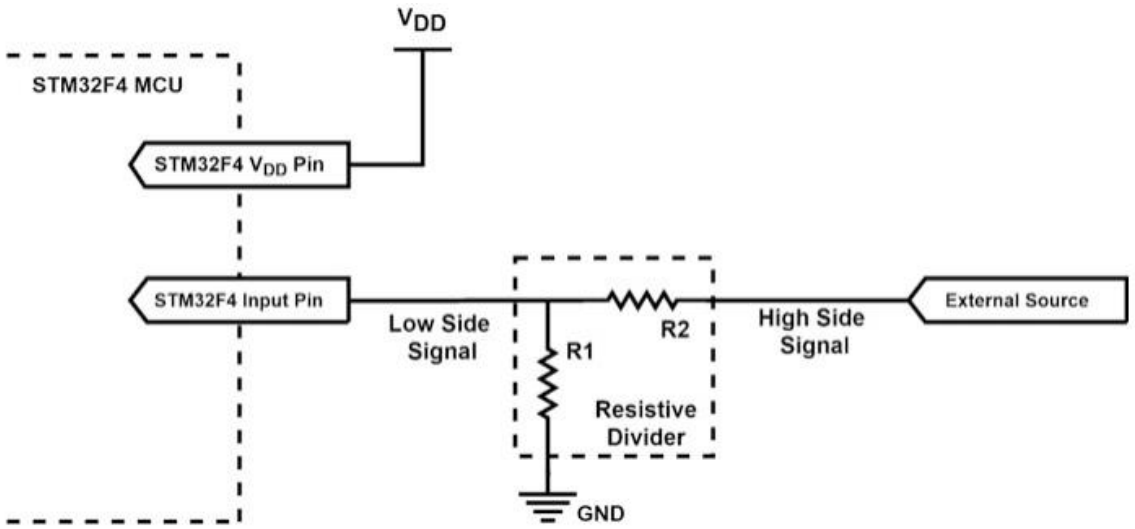


Fig. 4.4 Basic unidirectional voltage level shifter

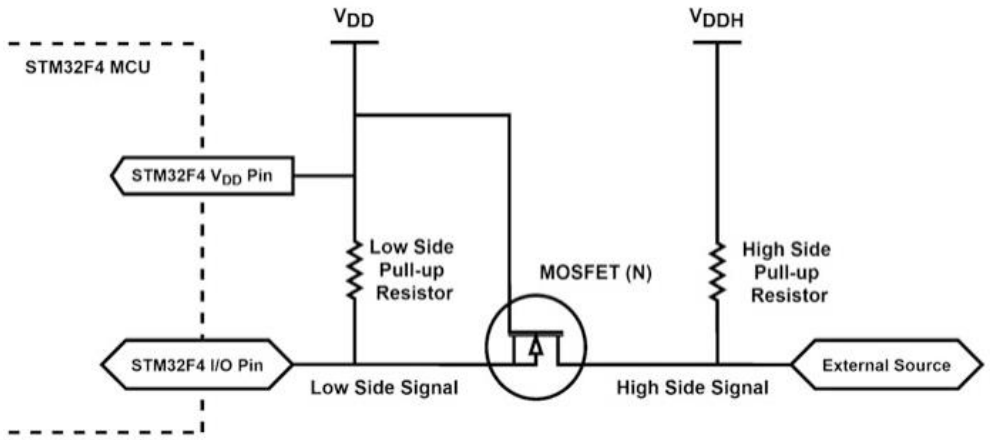


Fig. 4.5 Basic bidirectional voltage level shifter

- The basic bidirectional voltage level shifter contains a single N-channel MOSFET and a pair of pull-up resistors.
- We can divide the circuit in this figure into two parts as low and high side.
- Let's consider the scenario in which the low side is set as output and high side as input. Here, when the low side signal is set to V*DD*, the MOSFET turns off, and high side signal is pulled up to *VDDH* (high voltage level) through the pull-up resistor.
- When the low side signal is set to 0 V, the MOSFET source pin is set to ground voltage. Hence, the MOSFET turns on and the high side is pulled down to 0 V.
- Let's next consider the scenario in which the high side is set as output and low side as input. Here, the MOSFET substrate diode conducts current and pulls the low side signal down to 0.7 V when the high side signal is set to 0 V. When the high side signal is set to *VDDH*, the substrate diode does not conduct current. Hence, the low side signal is pulled up to *VDD* through the pull-up resistor.

Digital Output to a Load Requiring High Current and Voltage Values

- Most microcontrollers can feed 3.3–5 V and 25–40 mA through their GPIO pins. These values are 3.3 V and 25 mA for separate pins of the STM32F4 microcontroller.
- Besides, the total output current that can be drawn from all pins of the STM32F4 microcontroller at once cannot exceed 120 mA.
- We may want to drive heavy loads, such as a motor or power LEDs, by the microcontroller. Voltage and current output from the microcontroller may not be sufficient for them.
- There are three methods to handle these cases using transistors, optocouplers, and relays.
- The first method in handling a load requiring high voltage and current is using a transistor circuit between the load and microcontroller pin. This setup is only suitable for DC voltages. The reader can use either a BJT or MOSFET for this purpose.

- As can be seen in Fig. 4.6, BJT or MOSFET turns off when the microcontroller pin is set to logic level 0 (in other words 0 V). Hence, the current cannot flow through the load.
- When the microcontroller pin is set to logic level 1 (in other words 3.3 V), small current flows through the bias resistor and base-emitter junction of the BJT. Hence, BJT turns on and current flows from the collector (connected to load) to the emitter pin (connected to ground).
- MOSFET turns on and current flows from drain to source (connected to ground). To note here, *VDD*2 is the separate DC power supply voltage where load is connected to. The *VDD*2 voltage level can be same or higher than *VDD*.

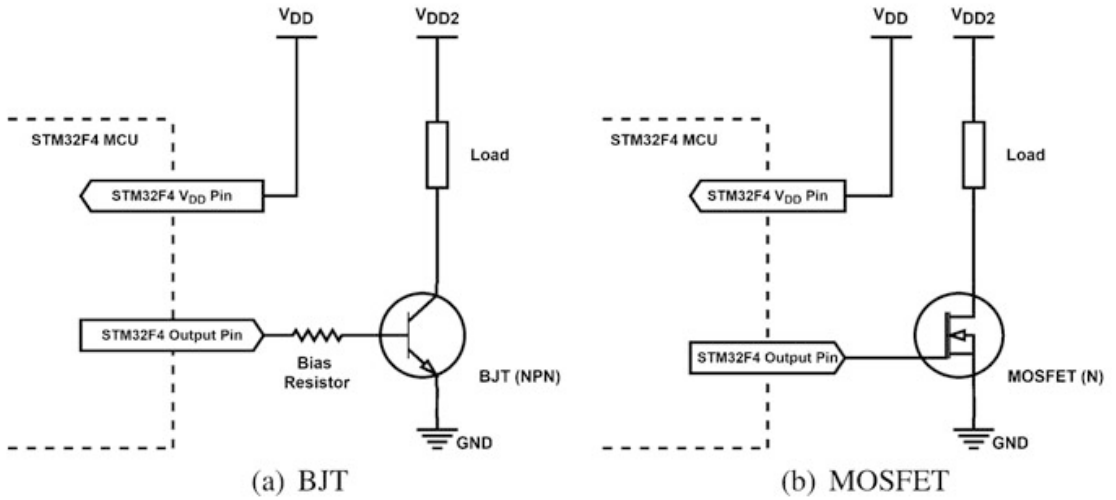
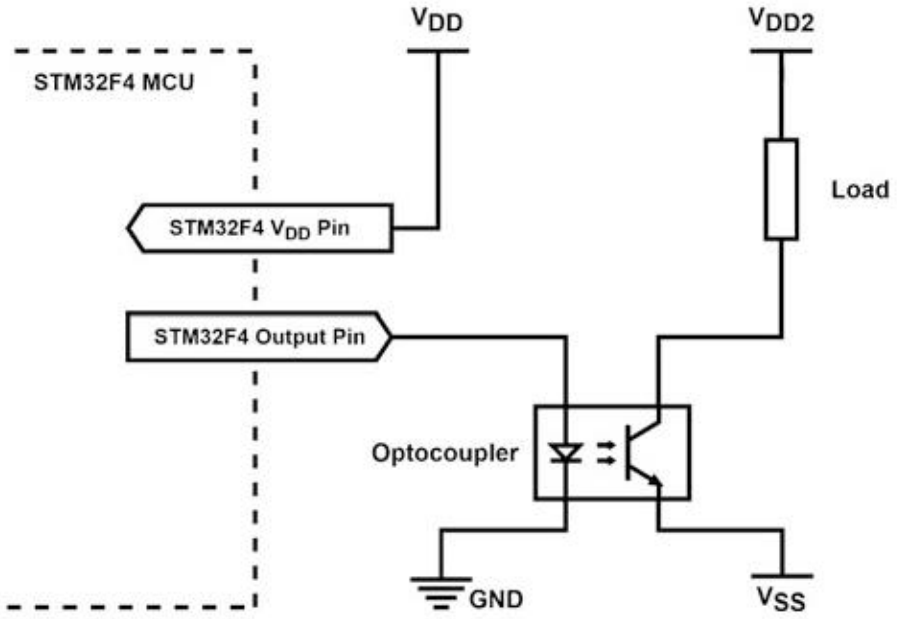


Fig. 4.6 Driving a heavy load with transistor. (**a**) BJT. (**b**) MOSFET

- The second method in handling a load requiring high voltage and current is using an optocoupler between the load and microcontroller pin. This setup is only suitable for DC voltages. Optocoupler is an electronic component made up by a simple phototransistor and LED that separates two circuits from each other using light. This setup allows handling loads with current and voltage values up to 3 A and 20 V, respectively. Moreover, the optocoupler completely separates two sides of the circuit (load and microcontroller) including their ground. This means different supply voltages and grounds can be used in the microcontroller and load connected to it.



- The third method in handling a load requiring high voltage and current is using a relay between the load and microcontroller pin.
- Unlike the previous two options, a relay can control both AC and DC voltages.
- Relay is an electromechanical or solidstate element that can open and close. The electromechanical relay is composed of a contact switch that opens or closes when current flows in its internal coil or coils.
- It can handle very high loads having current and voltage values such as 10– 15 A or 110–220 V, respectively.
- The solid-state relay (SSR) usually consists of an optocoupler at its input stage and triac or MOSFET at its output stage for AC or DC output, respectively.
- SSRs can handle very high loads such as the ones requiring 500 A current.
- Some small power, electromechanical or solid state, relays can be directly connected to the GPIO pin of the microcontroller. However, a transistor or another supplementary circuit is required to drive the relay itself most of the times.

- As can be seen in Fig. 4.8, when the microcontroller pin is set to logic level 0 (in other words 0 V), BJT turns off and current cannot flow through internal coil of the relay. Hence, internal contact switch does not close and load current does not flow.
- When the microcontroller pin is set to logic level 1 (in other words 3.3 V), BJT turns on and current flows through internal coil of the relay. This closes the internal contact switch of relay and high current can flow through the load.
- The diode connected against the coil is of type fly back. It protects the circuit from any back electromotive force (EMF) by the coil of the relay.
- To note here, *VDD*2 is the separate DC power supply voltage where the internal coil of relay is connected to. The *VDD*2 voltage level can be same or higher than *VDD*.
- *VDD*3 is the separate AC or DC power supply voltage where the load is connected to. The *VDD*3 voltage level can be any value that the relay can support.

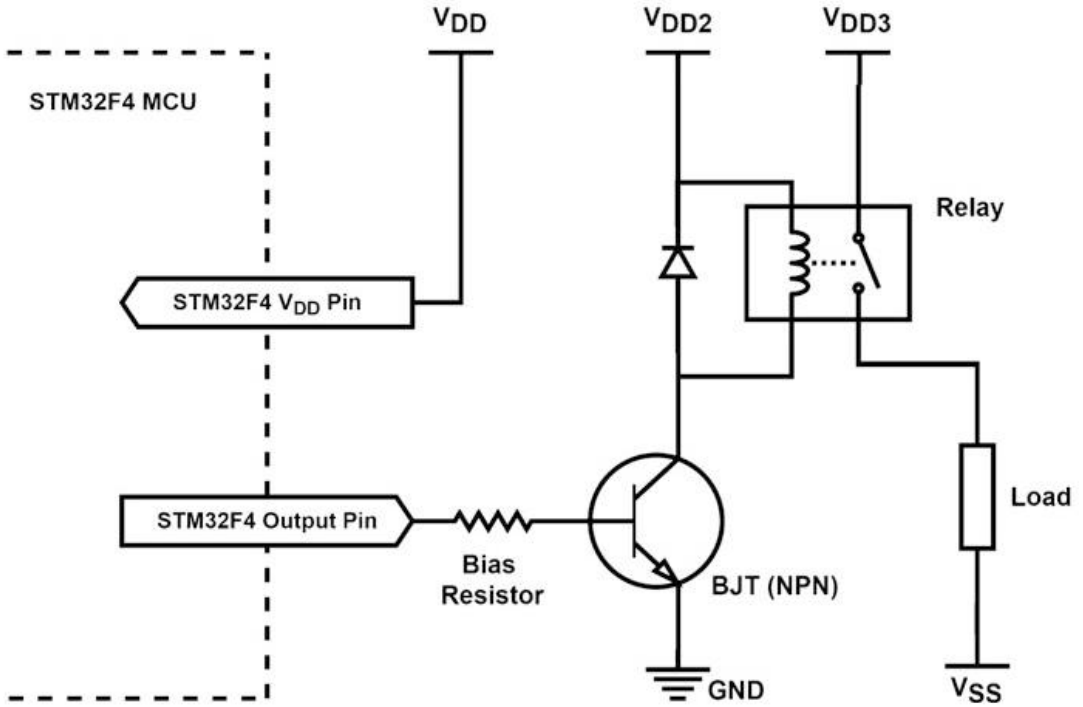


Fig. 4.8 Driving a heavy load with relay

Setting Up GPIO Registers

- STM32CubeIDE allows a simpler method to set up port registers when the C language is used in programming. To be more specific, HAL library functions are used for this purpose. HAL is the hardware abstraction layer which allows programmers to form device-independent code while accessing the hardware. Hence, the same code may be used in other devices with minor modifications.
- A GPIO pin within a port can be configured in several modes as input, output, analog, external interrupt, or alternate function. At the moment, we will only explain configuring the GPIO pin as input or output using STM32CubeMX.
- When the GPIO pin is configured as input, it can be left floating or its weak internal pull-up or pulldown resistor can be enabled by software.
- When the GPIO pin is configured as output, it can be configured as open-drain or push-pull mode. Its drive speed can also be selected depending on the application.

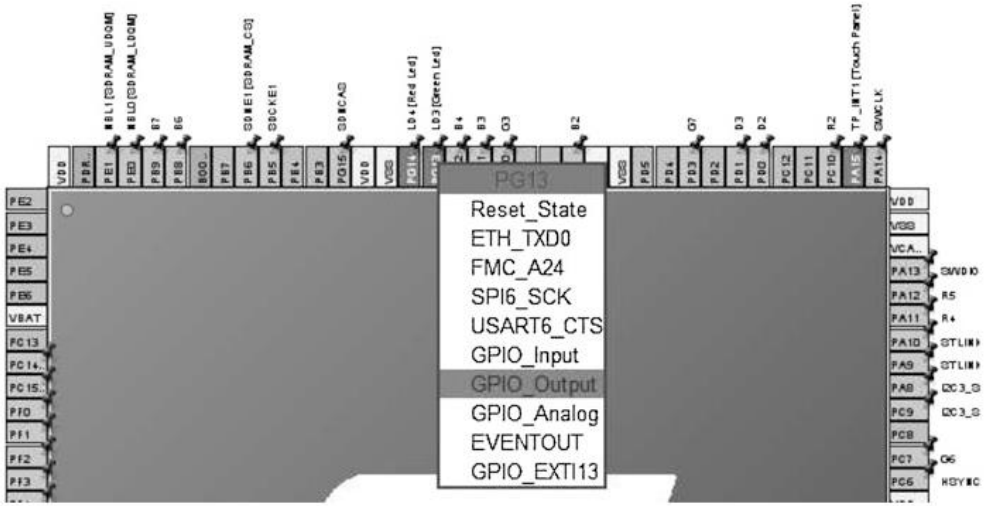


Fig. 4.10 GPIO configuration via STM32CubeMX

- A sample procedure while configuring and using a GPIO pin is as follows. We will assume that the project has already been created in STM32CubeIDE as explained in Sect. 3.1.5.
- To modify the pin usage, first open the “Device Configuration Tool” window by clicking “.ioc” file in the project explorer window. Then, click the GPIO pin on the STM32F4 microcontroller and select the GPIO type as in Fig. 4.10 on the “Device Configuration Tool” window.
- If the GPIO is selected as input, then its pull-up/pull-down resistor can be activated using “GPIO Pull-up/Pull-down” option under the “System Core—GPIO” section.
- If the mode is selected as output, then speed can be configured through “Maximum output speed” and mode can be configured through “GPIO mode” under the “System Core— GPIO” section.
- The main file will be generated when the “.ioc” file is saved with changes. Hence, STM32CubeIDE will form necessary functions to set up GPIO registers via HAL library functions

- We can use GPIO pins through HAL library functions in C language. After configuring the pin, the level of the pin can be read using the function HAL_GPIO_ReadPin() in the code if it is configured as input.
- If the GPIO pin is configured as output, then it can be set to logic level 1 or 0 by the function HAL_GPIO_WritePin() in the code. Alternatively, the function HAL_GPIO_TogglePin() can be used to toggle the output level.

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx,uint16_t
    GPIO_Pin)
/*
GPIOx: The port name where x can be (A...H) to select the
    specific port for the device.
GPIO_Pin: The pin to be read. This parameter can be one of
    GPIO_PIN_0 ...  GPIO_PIN_15
*/

void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin,
    GPIO_PinState PinState)
/*
GPIO_Pin: The pin to be written to. This parameter can be one of
    GPIO_PIN_0 ... GPIO_PIN_15
PinState: Specifies the value to be written to the selected bit.
    This parameter can be one of the GPIO_PinState value
GPIO_BIT_RESET: To clear the port pin

GPIO_BIT_SET: To set the port pin
*/

void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin)
/*
GPIO_Pin: The pin to be toggled. This parameter can be one of
    GPIO_PIN_0 ... GPIO_PIN_15
*/
```

- Next, we will provide example codes to show how these functions can be used to control GPIO pins.
- Our first code, given in Listing 4.1, uses the HAL library functions to toggle the green LED (connected to pin PG13) on the STM32F4 board in an infinite loop using delays. In order to observe how this code runs, create an empty project under STM32CubeIDE and configure pin PG13 as output.
- Then, use the code given in Listing 4.1 to toggle the green LED on the STM32F4 board. The code also consists of an alternative way of performing the same operation. This section is labeled as "Alternatively" within the code.

Listing 4.1 Toggle the onboard green LED by delay func

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, G
HAL_Delay(1000);

HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, G
HAL_Delay(1000);

// Alternatively
/*
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_13);
HAL_Delay(1000);
*/

/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The second code, given in Listing 4.2, toggles the red and green LEDs on the STM32F4 board via user button presses. Here, pins PG13 and PG14 (connected to the green and red LEDs) are set as output. The pin PA0 (connected to the user button) is set as input. In order to observe how this code runs, create an empty project and configure the input and output pins via STM32CubeMX interface. Then, use the code given in Listing 4.2 to toggle the green and red LEDs via the user button. Within the code, there is an additional delay function to prevent switch bouncing due to button presses.

Listing 4.2 Toggle the onboard red and green LEDs by the user button presses, the

```
/* USER CODE BEGIN PV */
uint32_t counter = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
{
HAL_Delay(100); // Avoid Switch Bouncing
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else
{
HAL_Delay(100); // Avoid Switch Bouncing
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The third code, given in Listing 4.3, has two counters. The first counter can be used to turn on and off the green and red LEDs on the STM32F4 board via increasing its value at each button press. When mod four of the first counter has value zero, both LEDs turn off. When mod four of the first counter value has value one, only the red LED turns on. When mod four of the first counter value has value two, only the green LED turns on. When mod four of the first counter value has value three, both LEDs turn on. The second counter increases in a continuous manner in an infinite loop.

Listing 4.3 Turn on the green and red LEDs by specific button press count values, the C code

```
/* USER CODE BEGIN PV */
uint32_t counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
{
while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET);
counter1++;

if (counter1 % 4 == 0)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else if (counter1 % 4 == 2)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 3)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
}
counter2++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- In Listing 4.3, there is no function added to the code to avoid switch bouncing. Instead, a hardware circuit is formed by the internal pull-down resistor and 100 ηF external capacitor (available on the STM32F4 board). Still some glitches may be observed when the button is pressed. There is a while loop placed in the code to prevent the multiple toggling operations at each button press. However, these loops block other routines, such as counter increment. The reader can observe this by placing a breakpoint to the counter increment line and pressing the button.
- The fourth code, given in Listing 4.4, is the modified form of the code given in Listing 4.3. It uses an additional shift register to eliminate switch bouncing.
- In Listing 4.4, there is a 16-bit shift register for each pin input and the read value is shifted through it. When the 0x7FFF pattern is observed for the user button, then LEDs turn on and off accordingly. The reader can observe that this code does not block the counter increment by placing a breakpoint to the counter increment line and pressing the button.

Listing 4.4 Turn on the green and red LEDs by specific button press count values with switch bouncing handled, the C code

```
/* USER CODE BEGIN PD */
#define DEBOUNCE_PATTERN 0x7FFF
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t shiftRegButton;
uint32_t counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
shiftRegButton = (shiftRegButton << 1) + HAL_GPIO_ReadPin(GPIOA,
    GPIO_PIN_0);
HAL_Delay(1);

if (shiftRegButton == DEBOUNCE_PATTERN)
{
counter1++;

if (counter1 % 4 == 0)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 1)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
else if (counter1 % 4 == 2)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
}
else if (counter1 % 4 == 3)
{
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_SET);
}
}

counter2++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

Listing 4.5 Toggle the onboard green LED by delay functions, the C++ code

```
#include "mbed.h"

//DigitalOut greenLED(LED1);
//DigitalOut greenLED(PG_13);

#define GRN_LED_MASK 0x2000
//#define GRN_LED_MASK 0b10000000000000
PortOut greenLED(PortG, GRN_LED_MASK);

int main()
{
while (true)
{
greenLED = 0;
//greenLED.write(0);
//greenLED = 0;

thread_sleep_for(1000);

//greenLED = 1;
//greenLED.write(1);
greenLED = GRN_LED_MASK;

thread_sleep_for(1000);

//Alternatively
/*
greenLED = !greenLED;
thread_sleep_for(1000);
*/
}
}
```

Listing 4.6 Toggle the onboard red and green LEDs by the user button presses, the C++ code

```
#include "mbed.h"

DigitalOut greenLED(LED1);

DigitalOut redLED(LED2);
//DigitalIn button(BUTTON1);

#define BUTTON_MASK 0x0001
//#define BUTTON_MASK 0b00000001
PortIn button(PortA, BUTTON_MASK);

int main()
{
greenLED = 1;
redLED = 0;
while (true)
{
if (button == 1)
{
thread_sleep_for(100);
greenLED = 0;
redLED = 1;
}
else
{
thread_sleep_for(100);
greenLED = 1;
redLED = 0;
}
}
}
```

Listing 4.7 Turn on the green and red LEDs by specific button press count values, the C++ code

```
#include "mbed.h"

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
DigitalIn button(BUTTON1);

int counter1 = 0, counter2 = 0;

int main()
{
greenLED = 0;
redLED = 0;

while (true)
{
if (button)
{
while (button);
counter1++;

if (counter1 % 4 == 0)
{
greenLED = 0;
redLED = 0;
}
else if (counter1 % 4 == 1)

{
greenLED = 0;
redLED = 1;
}
else if (counter1 % 4 == 2)
{
greenLED = 1;
redLED = 0;
}
else if (counter1 % 4 == 3)
{
greenLED = 1;
redLED = 1;
}
}

counter2++;
}
}
```

Listing 4.8 Turn on the green and red LEDs by specific button press count values with switch bouncing handled, the C++ code

```
#include "mbed.h"
#define DEBOUNCE_PATTERN 0x7FFF

DigitalOut greenLED(LED1);
DigitalOut redLED(LED2);
DigitalIn button(BUTTON1);

int counter1 = 0, counter2 = 0;
uint32_t shiftRegButton;

int main()
{
greenLED = 0;
redLED = 0;

while (true)
{
shiftRegButton = (shiftRegButton << 1) + button;
thread_sleep_for(1);

if (shiftRegButton == DEBOUNCE_PATTERN)
{
counter1++;

if (counter1 % 4 == 0)
{
greenLED = 0;
redLED = 0;
}
else if (counter1 % 4 == 1)
{
greenLED = 0;
redLED = 1;

}
else if (counter1 % 4 == 2)
{
greenLED = 1;
redLED = 0;
}
else if (counter1 % 4 == 3)
{
greenLED = 1;
redLED = 1;
}
}

counter2++;
}
}
```

The Interrupt Concept in Embedded Systems

- The interrupt usage is almost inevitable in embedded systems. On the other hand, interrupts depend on a signal generated either by hardware or another dedicated software module. Hence, handling and using the interrupt is slightly different from standard programming practices.
- An embedded system should be interacting with the outside world by its nature. Moreover, it should respond to a request coming from the user. One option to handle such requests is forming an infinite loop in the code and checking requests in a continuous manner. Although this method is useful in its own rights, it blocks the CPU.
- To overcome this problem and freeing up the CPU from waiting in an infinite loop, a different programming approach is needed. The interrupt concept comes into play for this purpose. Within this approach, asynchronous or periodic requests can be handled without letting the CPU to wait for them indefinitely. One can think of interrupt as a response mechanism to an input signal such that when it occurs, a predefined function is executed by the CPU. The difference of the interrupt is that when the required input is not fed to the microcontroller, the CPU performs its usual synchronous operation. Hence, it does not wait for an event to occur in the infinite loop.

- To handle GPIO interrupts, we should set up the microcontroller such that related GPIO pins should be able to generate interrupt requests. In other words, the related pins should be allowed to generate interrupts. Hence, the programmer should adjust properties of each pin based on its registers.
- The user should also set priority to each interrupt request. In other words, the user should rank the interrupt requests coming from different buttons.
- As these settings are done, the pins will be ready to form interrupt requests. Besides, the CPU should be set to accept and handle these requests. This is done by setting a special register within the CPU.
- As the microcontroller is set up to handle the interrupt requests, let us observe what happens when they occur. When a button connected to the pin is pressed, the voltage difference there is sensed by an edge detector circuit connected to the pin of the microcontroller. As a result, a flag is set indicating that the pin generated an interrupt request.

- A special module handles this interrupt request. To do so, it first checks whether the request is allowed or not. Then, the module checks priority of the incoming interrupt such that it should precede the existing operation.
- What we mean by responding to the interrupt request is as follows. The CPU is diverted to the function to be executed as a response to the interrupt request. This function is called interrupt service routine (ISR) or callback function.
- As a note, these specific callback functions are defined within the code. However, they are not called by their name in the code. Instead, the hardware signal generated by the associated button press calls each function.
- As the CPU starts executing the callback function, the related flag should be reset. Hence, the CPU knows that the interrupt is handled and there is no need to pay attention to it again. As the operation within the callback function is done, the CPU turns back to its previous operation.

- The GPIO interrupt example we have provided falls into the maskable interrupts case. This means that the user can disable an interrupt request within this group. Hence, it may not be served at all.
- There is another group called non-maskable interrupts. As the name implies, an interrupt request within this group must be handled. These interrupts represent key operations such that when not served, they cause the microcontroller to malfunction.
- Reset is one such interrupt. As a matter of fact, when we debug our code and embed it on the microcontroller, a reset signal is generated such that the CPU goes to the beginning of the code to be executed. We may also generate the reset signal by pressing the reset button on the board. When this signal is generated, the CPU stops what it is doing and turns back to the beginning of the code.

Interrupts in the STM32F4 Microcontroller

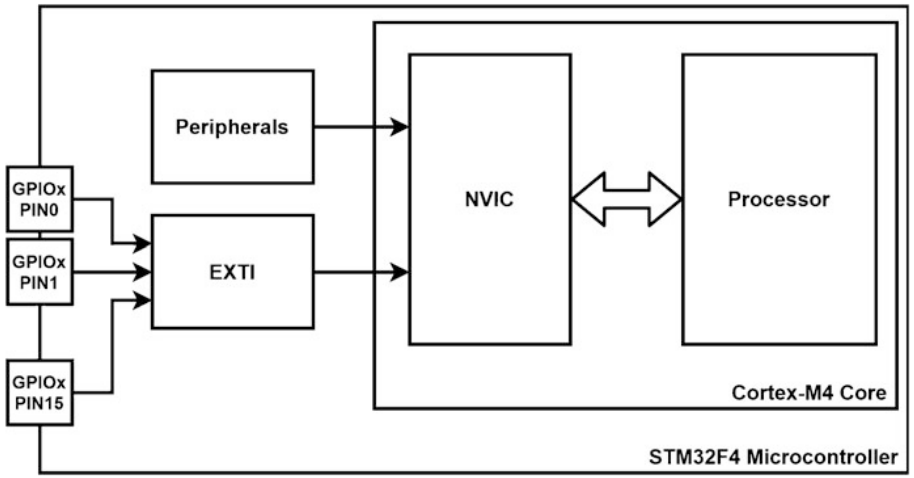


Fig. 5.1 Block diagram of the STM32F4 interrupt-based modules

- There are four main blocks related to interrupt handling in the STM32F4 microcontroller.
- The first block contains peripheral units (such as timers, ADC, and UART modules) which can generate interrupt requests.
- Within the second block, GPIO pin interrupts are handled by the extended interrupts and events controller (EXTI).
- In the third block, all interrupt requests (whether generated by a peripheral unit or EXTI) are handled by the nested vector interrupt controller (NVIC).
- In the fourth block, accepted interrupt requests are processed by the CPU. Next, we will focus on each module separately to emphasize its role in interrupt handling and execution stages.

Interrupt Operations in Peripheral Units

- As explained in the previous section, peripheral units can generate interrupt requests. This is done in several ways. For example, a GPIO pin can generate an interrupt request when it observes a low to high or high to low voltage transition.
- The timer can generate periodic interrupt requests. The ADC module can generate an interrupt request when the analog to digital conversion operation is finished.
- The UART module can generate an interrupt request after receiving or submitting a packet.
- The EXTI module handles GPIO interrupt operations in the STM32F4 microcontroller. Therefore, we will introduce this module next.
- There is a general structure for interrupt operation via peripheral units. Each peripheral unit has a specific register to enable or disable its interrupts. Besides, the unit has a flag indicating the interrupt request. In other words, if an interrupt request is generated by the peripheral unit, a flag is raised to indicate that. Then, the request is handled by the NVIC module.

Extended Interrupts and Events Controller

- The STM32F4 microcontroller has a dedicated module, called extended interrupts and events controller (EXTI), to handle external interrupts from some peripheral units.
- As can be seen in Fig. 5.1, the EXTI module stands between the GPIO pins and NVIC module. We provide a more detailed block diagram of the EXTI module in Fig. 5.2.

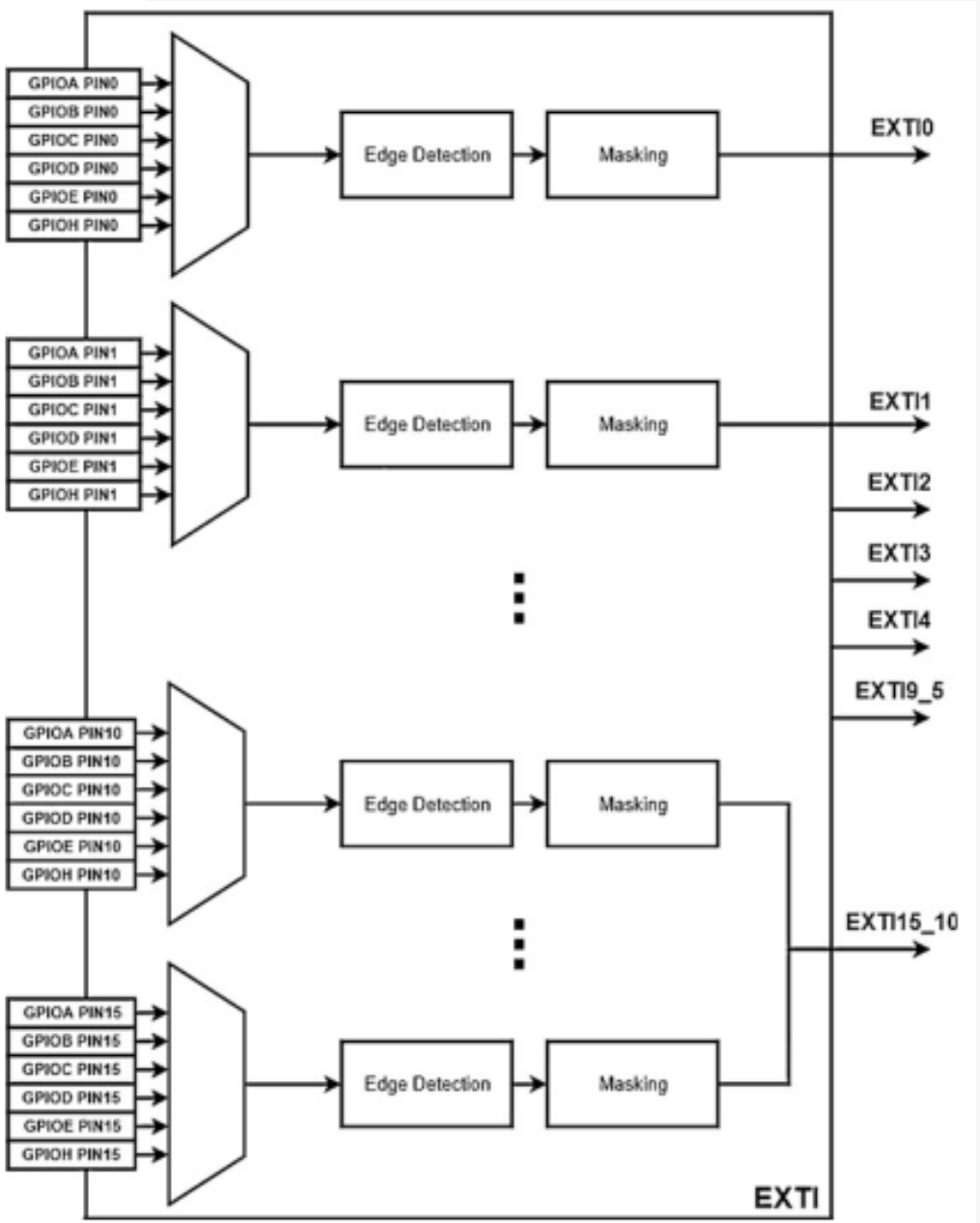


Fig. 5.2 Detailed block diagram of the EXTI module

- As can be seen in Fig. 5.2, GPIO pins are grouped within themselves and fed to separate multiplexers. Each multiplexer has an output pin called EXTI0, EXTI1, and so on.
- To be more specific, each zeroth pin of all ports is connected to the same multiplexer. Output of it is called EXTI0. Likewise, each first pin of all ports is connected to another multiplexer having output EXTI1.
- Moreover, each EXTI output is not directly connected to the NVIC module. EXTI outputs 1 to 4 are directly connected to NVIC.
- EXTI outputs 5 to 9 are grouped together and fed as a single input to NVIC. Likewise, EXTI outputs 10 to 15 are grouped together and fed as another single input to NVIC.
- Hence, the EXTI module groups GPIO pins such that they can be connected to the NVIC module. The drawback of this setup is that we cannot distinguish two interrupts originating from the same group. As an example, it is not possible to distinguish interrupts from GPIO pins PA6 and PA7.

The EXTI module can be used to enable and configure GPIO interrupts. This is done by its dedicated registers. Related to this, the module also allows masking an interrupt for its specific output.

As for configuration, the EXTI module allows generating a GPIO interrupt request by observing a low to high or high to low transition in the pin. Hence, the user can generate an interrupt by either an active high or low input from the button connected to that pin.

As the EXTI module is used to enable the GPIO interrupt for a specific pin and the required transition (either low to high or high to low) is observed there, an interrupt request is generated. Hence, a bit corresponding to the EXTI line is set. Afterward, NVIC handles the operations to be explained next.

Nested Vectored Interrupt Controller

- The nested vectored interrupt controller (NVIC) manages interrupts from peripheral units and internal exceptions originating from the CPU (such as fault handling).
- If a single interrupt request comes from a peripheral unit while the CPU is in thread mode (performing its usual sequential operation), then NVIC checks whether that request is allowed or not. It also checks whether the CPU accepts interrupt requests in general.
- If this is the case, then NVIC directs the CPU to execute the corresponding callback function for the incoming interrupt request.
- Hence, the CPU goes to the handler mode. As the callback function is executed, the CPU turns back to thread mode waiting for the next interrupt request. This is the simplest scenario for an interrupt operation.

- If more than one interrupt request comes at the same time, or an interrupt request comes while another one is being serviced, then NVIC organizes their execution based on their priority level. To do so, interrupt requests coming from different sources should be ordered.
- This is done within NVIC. Although there is a standard order for interrupt priority levels, the user can adjust them by his or her needs while forming the project. The important point here is that a priority level cannot exceed three important (and non-maskable) interrupts.
- As the priority level is set and a new interrupt request comes while another one is being serviced, the following scenarios can occur.
- First, if the CPU is in thread mode, then the priority of the incoming interrupt precedes it.
- Second, the CPU may be in handler mode responding to another interrupt with higher priority. In this case, response to the incoming low priority interrupt should wait till the end of this operation. Then, the low priority interrupt is handled.
- Third, the CPU may be in handler mode responding to an interrupt request with lower priority. Since the new incoming interrupt request has higher priority, the low priority one is suspended and the incoming new interrupt is handled. Afterward, the CPU turns back to handle the low priority interrupt request.

Interrupt Operations in the CPU

- As the NVIC module handles the interrupt request and directs the program flow, then the request is processed by the CPU.
- The first operation here is checking whether the interrupt requests are generally allowed by the CPU.
- If this condition is satisfied and CPU allows processing interrupt requests, then the corresponding callback function is executed by the CPU. Hence, CPU goes to handler mode.
- To note here, it is the user's responsibility to form the callback function. Besides, the corresponding interrupt flag should be reset within this function. Hence, the CPU will not serve it again.
- This reset operation is done automatically for most peripheral units. As the interrupt is serviced, then the CPU turns back to the thread mode again.

Interrupt Setup in the STM32F4 Microcontroller

- STM32CubeIDE adjusts all interrupt related hardware properties, such as interrupt priority and NVIC module setup, automatically as the project is compiled. Let's look at an example while configuring a GPIO pin interrupt.
- First, open the "Device Configuration Tool" window by clicking the ".ioc" file in the project explorer window. Hence, STM32CubeMX starts. Then, click on the GPIO pin on the STM32F4 microcontroller and select the GPIO type as "GPIO_EXTI" for pin interrupt source as in Fig. 5.3.

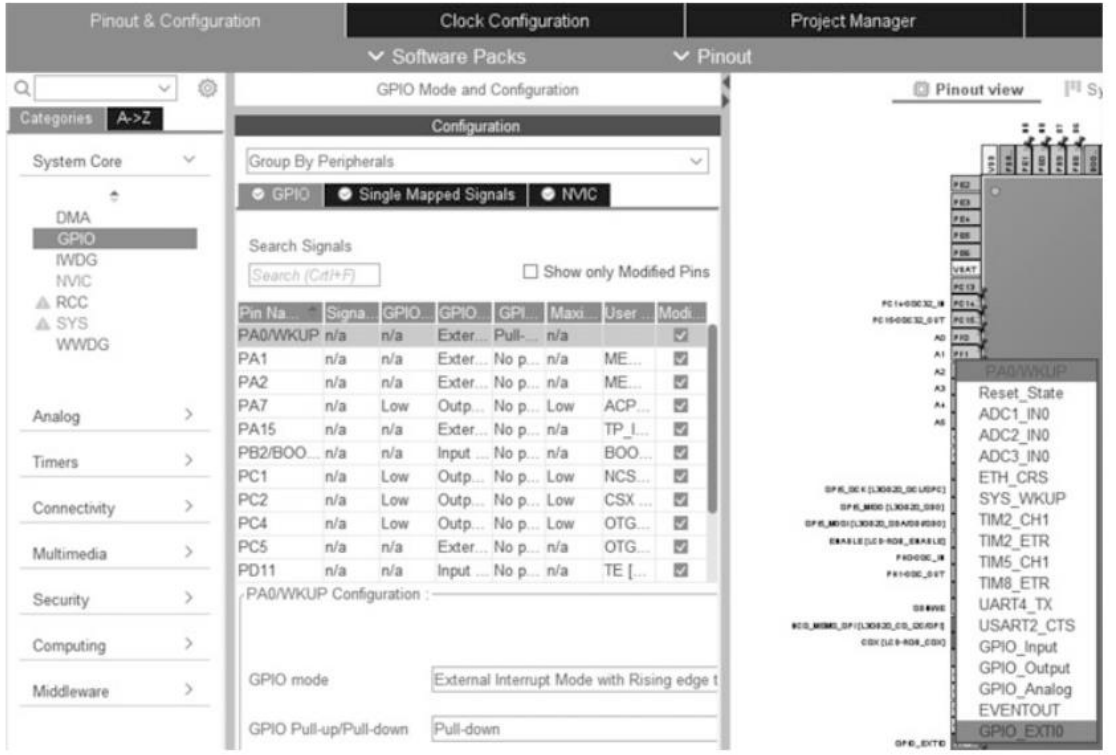


Fig. 5.3 Interrupt configuration in STM32CubeMX

- Under the “System Core—GPIO” section, GPIO mode can be selected among the following options as “external interrupt mode with rising edge trigger detection,” “external interrupt mode with falling edge trigger detection,” “external interrupt mode with rising/falling edge trigger detection,” “external event mode with rising edge trigger detection,” “external event mode with falling edge trigger detection,” and “external event mode with rising/falling edge trigger detection.”
- The difference between the event and interrupt at this stage is as follows. If an event-based option is selected, then the CPU goes to the main loop in the code after waking up from a sleep mode to be explained in Sect. 5.4.
- In the interrupt selection, the CPU starts executing the callback function after waking up from the sleep mode. Select appropriate type such as “external interrupt mode with falling edge trigger detection” here. In the same section, the pull-up/pull-down resistor can be activated using the “GPIO Pulldown” option. Select pull-up/pull-down resistor here. Afterward, enable the EXTI line interrupt under NVIC tab located in the same window or under “System Core— NVIC” section. The reader can also change the interrupt priority under this section.

Interrupt Usage via C Language

HAL library has special functions to handle interrupts. These will be of use when an interrupt-based code is written in C language under STM32CubeIDE. Next, we only provide the one related to GPIO interrupts in detail.

```
void HAL_GPIO_EXTI_Callback (uint16_t GPIO_Pin)
/*
GPIO_Pin: The pin to be connected to the corresponding EXTI line.
     This parameter can be one of GPIO_PIN_0 ... GPIO_PIN_15
*/
```

When a GPIO interrupt occurs, the NVIC module directs the CPU to its corresponding ISR. The HAL library handles the ISR such that within it the `HAL_GPIO_EXTI_Callback` is called. Without complicating the concepts, we can safely say that `HAL_GPIO_EXTI_Callback` is the callback function called when a GPIO interrupt occurs. We should define this function within the "main.c" file under STM32CubeIDE. To note here, this function is generated in weak form (without any content) automatically in the file "stm32l4xx_hal_gpio.c" when a project is generated. Hence, when the reader forgets defining the actual `HAL_GPIO_EXTI_Callback` function, the code still runs. When the user defines his or her function, then this weak function is ignored.

- The first example, with the C code given in Listing 5.2, aims to familiarize the reader with the setup and usage of interrupts.
- Here, when the onboard button (connected to PA0 pin of the STM32F4 board) is pressed and released, the green and red LEDs (connected to PG13 and PG14 pins of the STM32F4 board) toggle.
- The toggling operation is done in the callback function. As a side note, this example performs the same operation as in Listing 4.2 but with interrupt usage.

Listing 5.2 Toggle the onboard red and green LEDs by GPIO interrupt, the C cod

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

In order to execute the code in Listing 5.2, create an empty project under STM32CubeIDE and configure the LED pins of the STM32F4 microcontroller as output.

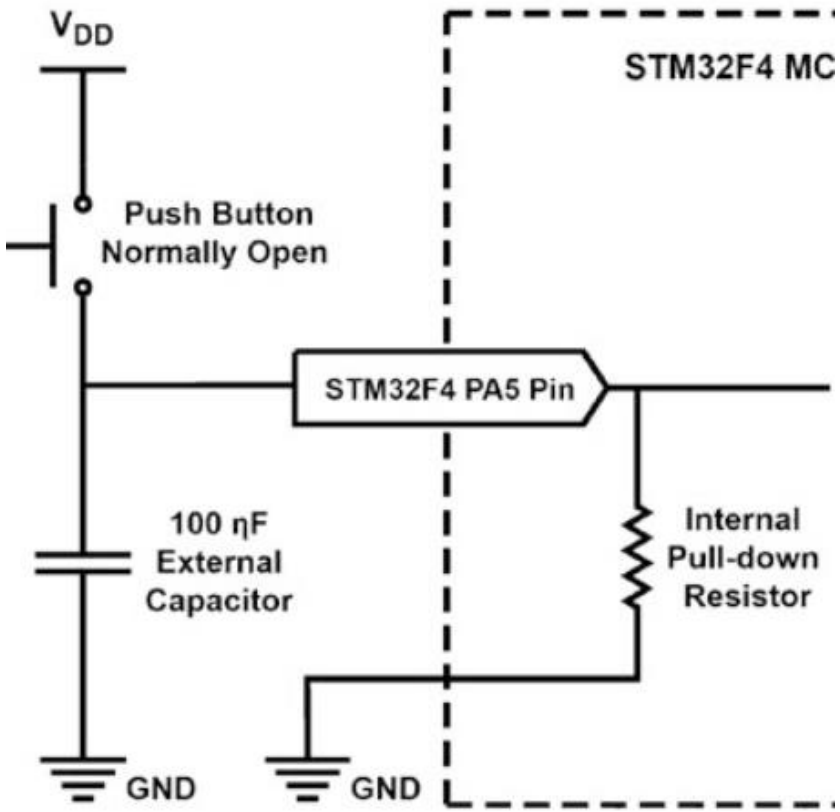
Then, configure the button pin as GPIO_EXTI and select the mode as “external interrupt mode with falling edge trigger detection.”

As the project is compiled and run, the CPU executes the callback function when the button is pressed and then released. In the callback function, the green and red LEDs toggle according to the interrupt source pin.

By adding a breakpoint, the reader can observe that the counter in the infinite loop increases while the CPU is not in handler mode.

The counter in the callback function increases only once the button is pressed and then released.

- The second example aims to familiarize the reader with the nested interrupt and priority mechanisms. To do so, we should connect an external normally open button in active high configuration and a capacitor to the pin PA5 as in Fig. 5.4. Afterward, we should configure the onboard and external button pins as GPIO_EXTI and select their mode as “external interrupt mode with falling edge trigger detection.”
- We should modify the callback function in Listing 5.2 with the new callback functions as in Listing 5.3. We should also change the interrupt priority of EXTI line 0 interrupt to 1 and EXTI line[9:5] interrupt to 2 while configuring interrupt sources. The reader can execute the new code with the modified setup to see the effect of these changes.



Listing 5.3 The effect of priority on GPIO interrupts, the C code

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)

{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_Delay(5000);
}
if (GPIO_Pin == GPIO_PIN_5)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
HAL_Delay(5000);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The second example is formed in two separate parts.
- In the first part, a low priority interrupt comes when the CPU is handling a high priority interrupt.
- Hence, the low priority interrupt is served after the high priority one is served.
- First, press the onboard button on the STM32F4 board. Release the onboard button, wait for 1 s, and press and release the external button.
- You can observe that the second press will not be recognized until four more seconds pass. In the second part, a high priority interrupt comes when the CPU is executing a low priority interrupt.
- Due to interrupt priority, the high priority interrupt is served immediately. Now, press and release the external button and wait for 1 s. Then, press and release the user button on the STM32F4 board.
- This time the second press will be recognized immediately. This example shows how the priority mechanism works.

Listing 5.4 Toggle the onboard green LED using an interrupt, the C++ code

```
#include "mbed.h"

InterruptIn userButton(BUTTON1);
DigitalOut greenLED(LED1);

void getButtons()
{
greenLED = !greenLED;
}

int main()
{
greenLED = 0;
userButton.fall(&getButtons);
__enable_irq();

while (true);
}
```

- As can be seen in Listing 5.4, the CPU waits in an infinite loop formed by the code line while(true).
- When the user presses the button, this loop is interrupted.
- The CPU executes the function ISR_callback.
- Afterward, it turns back and waits in the infinite loop for another interrupt.

Power Management in Embedded Applications

- Power management is closely related to the interrupt usage. Therefore, we will form the link between power management and interrupt usage.
- Embedded systems can be used in stand-alone applications in which they depend on battery. In order to extend working time of the overall system, we have three options.
  - First, we can select a battery with larger capacity.
  - Second, we can design a system with battery recharging capability. These two options are related to the power supply side.
  - The third option is decreasing power consumption of the embedded system.
- Improving the battery capacity and using microcontroller power management options together provide the best possible combination.



Power management in microcontrollers starts by decreasing the power consumption of the device. To do so, all peripheral units should be turned off when not used so that they do not dissipate power. This is done by inhibiting the clock signal sent to them. As a note, all peripheral units are turned off after resetting the microcontroller. It is the responsibility of the programmer to turn on the peripheral unit to be used when needed. Besides, the user should also select the appropriate power mode for the microcontroller.

The microcontroller performs its operations in active (run) mode. However, this mode has the highest power consumption. Therefore, we should let the microcontroller run in an appropriate low power mode whenever possible.

The Link Between Power Management and Interrupt Usage

- One of the most important usage areas of interrupts is letting the CPU in low power modes. This is done as follows.
- If the operation to be performed by the CPU is infrequent, one option is letting the CPU to run continuously in an infinite loop to wait for the event to be responded. Since the event is rare, most of the power will be dissipated while waiting for the event to occur.
- The next option is setting the system such that it stays in an appropriate low power or sleep mode. When the external event occurs, it triggers an interrupt. Afterward, the CPU wakes up, performs the operation related to the event in active mode, and goes back to its previous low power mode again. Hence, the system stays in active mode for a short period of time or when only needed. For the remaining times, the system stays in low power mode and does not dissipate power as it does in the active mode.

Let’s consider an actual example on the usage of interrupts to decrease the overall power consumption in operation.

In our example, we press a button to open a gate. We can safely assume that the button press is rare.

If the interrupt-based setup is not used, the system waits in active mode all the times. Meanwhile, the power consumption will be maximum.

On the other hand, if the button press generates an interrupt, then we can let the system to sleep till the interrupt occurs.

Hence, it dissipates the lowest possible power during this time.

The system will be in active mode only while executing the callback function.

This will lead to the overall power consumption to be minimal.

This scenario indicates the importance of interrupt usage in power management.

Battery as Power Supply

The embedded system is powered by battery most of the times when used in a stand-alone application. Hence, the reader should know fundamental properties of the battery such that the stand-alone application can be realized efficiently.

There are rechargeable and disposable batteries. As the name implies, the rechargeable battery can be used more than once. Although this may seem reasonable, the charging circuit increases the overall cost. Besides, rechargeable batteries are expensive.

As a final note, lifetime of a rechargeable battery is around 1000– 2000 charging. On the other hand, the disposable battery can be used without any charging circuitry. It is cheap. However, the disposable battery can only be used once and should be replaced when depleted.



- Capacity of a battery is provided in milliampere hour (mAh) by the manufacturer. This indicates the current value the battery can provide within 1 h in predefined voltage level.
- The other explanation is as follows. Capacity indicates how many hours can the battery be used when the mentioned current is drawn from it.
- Let’s consider an example to explain this concept. Assume that we have a battery with 2000 mAh capacity. Assume further that we draw 150 mA current during 15s and 2 mA current during 45s from the battery within 1 min.
- We can calculate the time needed for the battery to deplete as follows. Every minute, the average current needed will be 150 × 1/4 + 2 × 3/4 = 39 mA. Therefore, the time needed for the battery to deplete can be found as 2000 mAh / 39 mA = 51.28 h in this usage scenario.

Power Management in the STM32F4 Microcontroller

- The STM32F4 microcontroller has several key features related to power management. These can be summarized as follows.
- The microcontroller allows multiple independent power supply usage. This leads to reducing power consumption while some peripherals are supplied at higher voltages.
- The microcontroller has several power modes. These allow the programmer to benefit from the microcontroller resources while saving power.
- Multiple interrupt sources can be used to wake up the CPU from a low power mode. Hence, different pin and peripheral configurations may be applied.
- For example, the STM32F4 microcontroller can be woken up with user interaction by a button connected to a GPIO pin or periodically using timers. The microcontroller has a battery backup domain. This can be used to keep some peripheral units active (including RTC to be introduced in Chap. 6) and certain backup registers (to be introduced in Chap. 9) without the main power supply. This allows saving the last state of the microcontroller when the main power is lost.

Power Modes

- The STM32F4 microcontroller supports four power modes as run (active), sleep, stop, and standby. These are tabulated in Table 5.2.

Table 5.2 The STM32F4 microcontroller power modes

Power mode	Regulator mode	CPU/SRAM/peripherals active	Max. clock speed (MHz)
Run	Main-scale 1 (OverDrive)	All active	180
	Main-scale 1	All active	168
	Main-scale 2 (OverDrive)	All active	168
	Main-scale 2	All active	144
	Main-scale 3	All active	120
Sleep	Main-scale 1 (OverDrive)	SRAM/peripherals	180
	Main-scale 1	SRAM/peripherals	168
	Main-scale 2 (OverDrive)	SRAM/peripherals	168
	Main-scale 2	SRAM/peripherals	144
	Main-scale 3	SRAM/peripherals	120
Stop	Main-scale 3	SRAM	–
	Main-scale 3 (UnderDrive)	SRAM	–
	Low power-scale 3	SRAM	–
	Low power-scale 3 (UnderDrive)	SRAM	–
Standby	Power-down mode	–	–

- Let’s take a closer look at Table 5.2. As can be seen in this table, our first option for operation is the run mode. This mode does not have any power saving property. Hence, all peripherals and clock sources can be activated within it.
- The user has five options for setting the main CPU clock between 180 and 120 MHz in the run mode based on the regulator used.
- Our second option is the sleep mode. The CPU is stopped in this mode. However, SRAM and all peripherals are active. These can wake up the CPU by an interrupt.
- Our third option is the stop mode. This mode is called deep sleep mode in Arm® literature. The stop mode achieves the lowest power consumption while retaining the contents of SRAM and registers. The CPU can be woken up from the stop mode by any of the EXTI line (the EXTI line source can be one of the 16 external lines, the PVD output, the RTC alarm/wake-up/tamper/time stamp events, the USB OTG FS/HS wakeup, or the Ethernet wakeup).
- Our fourth option is the standby mode. Only the RTC, 4 kB of backup SRAM, and 20 backup registers (80 bytes) are active in this mode. The CPU can exit from the standby mode either by an external reset pin, a watchdog reset, a rising edge on one of the enabled WKUPx pins or when a RTC event occurs. After waking up from standby mode, code execution restarts in the same way as after a reset operation.

STM32CubeMX for Power Usage Analysis

- STM32CubeMX has a power consumption calculator (PCC) tool which can be used to estimate active, inactive, and average current consumption values for a given scenario. The PCC tool also estimates the battery life when the one to be used in operation is selected. The PCC tool can be used by setting up multiple steps such that different power modes, clock source and frequency, and peripheral units can be selected at each step. Hence, the user can check complex usage scenarios with the tool.



Fig. 5.5 The PCC tool under STM32CubeMX

Listing 5.2 Toggle the onboard red and green LEDs by GPIO interrupt, the C cod

```
/* USER CODE BEGIN PV */
int counter1 = 0, counter2 = 0;
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
if (GPIO_Pin == GPIO_PIN_0)
{
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_14);
}
counter2++;
}
/* USER CODE END 0 */

/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOG, GPIO_PIN_14, GPIO_PIN_RESET);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
counter1++;
/* USER CODE END WHILE */
/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- On the left side menu, a *VDD* value and battery can be selected. On the right-hand side of the view, a new step can be added by clicking the “New Step” button. While adding or editing a step, power and memory settings, clocks, peripheral units, and other additional settings, such as duration of a step, can be adjusted. When a new step is added and there is a transition error between power modes, then the step turns to red on the list. Results are shown as graphs in the display section of the tool.



- We next provide an example on the usage of the PCC tool. Within this example, we will calculate the current consumption when the CPU runs for 10 s when the code in Listing 5.2 is executed.
- To do so, open the ".ioc" file in the project explorer window, click on "Tools," tab and add a new step. Here, choose the power mode as "RUN," power scale as "Scale-3-Low," memory fetch type as "Flash," and CPU frequency as "25 MHz." Enter the step duration as 10 s or 10000 ms and click the "Enable IPs from Pinout" button. You may also enable IPs (peripherals) from the peripheral selection menu in the middle.
- Once the configuration is complete, click the "Add" button. You can observe the current consumption profile graph. As can be seen there, approximately 11.41 mA average current is consumed in operation. Now, edit the added step and change the step duration as 0.1 s or 100 ms. Add another step, choose power mode as "STOP_NM (Normal Mode)" and clock configuration as "Regulator_LP," and set the step duration as 9.9 s or 9900 ms. This time, the average current consumption drops down to approximately 401.2 µA. We will use the PCC tool in the following sections to emphasize the usage of power modes in general.

Power Modes in C Language

Sleep Mode

- HAL library functions related to the sleep mode are as follows. The function HAL_PWR_EnterSLEEPMode can be used to enter the sleep mode. The function HAL_PWR_EnableSleepOnExit enables sleep-on-exit when returning from handler to thread mode. Likewise, the function HAL_PWR_DisableSleepOnExit disables sleep-on-exit when returning from the handler to thread mode.

```
void HAL_PWR_EnterSLEEPMode (uint32_t Regulator, uint8_t
    SLEEPEntry)
/*
Regulator: Specifies the regulator state. This parameter is not
    used for the STM32F4 family. Hence, we should set it as
    PWR_MAINREGULATOR_ON.
SLEEPEntry: Specifies if the sleep mode is entered with WFI or
    WFE instruction. This parameter can be one of
    PWR_SLEEPENTRY_WFI or PWR_SLEEPENTRY_WFE
*/

void HAL_PWR_EnableSleepOnExit (void)

void HAL_PWR_DisableSleepOnExit (void)
```

- We next provide an example on the usage of sleep mode in C language. To do so, we repeat the example given in Listing 5.2 to toggle onboard LEDs using GPIO interrupts. However, instead of actively waiting in an infinite loop when the button is not pressed, this time we let the CPU wait in sleep mode. To do so, the reader should create an STM32CubeIDE project first. Then, the infinite loop in Listing 5.2 should be replaced by the code block in Listing 5.7. As the project is executed, the reader can observe that both counters in the infinite loop and callback function do not increase when the button is not pressed.

Listing 5.7 A sample code for the usage of sleep mode functions, in C language

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_SuspendTick();
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
HAL_ResumeTick();

counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- The reader can observe that there are two extra functions used in Listing 5.7 as *HAL_SuspendTick()* and *HAL_ResumeTick().*
- These are related to the **SysTick** and clock setup to be explained in detail in Chap. 6. However, we had to cover them here since these two modules work in sleep mode.
- SysTick automatically generates interrupts at every millisecond and wakes up the CPU. To avoid this, we had to disable it before the sleep operation and enable it afterward as in Listing 5.7.
- We can analyze the power consumption of the modified project by the PCC tool. Here, the GPIOA and GPIOG ports are enabled. The power scale is set to “Scale-3- Low,” memory fetch type to “Flash,” and CPU frequency to “25 MHz” for both run and sleep modes. Under these conditions, the average current consumption will be 11.1 mA and 3.1 mA for the run and sleep modes, respectively.

```
void HAL_PWR_EnterSTOPMode (uint32_t Regulator, uint8_t
    STOPEntry)
 /*
Regulator: Specifies the regulator state. This parameter is not
    used for the STM32F4 family. Hence, we should set it as
    PWR_MAINREGULATOR_ON.
 STOPEntry: Specifies if stop mode is entered with WFI or WFE
    instruction. This parameter can be one of PWR_STOPENTRY_WFI
    or PWR_STOPENTRY_WFE.
 */

 HAL_StatusTypeDef HAL_PWREx_EnterUnderDriveSTOPMode (uint32_t
    Regulator, uint8_t STOPEntry)
```

Stop Mode

- HAL library functions related to the stop mode are as follows. The function HAL_PWR_EnterSTOPMode can be used to enter the stop mode 0 or 1 by specifying the regulator state. Likewise, the function HAL_PWREx_EnterUnder DriveSTOPMode leads the microcontroller to the underdrive mode with low power consumption.

Listing 5.8 A sample code for the usage of stop mode functions, in C language

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
HAL_SuspendTick();
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI
    );
HAL_ResumeTick();

counter1++;
/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

- We will next consider usage of the stop mode by repeating the example in Listing 5.2. This time we let the CPU wait in stop mode instead of actively waiting in an infinite loop when the user button is not pressed. As in the previous example, the reader should first create a project. Then, he or she should use the code block in Listing 5.8 to replace the infinite loop in Listing 5.2 and run the project. The reader can observe that the counter in the infinite loop does not increase when the button is not pressed.
- To note here, the functions HAL_SuspendTick() and HAL_ResumeTick() are used to stop and start the SysTick, respectively. They are added to avoid starting the SysTick during the debug session. They may not be needed if the project is compiled and run without debugging it.

```
void HAL_PWR_EnterSTANDBYMode (void)

void HAL_PWR_EnableWakeUpPin (uint32_t WakeUpPinx)
/*
WakeUpPinx: Specifies which Wake-Up pin to enable. This
    parameter can only be PWR_WAKEUP_PIN1.
*/

__HAL_PWR_CLEAR_FLAG (_FLAG_)
/*
__FLAG__: specifies the flag to clear. This parameter can be one
     of the PWR_FLAG_WU or PWR_FLAG_SB
*/
```

Standby Mode

- HAL library functions related to the standby mode are as follows. The function HAL_PWR_EnterSTANDBYMode can be used to enter the standby mode. The function HAL_PWR_EnableWakeUpPin can be used to enable the wake-up PINx functionality. The macro __HAL_PWR_CLEAR_FLAG should be used to clear the flag if it will be used in the wake-up process for the standby mode. We provide detailed information on these functions next.

- We next provide an example on the usage of standby mode. Here we wake up the CPU with the onboard button connected to PA0, toggle the onboard LEDs connected to pins PG13 and PG14, and then put CPU back to standby mode. To do so, the reader should first create an empty project and configure the LED pins of the STM32F4 microcontroller as output. Then, he or she should configure the button pin as SYS_WKUP and select the mode as “external interrupt mode with falling edge trigger detection.” Then, the reader should use the code block in Listing 5.9 and run the project.

- As the project is executed, the green and red LEDs turn on one after the other with 1 s delay. Then, they turn off together. However, we do not turn off the LEDs in code. Instead, they turn off since the GPIO ports connected to them are closed. As the CPU turns on after the standby mode, it resets itself. Hence, the LEDs turn on and off again.

Listing 5.9 A sample code for the usage of standby mode functions, in

```
/* USER CODE BEGIN 2 */
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_13);
HAL_Delay(1000);
HAL_GPIO_TogglePin(GPIOG,GPIO_PIN_14);
HAL_Delay(1000);

__HAL_PWR_CLEAR_FLAG(PWR_FLAG_WU);
HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);
HAL_PWR_EnterSTANDBYMode();
/* USER CODE END 2 */
```

Lab4 solution

```
#include <stdint.h>

int *memloc;

int main(void)
{
    memloc = (int *)0x20000040;
    unsigned char temp = 0xF0;


    for(int i=0; i<4; i++){
        *memloc = 0x00000000;
        for(int j=0;j<4;j++){
                *memloc = *memloc<<8;
                *memloc += temp;
                temp += 0x01;
        }
        memloc++;
    }

    memloc = (int *)0x20000040;

    for(int i=0; i<4; i++){
        for(int j=0;j<4;j++){
                if(*memloc & 0x01){
                        *memloc = *memloc>>8;
                        *memloc &= 0x00FFFFFF;
                }
                else{
                        *memloc = *memloc>>8;
                        *memloc |= 0xFF000000;
                }
        }
        memloc++;
       }

        for(;;);
}
```